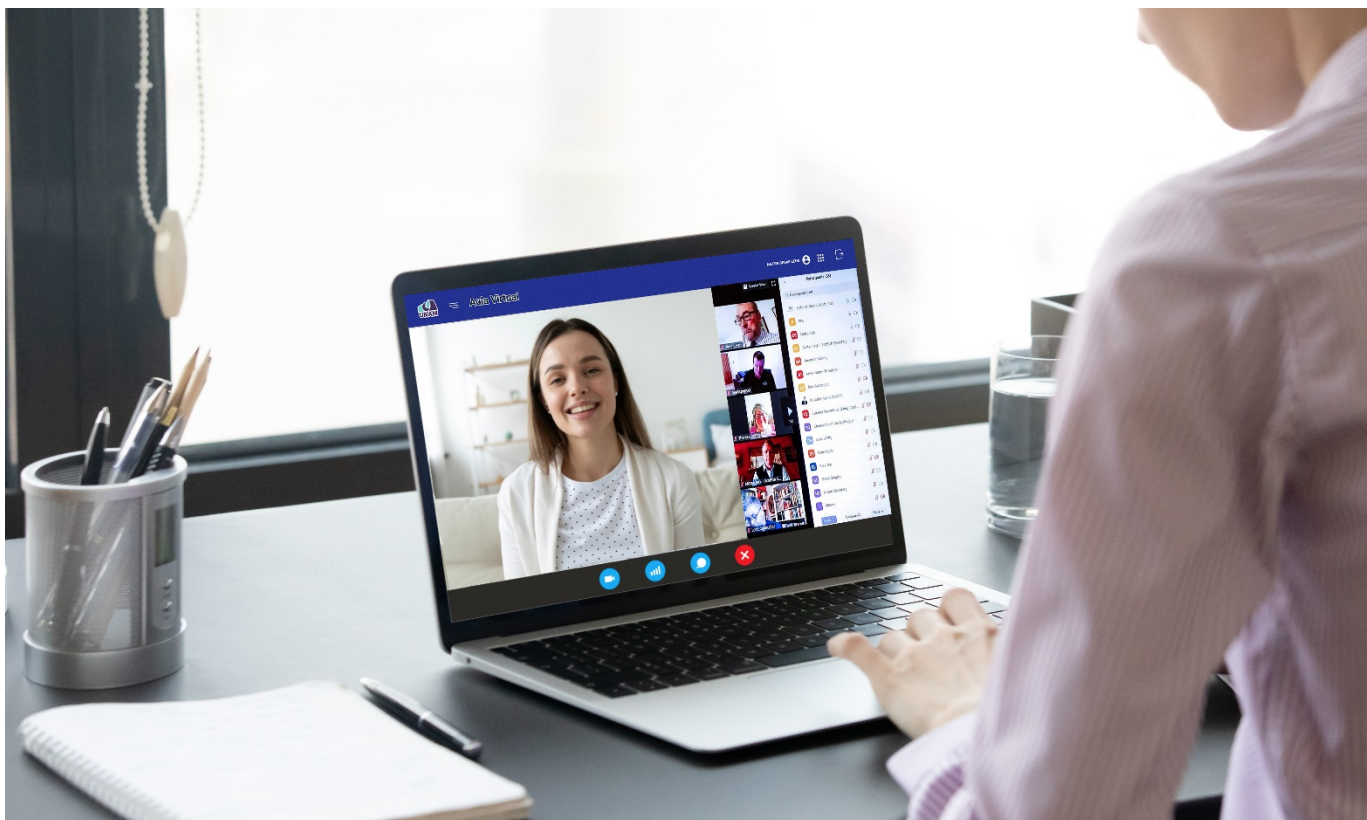




Tema:

GUIA DE ENSEÑANZA:

PROGRAMACION DE VIDEO JUEGOS I

Guía de Desarrollo

- **MODELO MDA**
- **MANUAL UNITY**
- **Desarrollo Juego 2D MarioBros ALL-STARs**

Ilo – Perú

Introducción

Todos los dispositivos se crean dentro de alguna metodología de diseño. Si haces un prototipo físico, la arquitectura de una interfaz de software, construyes un argumento o realizas una serie de experimentos controlados; las metodologías de diseño guían el proceso de pensamiento creativo y ayudan a garantizar un trabajo de calidad.

En concreto, los análisis iterativos, cualitativos y cuantitativos apoyan al diseñador de dos maneras importantes. Le ayudan a analizar el resultado final para mejorar la implementación. Le facilitan analizar la implementación para perfeccionar el resultado. Al abordar la tarea desde ambas perspectivas, se pueden considerar una amplia gama de posibilidades e interdependencias

Esto es especialmente importante cuando se trabaja con el ordenador y los videojuegos, donde la interacción entre los subsistemas codificados crea dinámicas complejas y dinámicas (y a menudo impredecibles) comportamientos. Los diseñadores y los investigadores deben considerar cuidadosamente las interdependencias antes de realizar cambios en el sistema, y los entendidos deben reconocerlas antes de llegar a conclusiones sobre la naturaleza de la experiencia generada

el marco MDA como una herramienta para ayudar a diseñadores, investigadores y estudiosos a realizar este recorrido.

1. MDA (Mechanics, Dynamics y Aesthetic)

Los juegos son creados por equipos de diseñadores y desarrolladores, y consumidos por los jugadores. Se compran, usan y eventualmente se desechan como la mayoría de los demás bienes de consumo.



La producción y el consumo de los artefactos* de juego.

La diferencia entre los juegos y otros productos de entretenimiento (tales como libros, música, películas y obras de teatro) es que su consumo es relativamente impredecible. La cadena de eventos que se producen durante el “gameplay” y las consecuencias de éstos se desconocen en el momento en que el producto está terminado.

El marco MDA formaliza el consumo de los juegos dividiéndolos en distintas partes



... Y estableciendo sus contrapartidas de diseño:



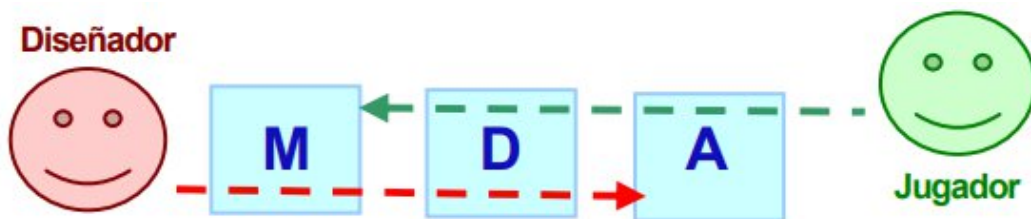
- **Las mecánicas** describen los componentes particulares del juego, al nivel de la representación de datos y algoritmos.
- **Las dinámicas** describen el comportamiento durante el tiempo de juego de las mecánicas que actúan sobre los “inputs” de los jugadores y sobre cada uno de los demás outputs que se generan en el tiempo de juego
- **La estética describe** las reacciones emocionales que se desean evocar en el jugador, cuando interactúa con el sistema de juego

Es fundamental para este marco la idea de que los juegos son más artefactos/dispositivos que medios de comunicación. Con esto queremos decir que la materia o esencia de un juego es el comportamiento y no la comunicación que fluye del juego hacia el jugador. Pensar en los juegos como dispositivos diseñados ayuda a enmarcarlo como sistemas que construyen el

comportamiento a través de la interacción. Se apoya claramente en opciones de diseño y análisis en todos los niveles de estudio y desarrollo.

2. MDA en detalle

Cada elemento del marco de MDA puede pensarse como una "lente" o una “perspectiva” del juego - separadas, pero causalmente vinculadas. [LeBlanc, 2004b]. Desde la perspectiva del diseñador, las mecánicas dan lugar al comportamiento dinámico en el sistema, que a su vez lleva a experiencias estéticas determinadas. Desde la perspectiva del jugador, las estéticas marcan la pauta, que nace de dinámicas visibles y observables y, finalmente de mecánicas usables.



El diseñador y el jugador tienen una perspectiva diferente.

Cuando se trabaja con los juegos, es útil tener en cuenta tanto la perspectiva del diseñador como la del jugador. Nos ayuda a observar cómo incluso pequeños cambios en una capa pueden conectar en cascada en otros. Además, pensar en el jugador fomenta impulsar la experiencia de juego (a diferencia de impulsar un simple rasgo del diseño).

Como tal, comenzamos nuestra investigación con una argumentación sobre estéticas, continuando hasta las dinámicas, y terminando con las mecánicas subyacente.

3. Estéticas/Experiencia de Juego

¿Qué hace a un juego divertido? ¿Cómo podemos reconocer un tipo de diversión cuando la vemos? Hablar de jugar y de juegos es difícil porque el vocabulario que se usa es relativamente limitado.

Al describir la estética de un juego, queremos alejarnos de palabras como "diversión" y "juego" hacia un vocabulario más orientado. Esto incluye la siguiente clasificación sin ánimo de limitarse a la misma:

1. Sensación

El Juego como placer de los sentidos

2. Fantasía

El Juego como imaginación

3. Narrativa

El Juego como una historia

4. Desafío/Reto

El Juego como carrera de obstáculos

5. Comunidad

El Juego como marco social

6. Descubrimiento

Como un territorio desconocido

7. Expresión

El Juego como autodescubrimiento

8. Sumisión

El Juego como pasatiempo

Por ejemplo, consideremos los juegos Charadas*, Quake, Los Sims y Final Fantasy. Si bien cada uno son "divertidos" a su manera, es mucho más descriptivo considerar los atractivos que generan sus respectivas experiencias de juego.

- **Charadas** : Comunidad, Expresión, Desafío/Reto.
- **Quake**: Desafío, Sensación, Competición, Fantasía.
- **Los Sims**: Descubrimiento, Fantasía, Expresión, Narrativa.
- **Final Fantasy**: Fantasía, Narrativa, Expresión, Descubrimiento, Desafío/Reto, Sumisión.
- **Dota 2**: Competición, Estrategia, Comunidad, Desafío/Reto.
- **Counter Strike**: Competición, Precisión, Tensión, Estrategia.
- **StarCraft**: Estrategia, Competición, Desafío/Reto, Toma de decisiones.

4. Modelos de Estéticas

Usando el vocabulario estético como una brújula, podemos definir modelos para el "gameplay". Estos modelos nos ayudan a describir las dinámicas de juego y las mecánicas

Por ejemplo: Charadas y Quake son ambos competitivos. Funcionan cuando los distintos equipos o jugadores de estos juegos ganan en detrimento de otros. Esto implica que los jugadores tienen adversarios (en Charadas, los equipos compiten, en Quake, el jugador compete contra oponentes de la computadora**) y que todos los equipos quieren ganar.

Es fácil ver que el soporte de confrontación del juego y una retroalimentación clara acerca de quién está ganando son esenciales para los juegos competitivos. Si el jugador no ve una

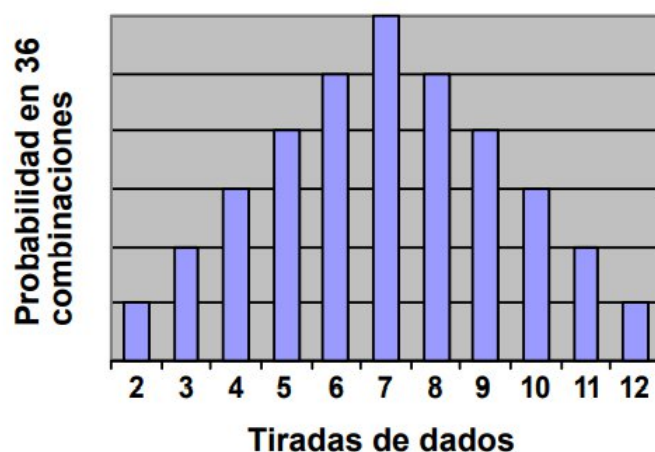
condición ganadora clara, o siente que no tiene posibilidad de ganar, el juego es de repente mucho menos interesante

5. Modelos de Dinámicas

Las Dinámicas trabajan para crear experiencias estéticas. Por ejemplo, el reto es creado por cosas como la presión del tiempo y el juego del rival. La comunidad puede motivarse a través del intercambio de información de ciertos miembros de una sesión (como de un equipo) u ofreciendo victorias que sean difíciles de lograr por uno mismo (por ejemplo, la captura de una base enemiga).

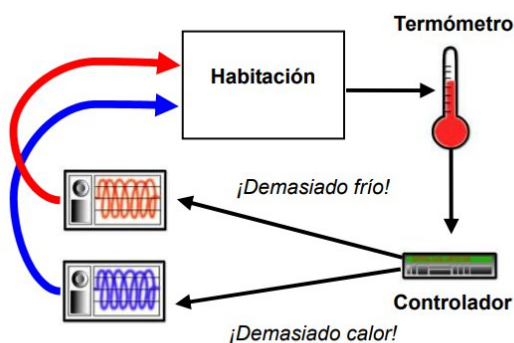
La expresión proviene de dinámicas que fomentan a usuarios individuales a dejar su huella: los sistemas de compra, edificar o ganar elementos para hacerlo, diseñar, construir y cambiar niveles o mundos, y por la creación de personajes únicos, personalizados. La tensión dramática (Narrativa) proviene de las dinámicas que generen un aumento de la tensión, liberar una parte de la historia, y el desenlace.

Al igual que con las estéticas, queremos que nuestro análisis de las dinámicas sea lo más concreto posible. Para desarrollar modelos que puedan predecir y describir las dinámicas en el juego, podemos evitar algunos errores comunes de diseño.



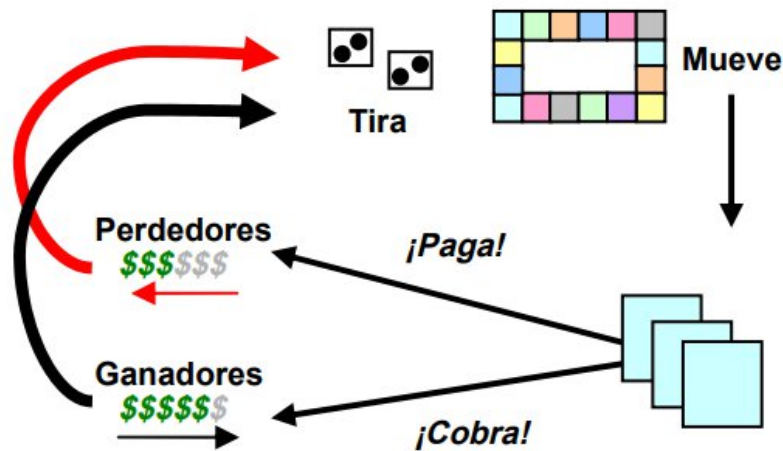
Distribución probabilística de la variable aleatoria 2D6.

Por ejemplo, el modelo de 2 dados de seis caras nos ayudará a determinar el tiempo promedio que necesita un jugador para avanzar en el tablero del Monopoly, dada la probabilidad de varias tiradas.



Del mismo modo, podemos identificar los sistemas de retroalimentación dentro del juego para determinar como los estados particulares o sus cambios afectan al estado global del juego. En

el Monopoly, como el líder o líderes se vuelven cada vez más ricos, pueden penalizar a los jugadores con el aumento de la eficacia necesaria*. Los jugadores pobres se vuelven cada vez más pobres



El sistema de retroalimentación en el Monopoly.

A medida que la brecha se ensancha, sólo unos pocos (y, a veces sólo uno) de los jugadores realmente se benefician. La tensión dramática y la acción se pierden.

Usando nuestra manera de entender la estética y la dinámica, se pueden imaginar maneras de solucionar el Monopoly - ya sea gratificar a los jugadores que están detrás para mantenerlos a una distancia razonable de los líderes, o hacer más difícil el progreso para los jugadores ricos. Por supuesto - esto podría afectar a la capacidad del juego de recrear la realidad de las prácticas de un monopolio – pero la realidad no siempre es "divertida".

6. Mecánicas

Las mecánicas son las distintas acciones, comportamientos y mecanismos de control ofrecidos al jugador dentro de un contexto de juego. Junto con el contenido del juego (niveles, bienes, y demás) las mecánicas sostienen las dinámicas integrales del juego.

Por ejemplo, las mecánicas de los juegos de cartas incluyen arrastrar, faroles y apuestas - a partir del cual las dinámicas emergen como un faro. Las mecánicas de los juegos tipo "shooter" incluyen armas, municiones y puntos de regeneración - las cuales a veces producen cosas como campamentos y francotiradores. En los juegos de golf las mecánicas incluyen bolas, clubes, bunker de arena y lagos de agua que a veces producen accidentes o palos ahogados.

El ajuste de las mecánicas de un juego nos ayuda a afinar las dinámicas generales del mismo. Considere nuestro ejemplo del Monopoly. Las mecánicas que ayudarían a los jugadores rezagados podrían incluir bonos o "subvenciones" para los jugadores pobres, o sanciones o "impuestos" para los jugadores ricos - tal vez calculados al cruzar la casilla de salida, salir de la cárcel, o tener monopolios sobre un determinado umbral de valor. Mediante la aplicación de dichos cambios a las reglas fundamentales del juego, podríamos hacer más competitivos a los jugadores más rezagados y ser capaces de mantener el juego durante más tiempo.

Otra solución a la falta de tensión a lo largo del juego del Monopoly sería añadir mecánicas que fomenten la presión del tiempo y la velocidad del juego. Quizás por la pérdida de recursos con

el paso del tiempo con un impuesto de tasa constante (por lo que las personas gastarían rápidamente), duplicando todos los pagos en monopolios (por lo que los jugadores se diferencian rápidamente), o distribuir de forma aleatoria todas las propiedades bajo un umbral de precio determinado.

7. Ajustes. Puesta a punto

Claramente, el último paso de nuestro análisis del Monopoly implica testear el juego y ajustarlo. Al afinar iterativamente el valor de las sanciones, tipo de gravamen o umbrales para las recompensas y castigos, podemos ajustar el juego del Monopoly hasta equilibrarlo.

A la hora de ajustar, nuestro vocabulario y modelos de estética nos ayudan a articular los objetivos de diseño, analizar defectos del juego, y medir nuestro progreso a medida que ajustamos. Si nuestros impuestos del Monopoly requieren de cálculos complejos, podemos estar frustrando el sentido de la inversión del jugador haciendo más difícil para ellos realizar un seguimiento de su efectivo, y, por lo tanto, de su progreso general o su posición competitiva

Del mismo modo, nuestros modelos de dinámicas nos ayudan a determinar de dónde pueden estar viniendo los problemas. Usando el modelo D6, podemos evaluar propuestas de cambio en el tamaño del tablero o su diseño, y determinar cómo las alteraciones podrán extender o acortar el tiempo de juego.

8. MDA en acción

Ahora, vamos a considerar el desarrollo o la mejora de la IA (Inteligencia Artificial) como parte de un juego. A menudo es tentador idealizar la unidad de AI como una caja negra de mecanismos que, en teoría, puede introducirse en una variedad de diferentes proyectos con relativa facilidad. Pero como el marco sugiere, las partes del juego no pueden ser evaluadas en vacío, aparte de su influencia sobre el comportamiento del sistema y la experiencia del jugador.

Primer paso:

Considere como ejemplo el juego de cuidado de niños [Hunicke, 2004]. Su supervisor ha decidido que sería beneficioso prototipar un juego de AI basado en el sencillo juego del pillar la pilla. Tu jugador será una canguro que debe encontrar y poner a un bebé a dormir. La demo estará diseñada para resaltar emociones simples de personajes (como un bebé), para juegos dirigidos a niños entre 3-7 años de edad.

¿Cuáles son los focos estéticos para este diseño? La Exploración y el Descubrimiento son probablemente más importante que el Desafío. Como tal, las dinámicas aquí presentes no se optimizan para “ganar” o “competir” sino para conseguir que el bebé exprese emociones como la sorpresa, el miedo y la expectación.

Los escondites podrían ser “pillados” de forma manual, los recorridos entre ellos no serían modificables; la mayor parte de la lógica del juego estaría dedicada a maniobrar el bebé a la vista y la creación de reacciones de tipo bebé. Las mecánicas de juego incluirían hablar con el bebé ("Te veo!" o "Boo!"), perseguir al bebé (con un avatar o con un ratón), esconderse, pillar y más como éstas.

Segundo paso

Ahora, consideremos una variante de este mismo diseño – realizado para funcionar con una

franquicia como "Rugrats" de Nickelodeon y dirigido a chicas de 7-12 años de edad. Estéticamente, el juego debe percibirse más difícil - quizás haya algún tipo de narrativa involucrada (que el juego tenga varios "niveles", cada uno de los cuales presenta una nueva pieza de la historia y tareas relacionadas).

En cuanto a las dinámicas, el jugador ahora puede rastrear e interactuar con varios personajes a la vez. Podemos añadir la mecánica de presión del tiempo (es decir, llevar a todos a la cama antes de las 9 horas), incluir un "Factor de desorden" o monitor de emociones de los personajes (pañales sucios, provocar el llanto, el llanto que hace perder puntos) y otras parecidas.

Para este diseño, las rutas estáticas ya no son suficiente - y es probablemente una buena idea dejar elegir sus propios escondites. ¿Cada bebé tendrá características, habilidades o desafíos propios? Si es así, ¿cómo se van a mostrar al jugador? ¿Cómo van a realizar un seguimiento de su estado interno, entender su mundo, a otros bebés, y al propio jugador? ¿Qué tipos de tareas y acciones deberá llevar a cabo el jugador?

Tercer paso

Por último, podemos concebir este mismo juego del pilla pilla como un juego de simulación militar estratégica hecho y derecho – a modo de agentes infiltrados o ladrones. Nuestro público objetivo es ahora de hombres de 14-35 años de edad.

Los focos estéticos ahora se amplian para incluir elementos de fantasía (Juegos de rol sobre una caza al espía en un entorno militar o un pícaro buscando un botín) y el desafío probablemente raye con la sumisión. Además de involucrarse en una trama llena de intriga y suspense, el jugador esperará una actividad coordinada por parte de los oponentes - pero probablemente con una expresión emocional mucho menos marcada. En todo caso, los agentes deben expresar miedo y disgusto ante el indicio de su presencia

Las dinámicas podrían incluir la posibilidad de ganar o poder comprar poderosas armas y equipos de espionaje, y desarrollar tácticas y técnicas para el movimiento furtivo, hacer engaños, evasión y escape. Las mecánicas incluyen tecnología expansiva, árboles de habilidades y una amplia variedad de tipos de unidades enemigas y niveles o zonas con rangos variables de movilidad, visibilidad y campo de visión, y otras similares.

Los agentes en este espacio, además de la coordinación de movimientos y los ataques deben operar sobre amplio rango de datos sensoriales. El razonamiento acerca de la posición y la intención del jugador debe transmitir desafío, pero en general contribuir a su éxito. ¿Los enemigos serán capaces de superar obstáculos y adentrarse por terrenos difíciles, o van a hacer trampa? ¿Será la propagación del sonido "realista" o bastarán métricas simples basadas en la distancia?

Para finalizar

Aquí vemos que cambios sencillos en los requisitos estéticos de un juego introducirán cambios de mecánicas en muchos niveles para su AI - a veces requiriendo el desarrollo de sistemas completamente nuevos para la navegación, razonamiento y la resolución de problemas estratégicos.

Por el contrario, vemos que no existe "la mecánica de la Inteligencia Artificial" como tal - la inteligencia o la coherencia proviene de la interacción de la lógica de AI con la lógica de juego. Utilizando el marco MDA, podemos razonar de forma explícita sobre los objetivos estéticos, poner en marcha las dinámicas que apoyan esos objetivos, y después, la amplitud de la gama de nuestras mecánicas en consecuencia

9. Conclusiones

El marco MDA apoya un enfoque iterativo formal para diseñar y ajustar. Se nos permite razonar de forma explícita acerca de objetivos de diseño específicos y anticipar cómo los cambios afectarán a cada uno de los aspectos de la estructura y las implementaciones/diseños resultantes

Al moverse entre tres niveles de abstracción de la MDA, podemos conceptualizar el comportamiento dinámico de los sistemas de juego. La comprensión de los juegos como sistemas dinámicos nos ayuda a desarrollar técnicas para el diseño y mejora iterativa - lo que nos permite controlar resultados no buscados, y ajustar comportamientos deseados.

Además, mediante la comprensión de cómo las decisiones formales sobre el juego afectan a la experiencia del usuario, somos capaces de descomponer mejor esta experiencia, y usarlo para generar nuevos diseños, investigaciones y críticas, respectivamente.

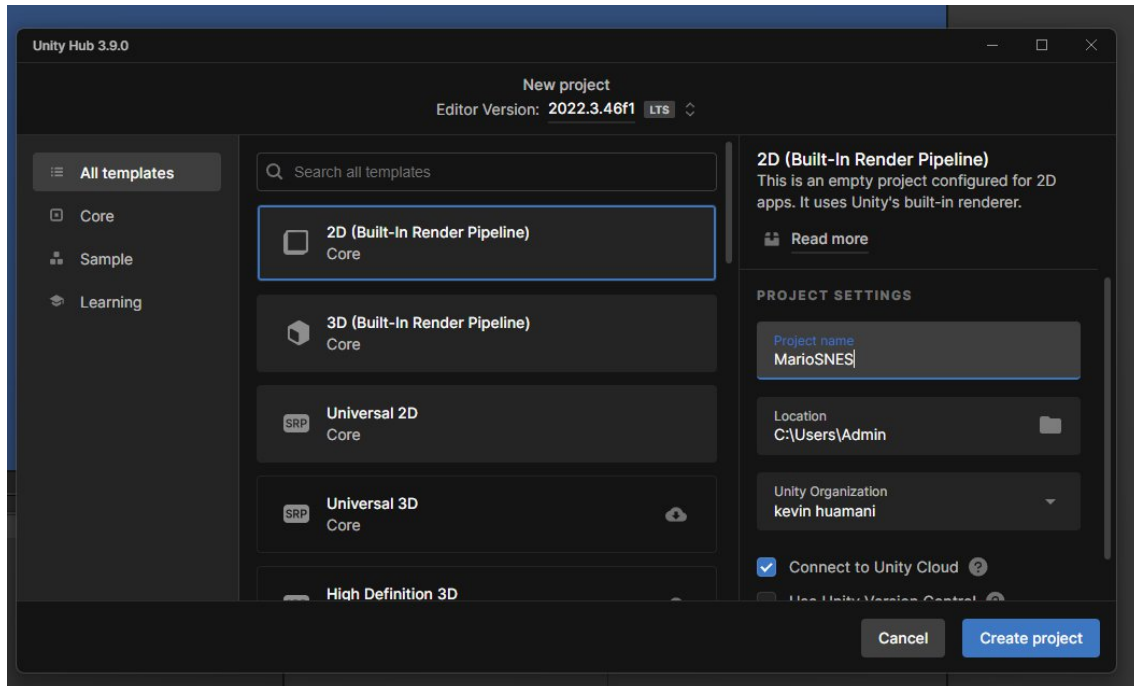
DESARROLLO EN UNITY

Este proyecto práctico recreara los escenarios, mecánicas y lógica del juego MarioBros uno de los juegos emblemas de NINTENDO una de las compañías más grande de videojuegos

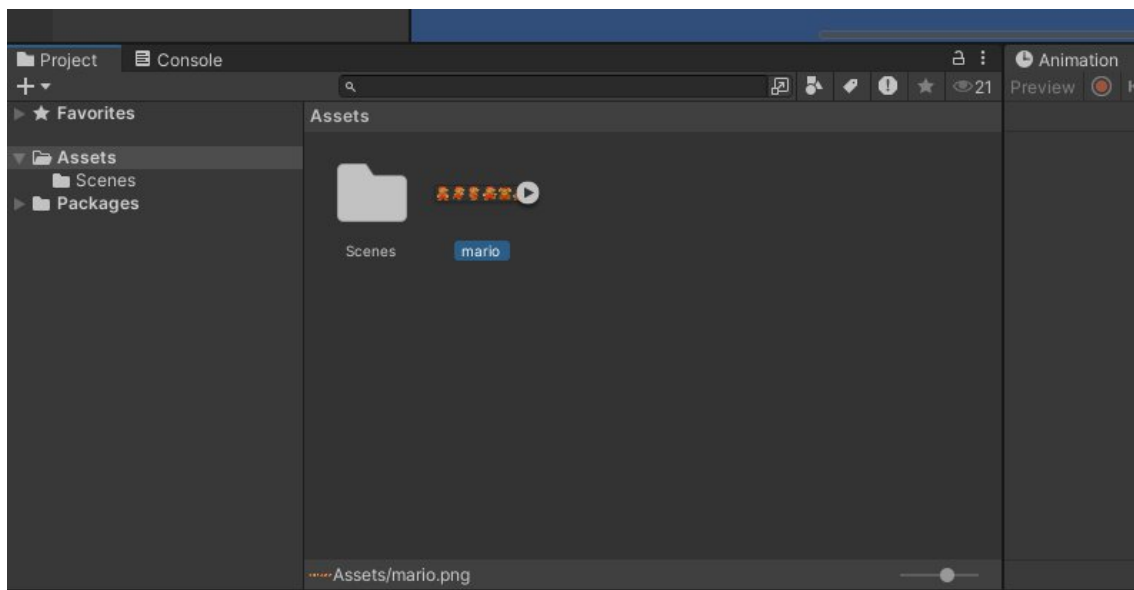
1. Crea un nuevo Proyecto 2D en UNITY

Descarga los recursos que usaremos Aquí

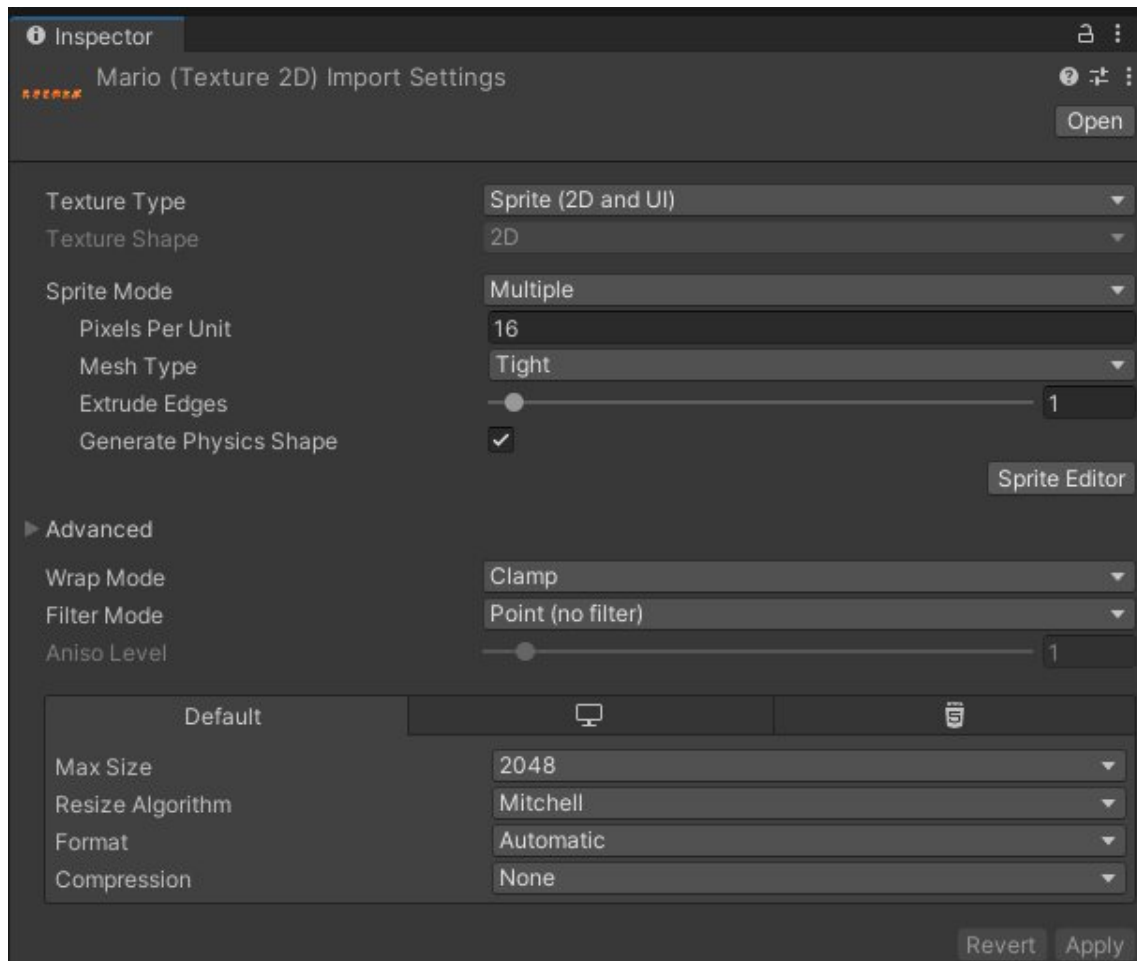
<https://www.mediafire.com/file/7oq3jxecwn7dkkb/MarioBros.zip/file>



Agrega el Sprite de Mario a los recursos de tu proyecto y selecciónalos como Multiple, Point (no filter) y formato de compresión a None



Ajustar los pixeles por Unidad a 16

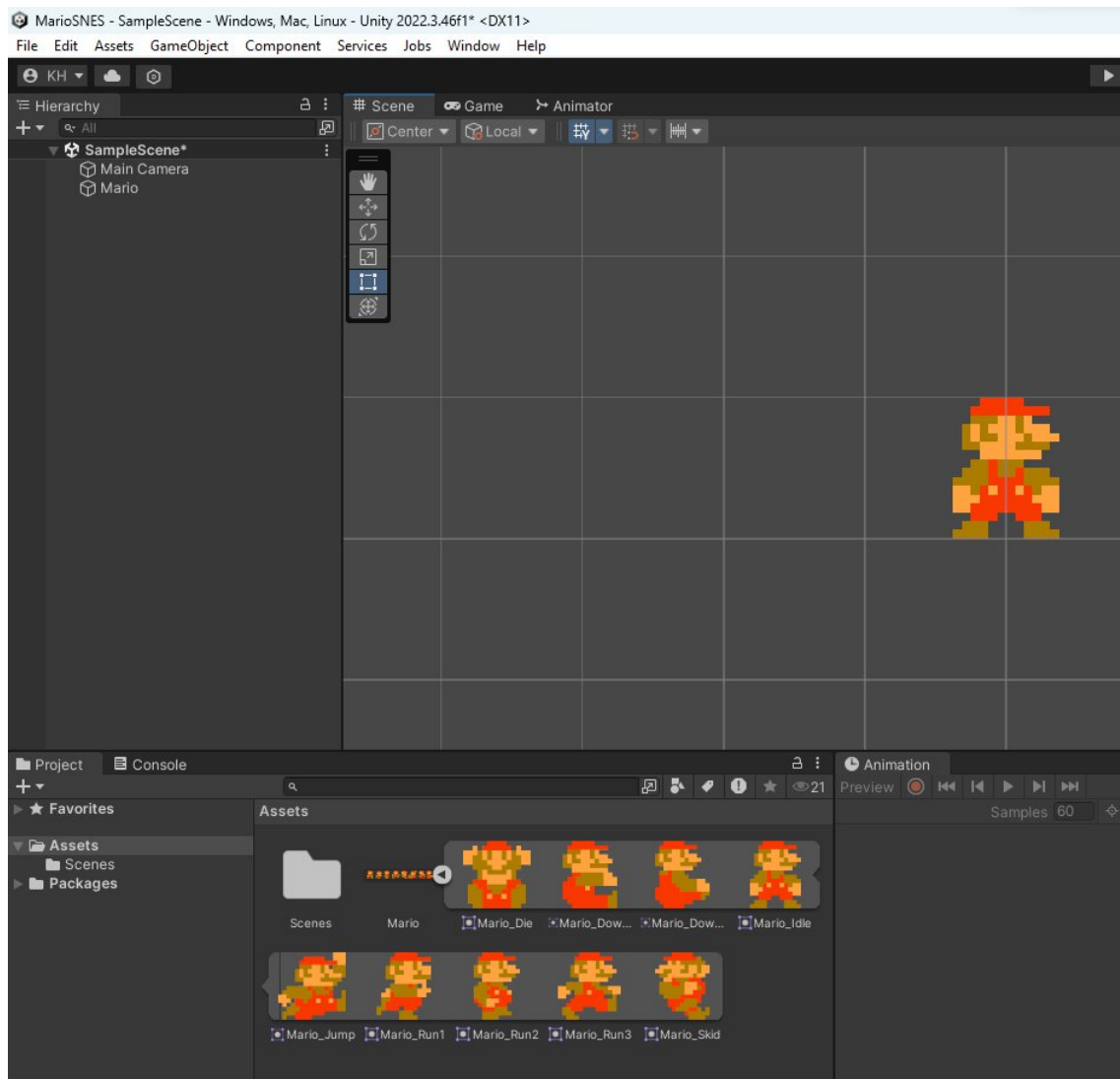


En el editor de Sprite seleccionar Slice>Slice, Pivot> Bottom, Aplicar y cerrar

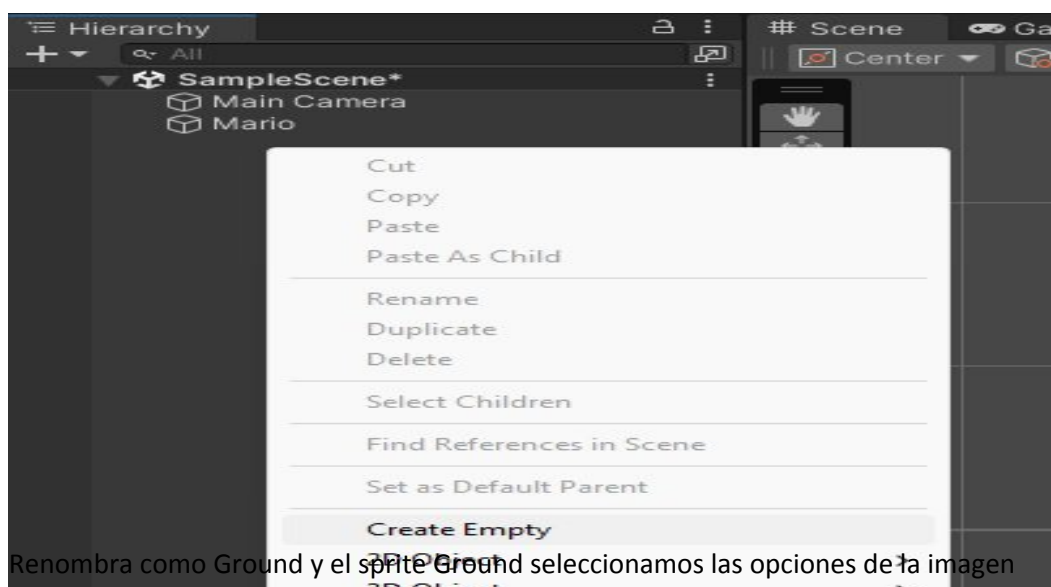


Renombra cada Sprite como **Mario_Idle**, **Mario_Run1 -3**, **Mario_Skid**, **Mario_Jump**, **Mario_DownPole1-2** y **Mario_Die** Para poder identificarlos mejor a futuro tendrán esos nombres

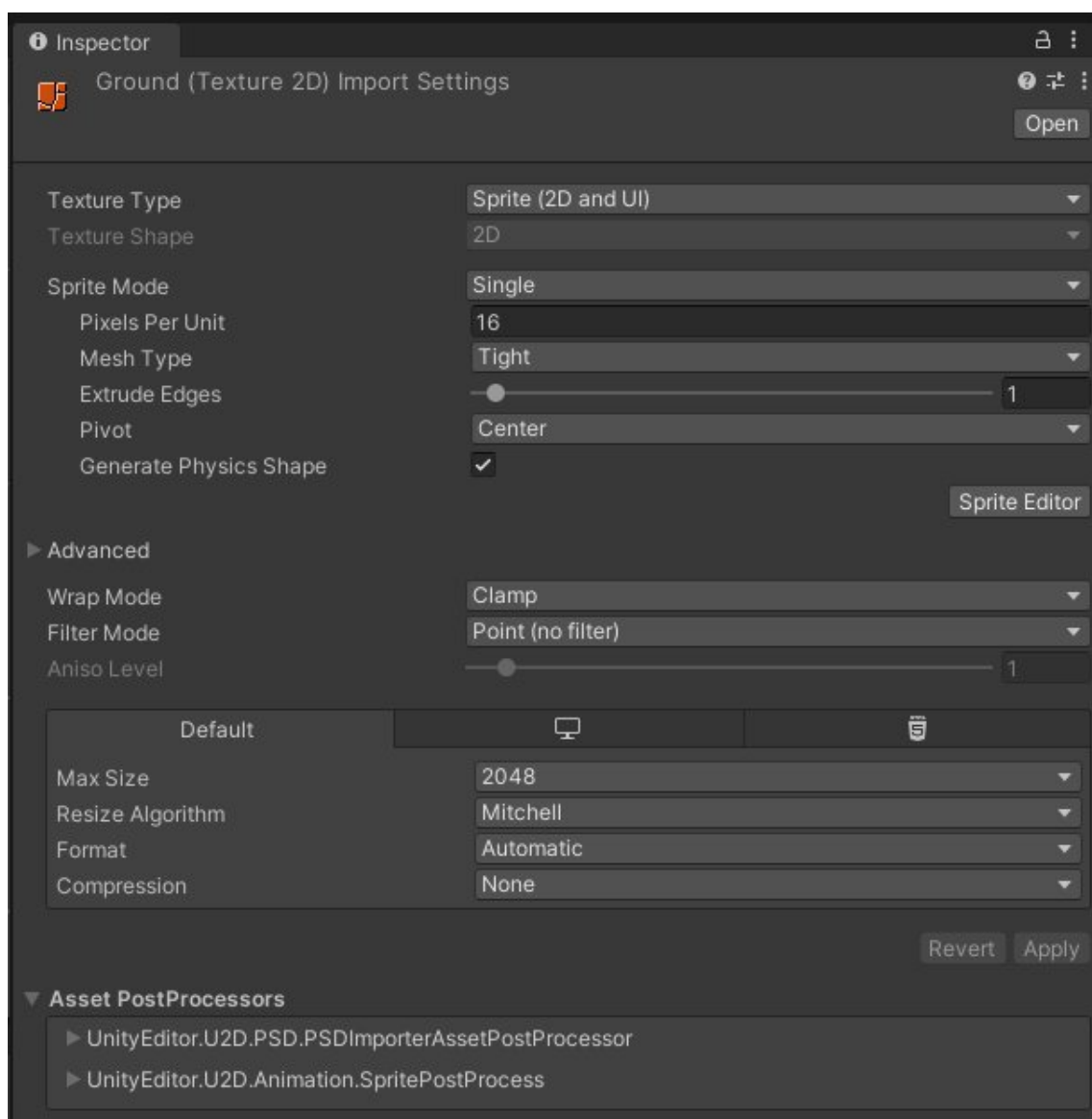
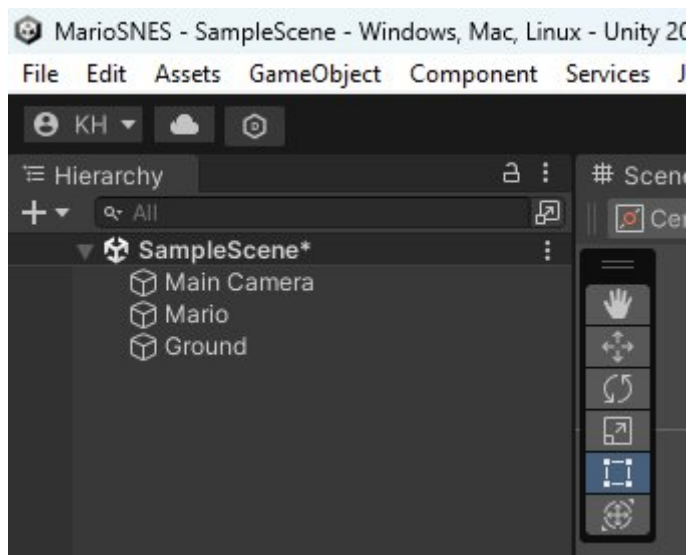
Agrega el Sprite Mario_Idle a la jerarquía de unity y luego renombramos nuestro GameObject como Mario



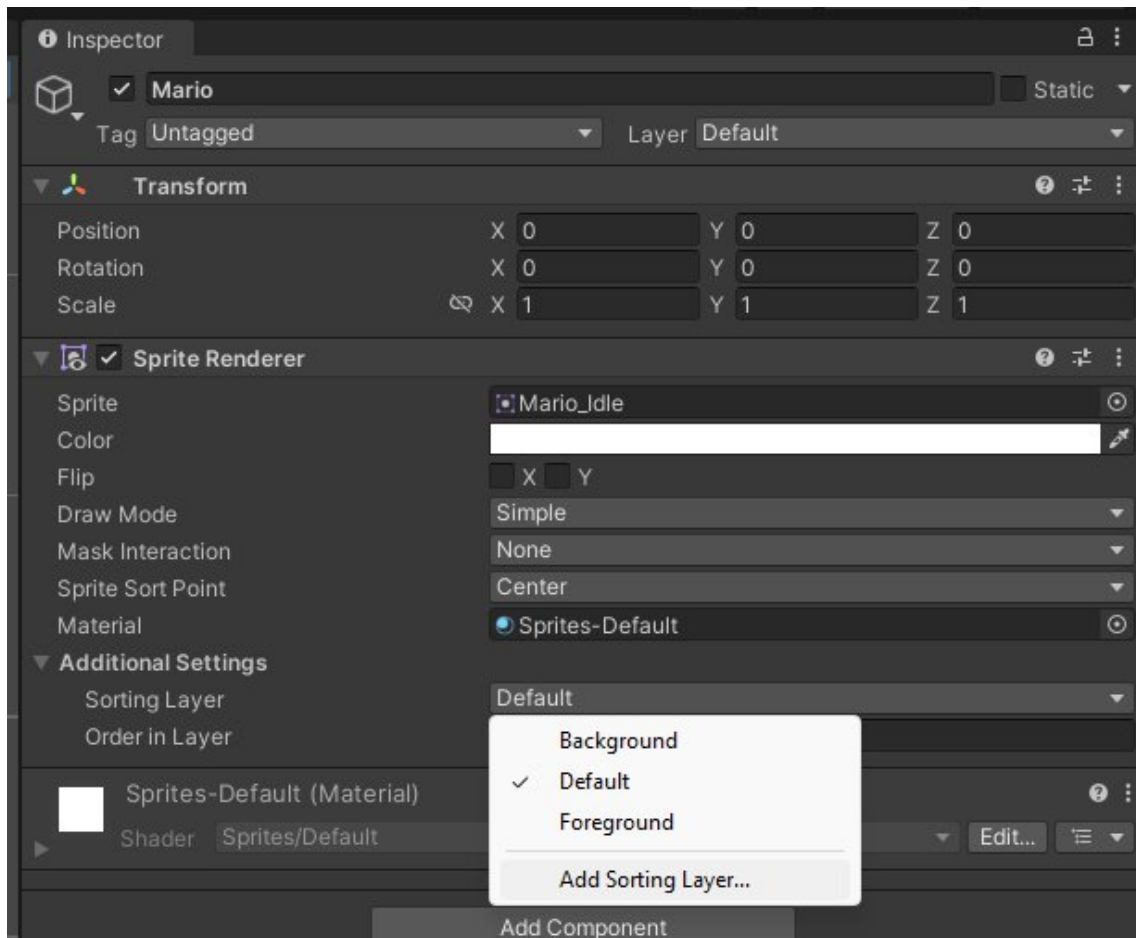
Agregamos otro Objeto Vacío para añadir el terreno del juego



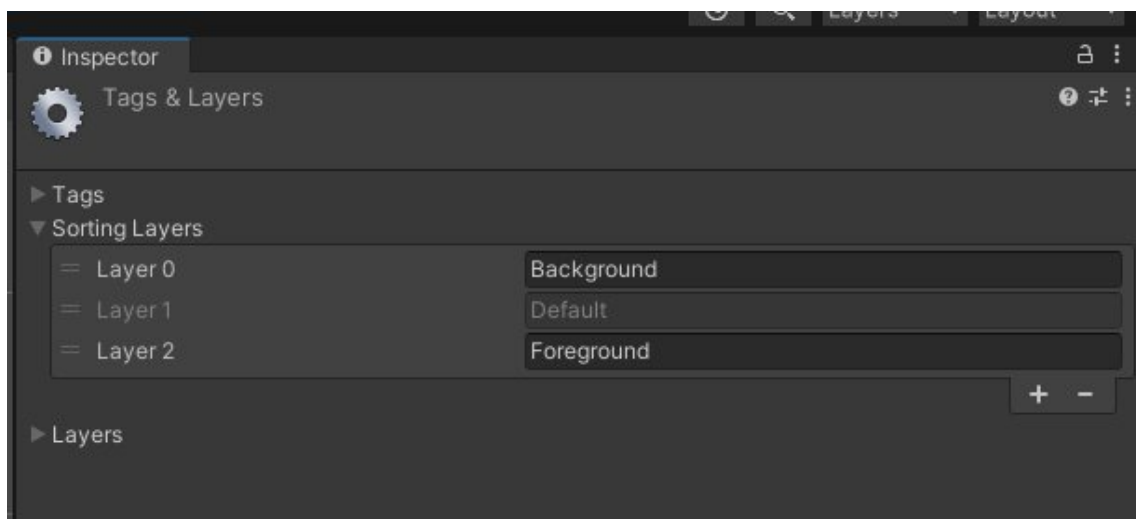
Renombra como Ground y el sprite Ground seleccionamos las opciones de la imagen



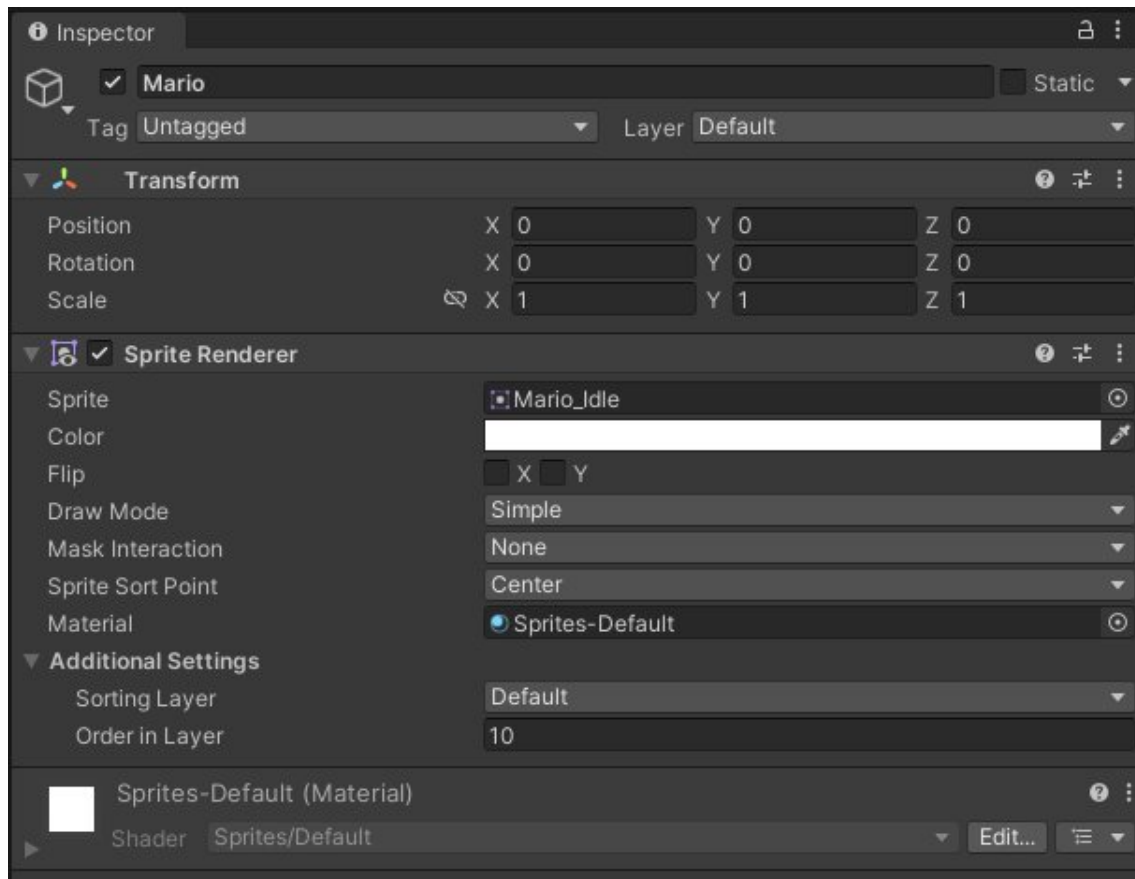
Seleccionar a Mario en nuestra jerarquía y luego agregamos Capas para manejar el orden en que se dibujaran los objetos en nuestro juego



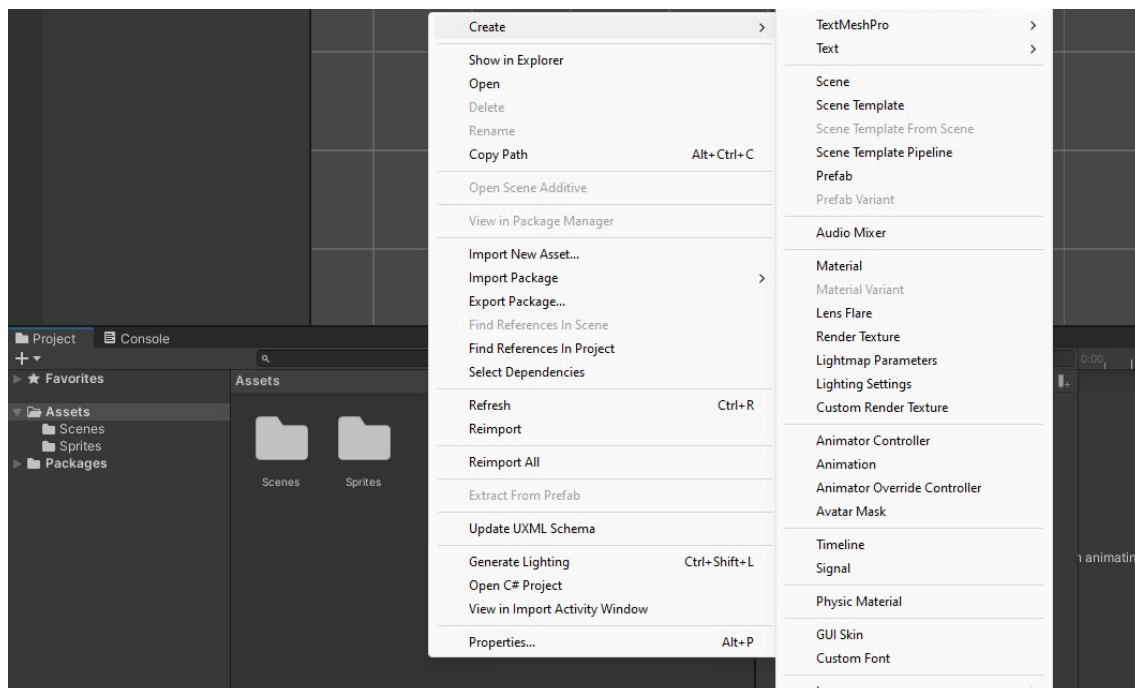
Creamos 2 Nuevos layers Background y Foreground



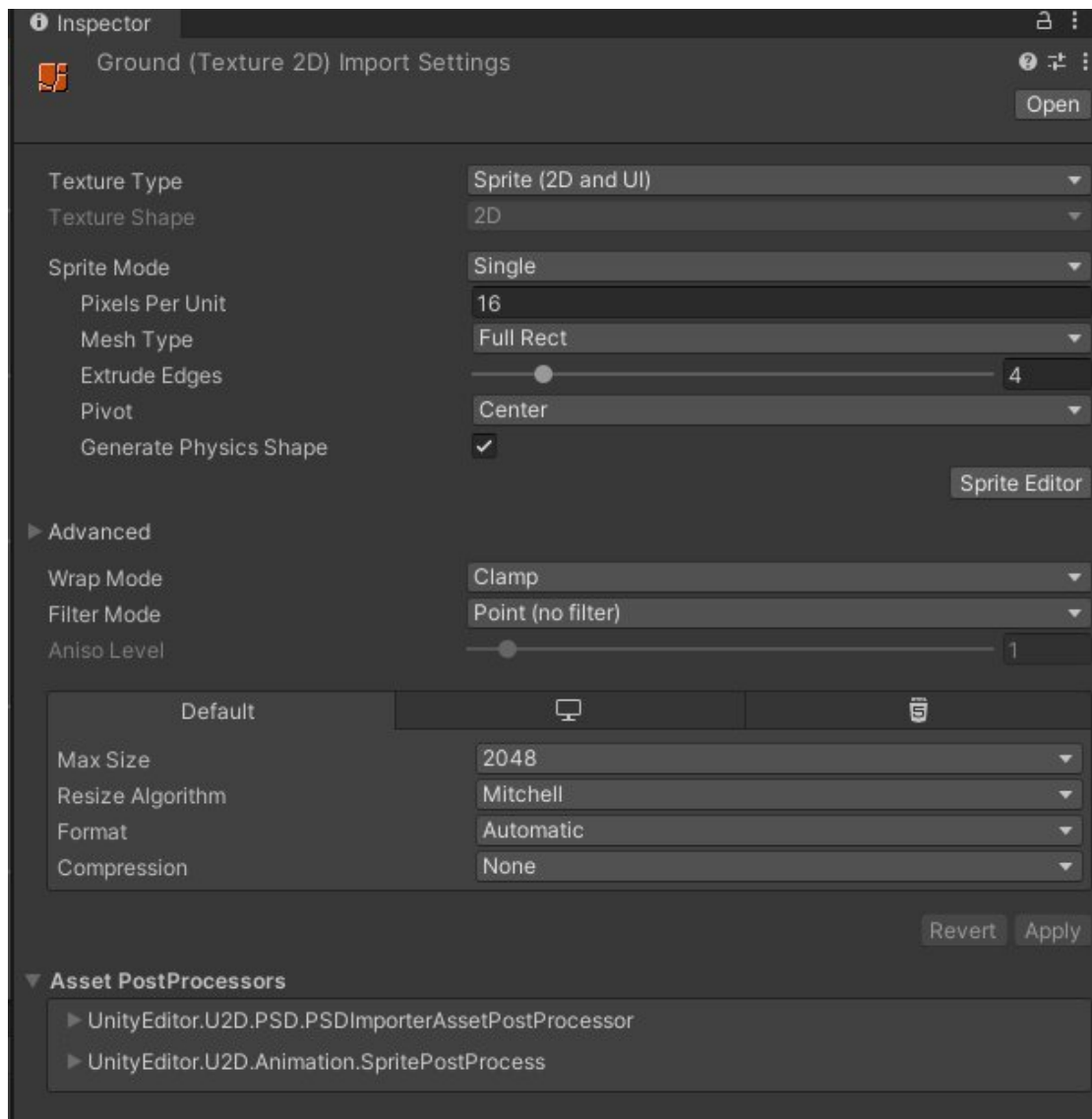
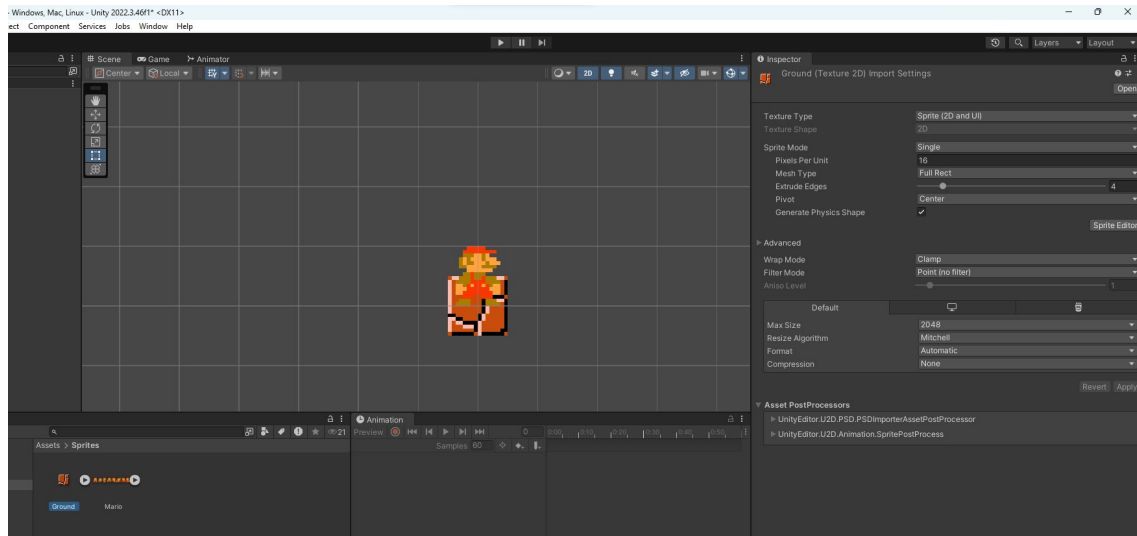
Selecciona a mario en la jerarquía y luego modificamos el Order Layer a 10



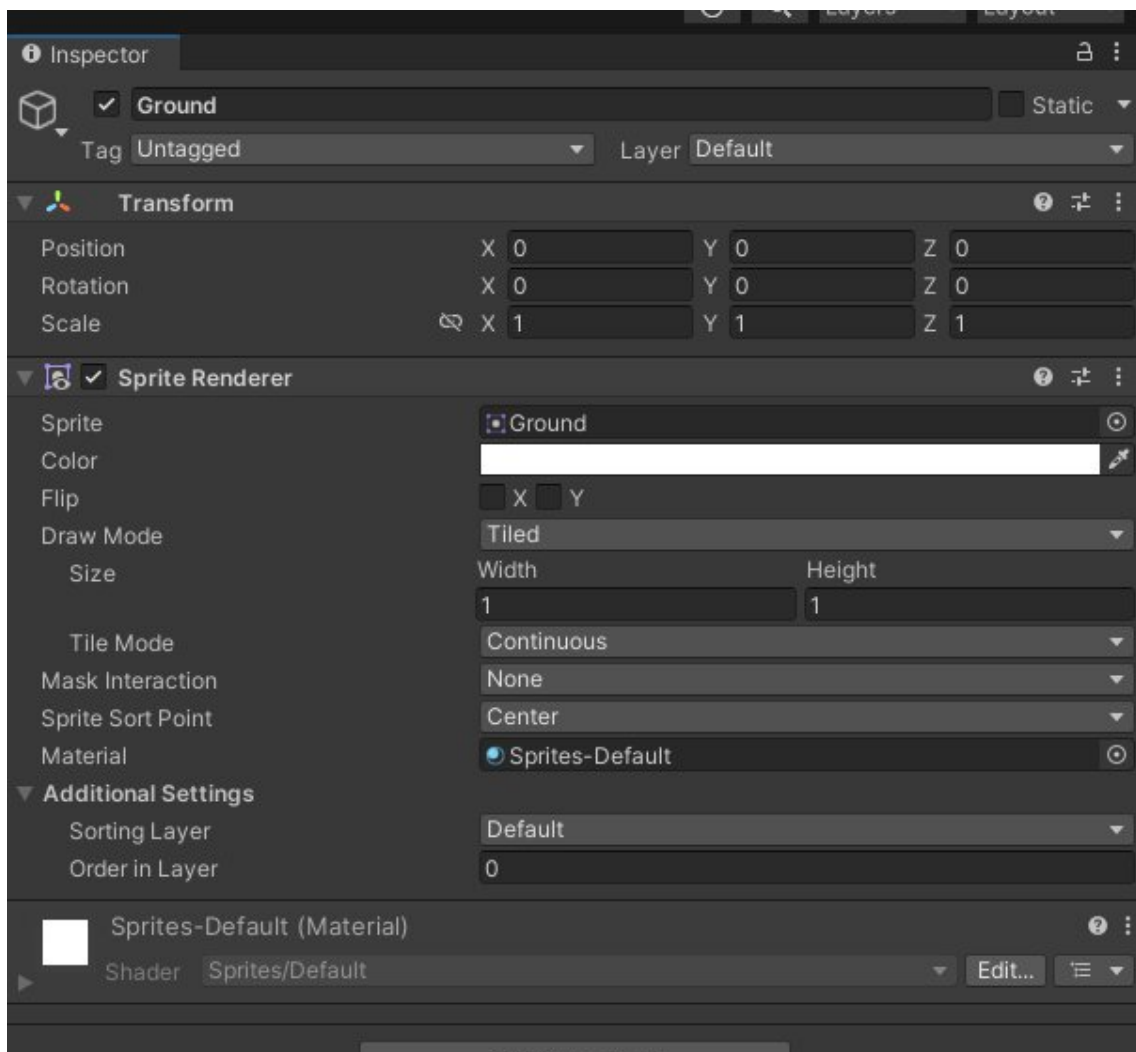
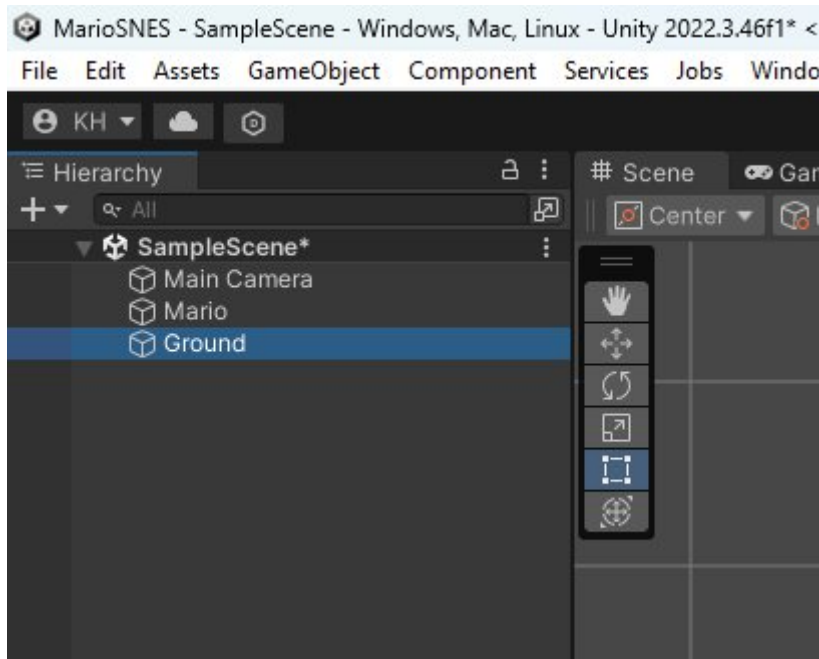
Crea la carpeta Sprites para tu proyecto y mueve los sprite de mario y ground adentro



Selecciona el sprite de Ground y cambiamos **Mesh Type** a **Full Rect**

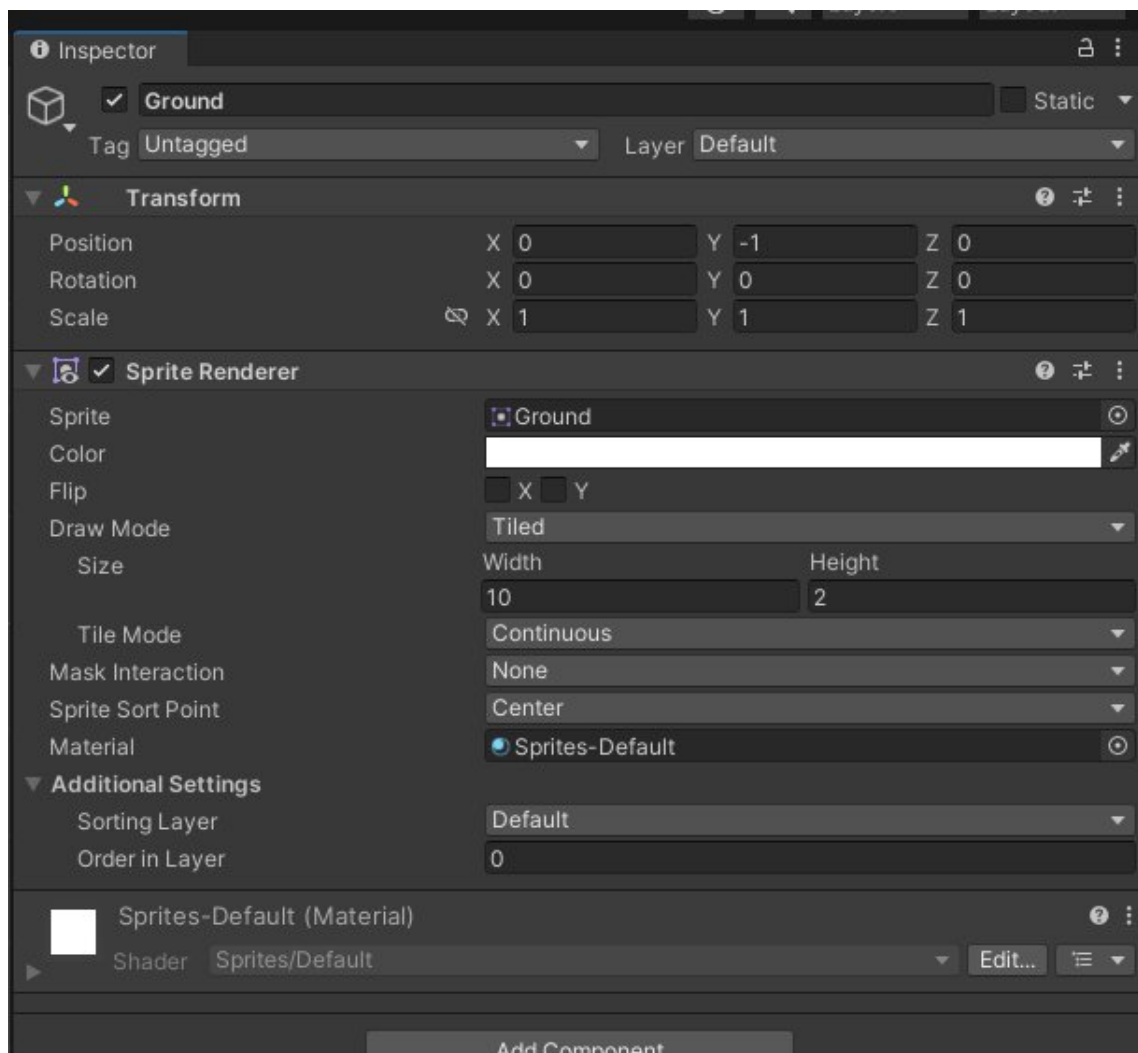
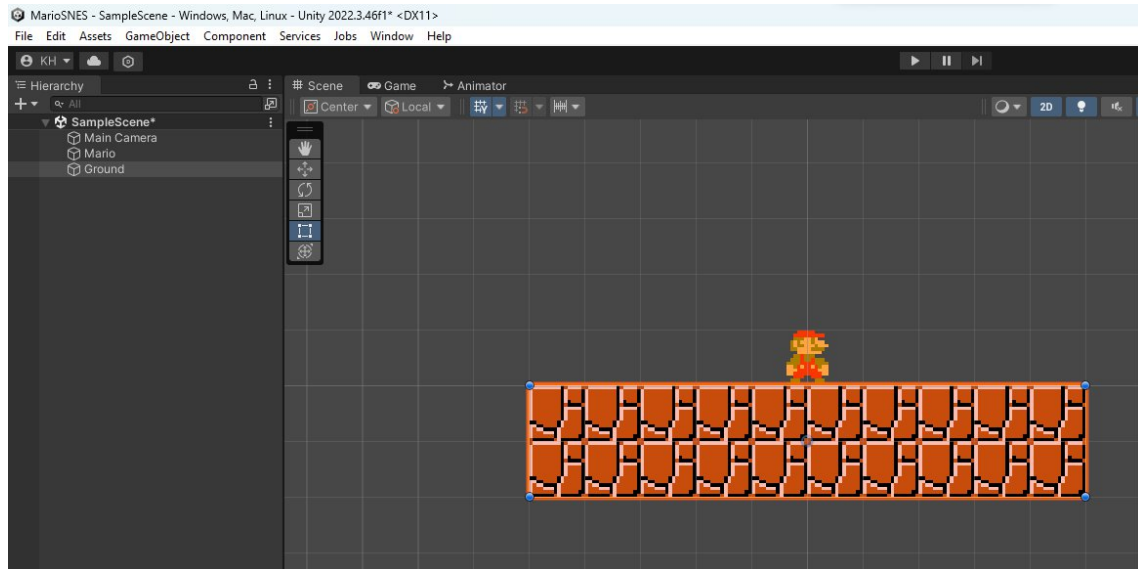


Selecciona al objeto Ground y cambiamos en el inspector **Drawn Mode** a **Tiled**, **Width** y **Height** a **1**



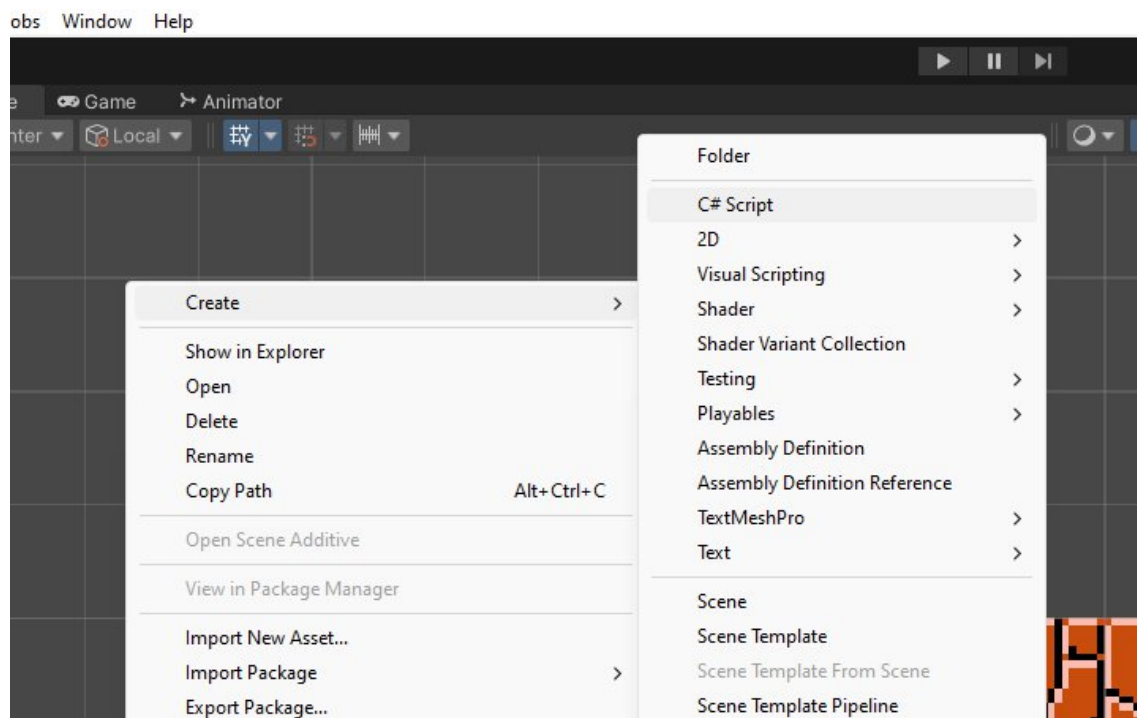
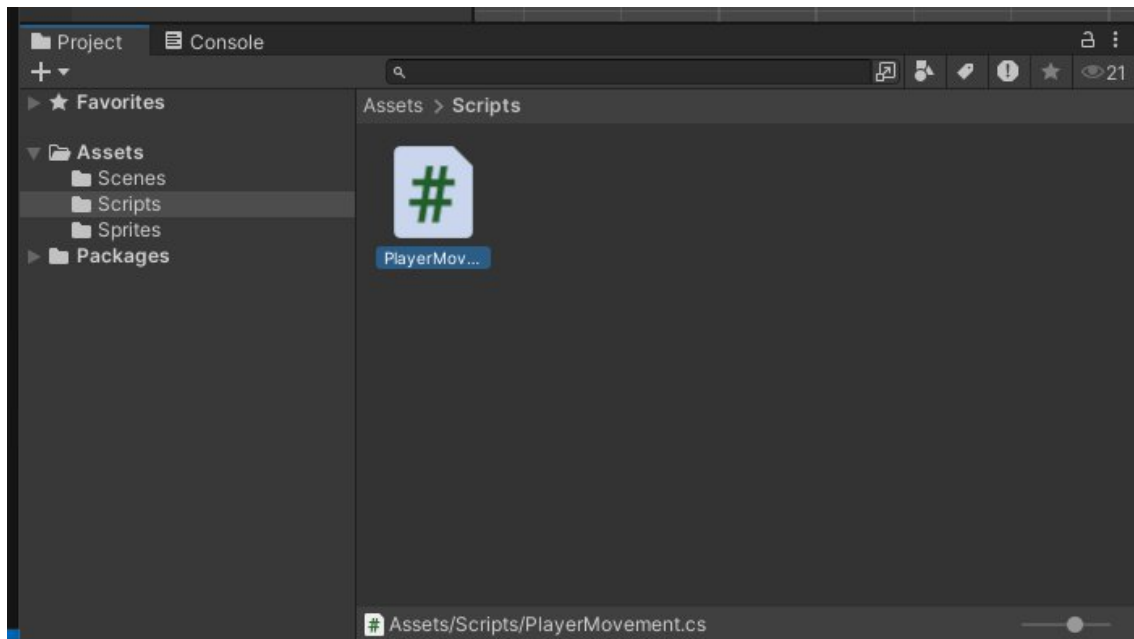
Usando este objeto como Tiled permite modificar su tamaño y hacer que el sprite del suelo se repita, colocamos Width a 10 y Height a 2

También modificamos el Transform del objeto para localizarlo en el suelo de nuestro juego con Y = -1, X = 0



Creamos un nuevo Folder al proyecto para agregar los Scripts de C# que utilizaremos

Click derecho Create C# Script> PlayerMovement



Sobre los eventos en unity, existen muchos y cada uno tiene su propósito los cuales iremos aprendiendo durante el desarrollo del juego

Super Mario Bros. en Unity

Eventos de Unity

```
private void Awake()
{
}
private void Start()
{
}
private void Update()
{
}
private void OnDestroy()
{
}
private void OnEnable()
{
}
private void OnCollisionEnter2D(Collision2D collision)
{
}
private void OnApplicationPause(bool pause)
{
}
```

*Hay muchísimos.
Estos son solo algunos
ejemplos

UnityWok



Para aplicar una fuerza sobre un objeto utilizamos

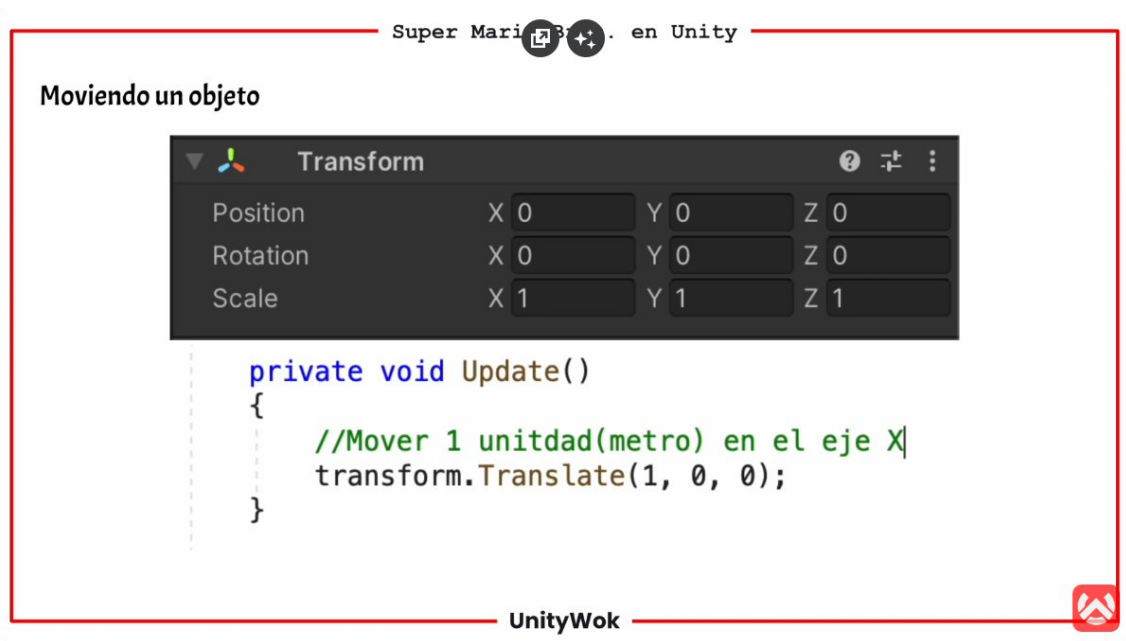
Super Mario Bros. en Unity

Update() - se ejecuta cada frame

```
private void Update()
{
    //Mover 1 unidad(metro) en el eje X
    transform.Translate(1, 0, 0);
}
```

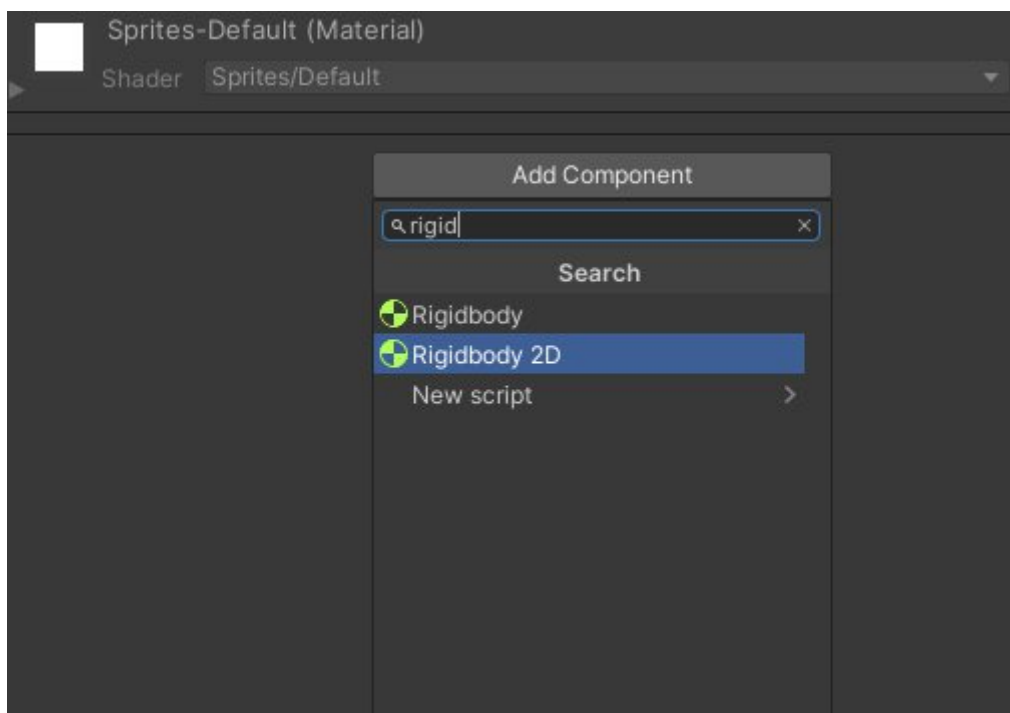
UnityWok



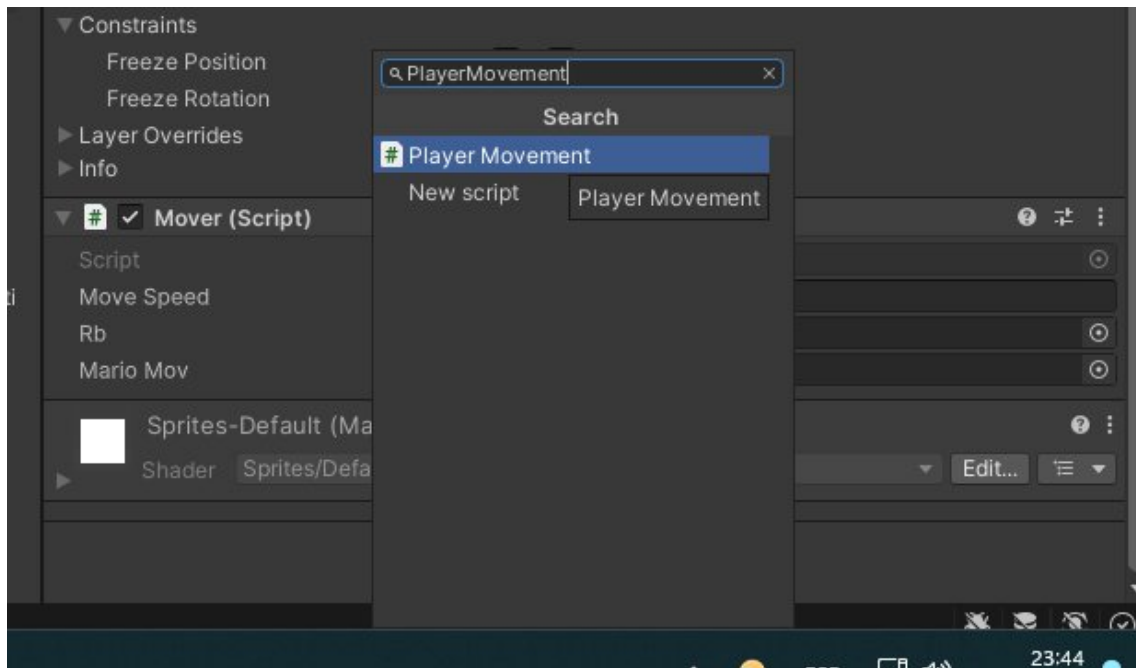


Para realizar el movimiento de mario usaremos las mecánicas AWSD ó ↓ ← ↑ → para el movimiento de mario

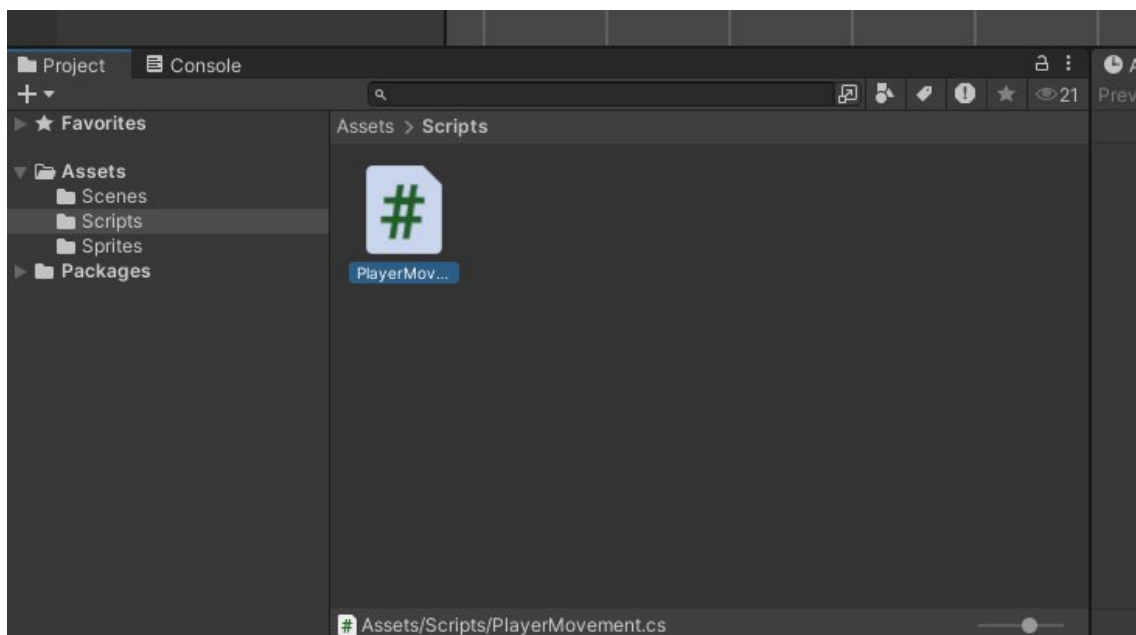
Seleccionamos al objeto Mario en la jerarquía de unity y agregamos un RidigBody2D



Tambien agregamos un nuevo Script a nuestro GameObject Mario Add>PlayerMovement



Abrimos nuestro Script PlayerMovement para definir nuestra lógica



Definimos nuestras variables y recogemos el input del usuario para realizar los movimientos del jugador, además definimos funciones como

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f;      // Velocidad de movimiento
    public float jumpForce = 10f;     // Fuerza de salto ajustable
    public Rigidbody2D rb;            // Referencia al componente
    public SpriteRenderer spriteRenderer; // Para cambiar la dirección
    public Transform groundCheck;     // Verificación de si Mario está en
    public LayerMask groundLayer;     // Capa que define el suelo
    private Vector2 movement;         // Vector para almacenar el
    public bool isCrouching = false;  // Verificar si Mario está agachado
    private bool isJumping = false;   // Verificar si Mario está saltando

    void Update()
    {
        // Solo permitir movimiento en el eje horizontal
        movement.x = Input.GetAxis("Horizontal");

        // Cambiar la dirección del sprite según el movimiento
        if (movement.x > 0) // Si Mario se mueve a la derecha
        {
            spriteRenderer.flipX = false; // No voltear el sprite
        }
        else if (movement.x < 0) // Si Mario se mueve a la izquierda
        {
            spriteRenderer.flipX = true; // Voltear el sprite
        }

        // Si está en el suelo y el usuario presiona hacia abajo, Mario
        // se agacha
        if (IsGrounded() && Input.GetAxis("Vertical") < 0)
        {
            isCrouching = true;
        }
        else
        {
            isCrouching = false;
        }
    }
}
```

```

        // Detectar si el jugador quiere saltar y está en el suelo
        if (Input.GetButtonDown("Jump") && IsGrounded())
        {
            isJumping = true;
        }
    }

    void FixedUpdate()
    {
        // Mover el personaje solo si no está agachado
        if (!isCrouching)
        {
            rb.velocity = new Vector2(movement.x * moveSpeed,
rb.velocity.y);
        }

        // Aplicar el salto si se ha detectado el input y el personaje
está en el suelo
        if (isJumping)
        {
            rb.velocity = new Vector2(rb.velocity.x, jumpForce); //
Aplicar el salto directamente a la velocidad
            isJumping = false; // Evitar que salte de nuevo hasta que
vuelva a tocar el suelo
        }
    }

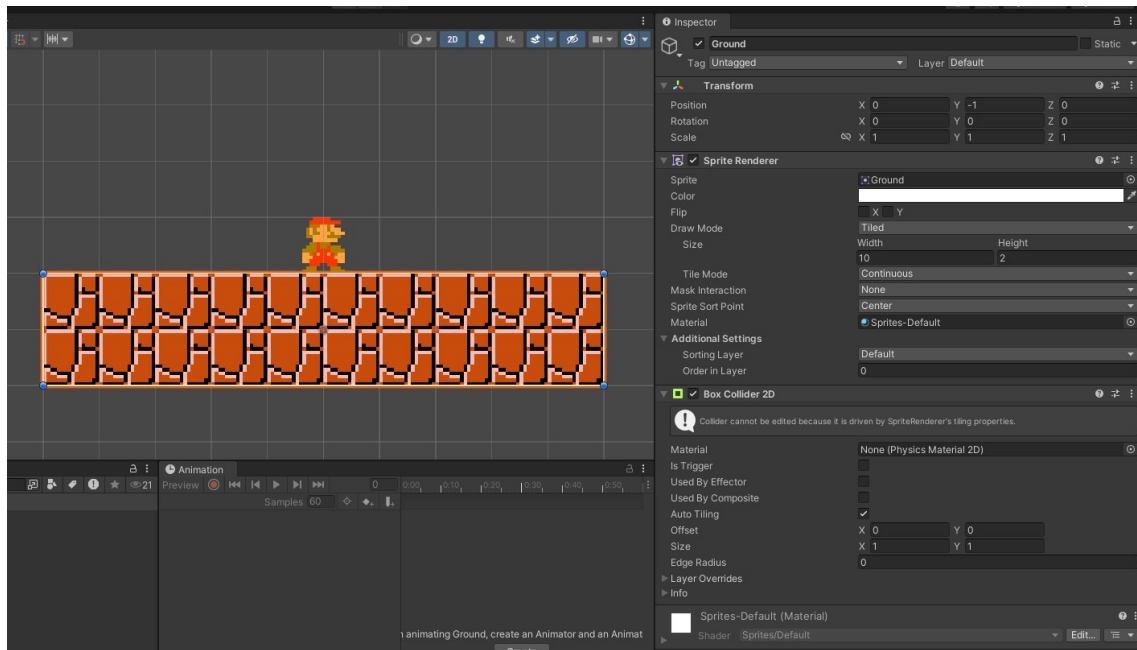
    // Verificar si Mario está en el suelo
    private bool IsGrounded()
    {
        // Verificar si hay colisión en el área de GroundCheck
        bool grounded = Physics2D.OverlapCircle(groundCheck.position,
0.2f, groundLayer);

        // Mostrar en la consola si Mario está en el suelo o no
        Debug.Log("IsGrounded: " + grounded);

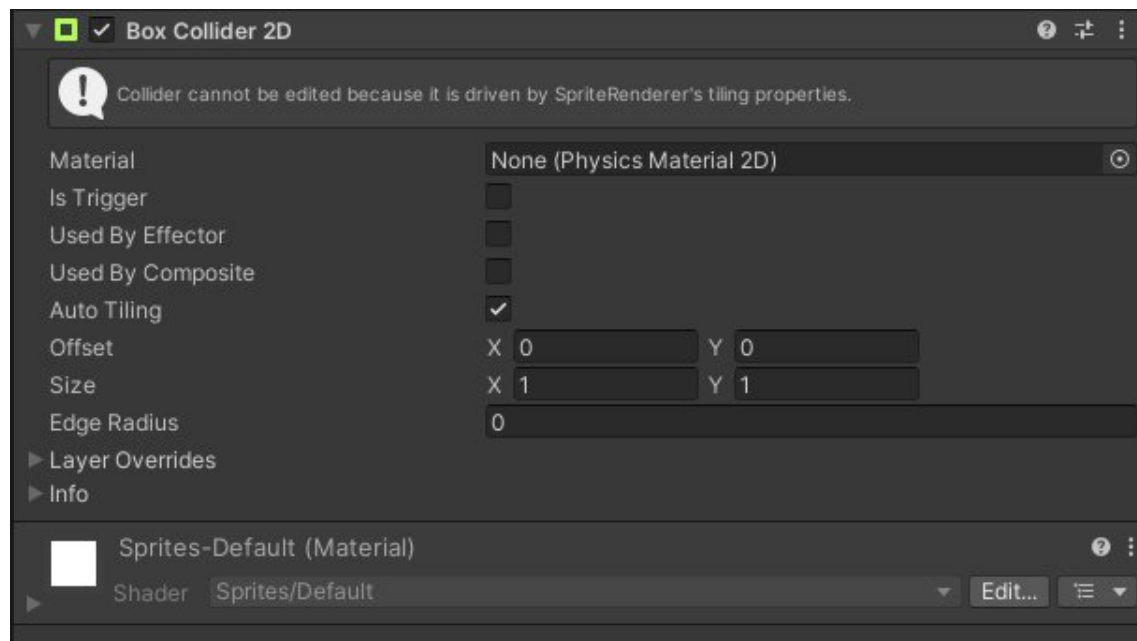
        return grounded;
    }
}

```

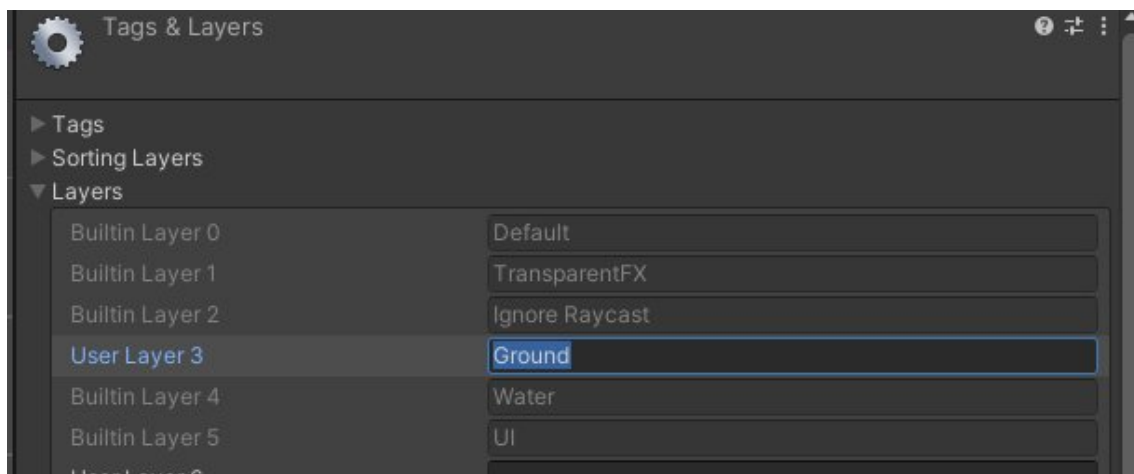
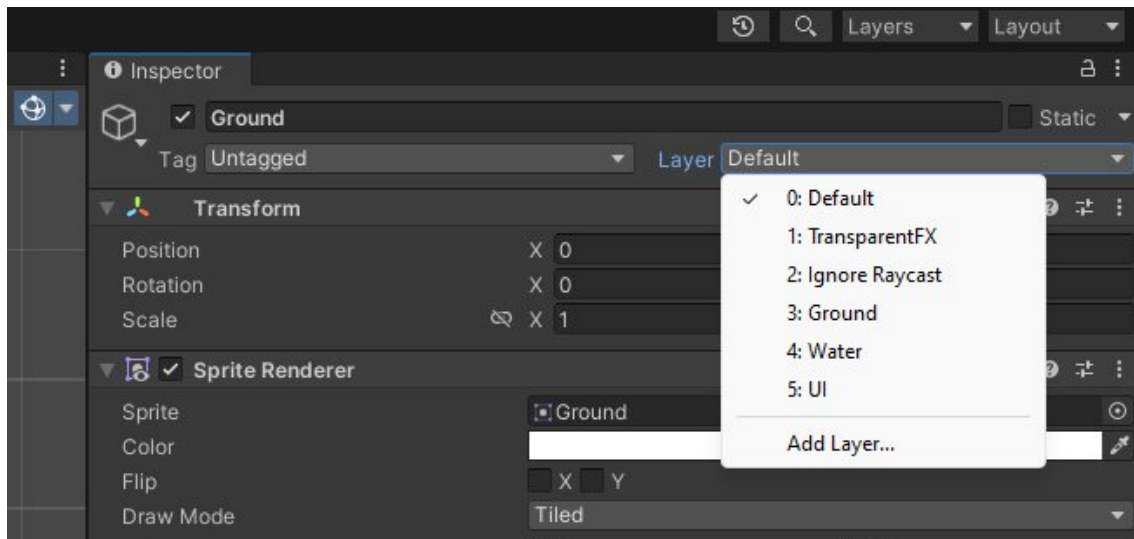
Ahora Seleccionamos el objeto Ground y agregamos el componente Box Collider 2D



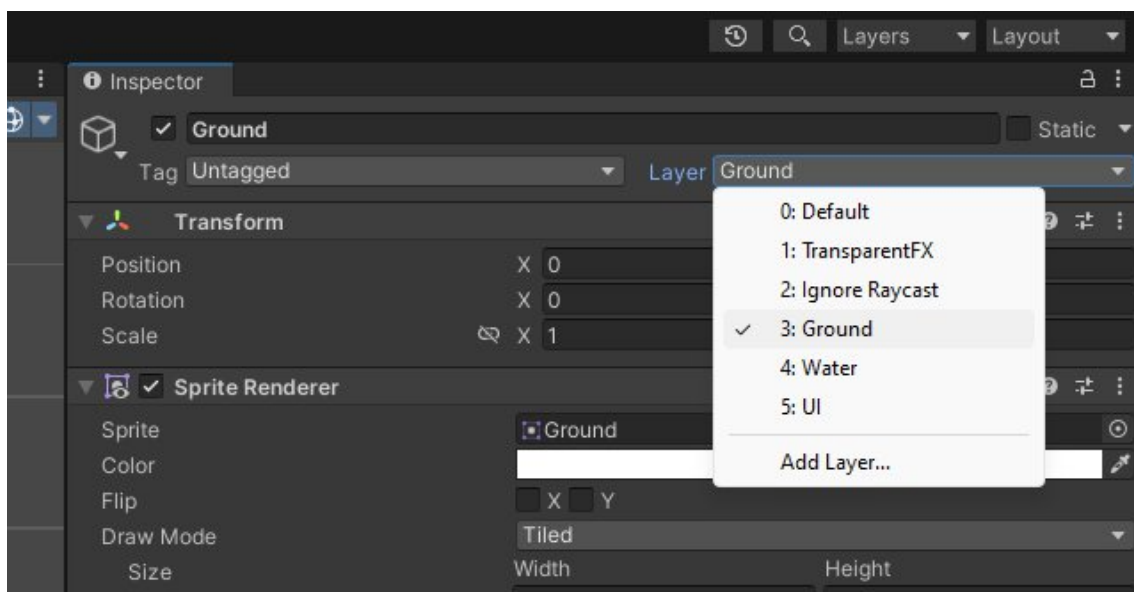
Le damos check a Auto Tiling para que el box collider se ajuste al tamaño de nuestro objeto



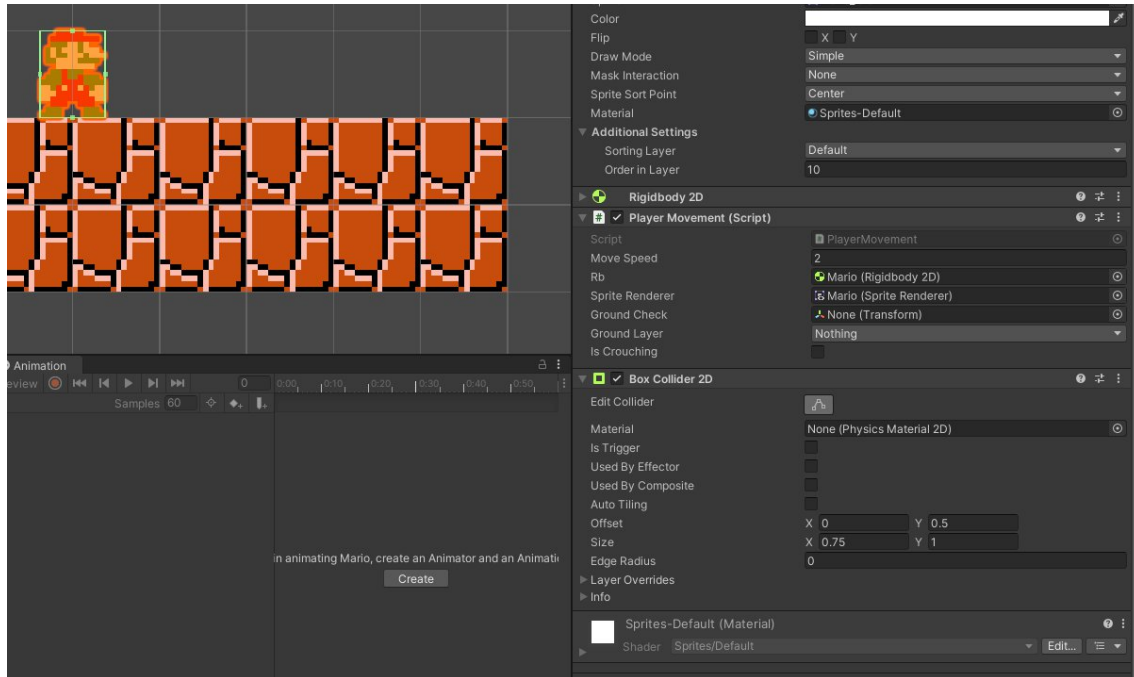
En la parte superior del Inspector Agregamos un Layer llamado Ground



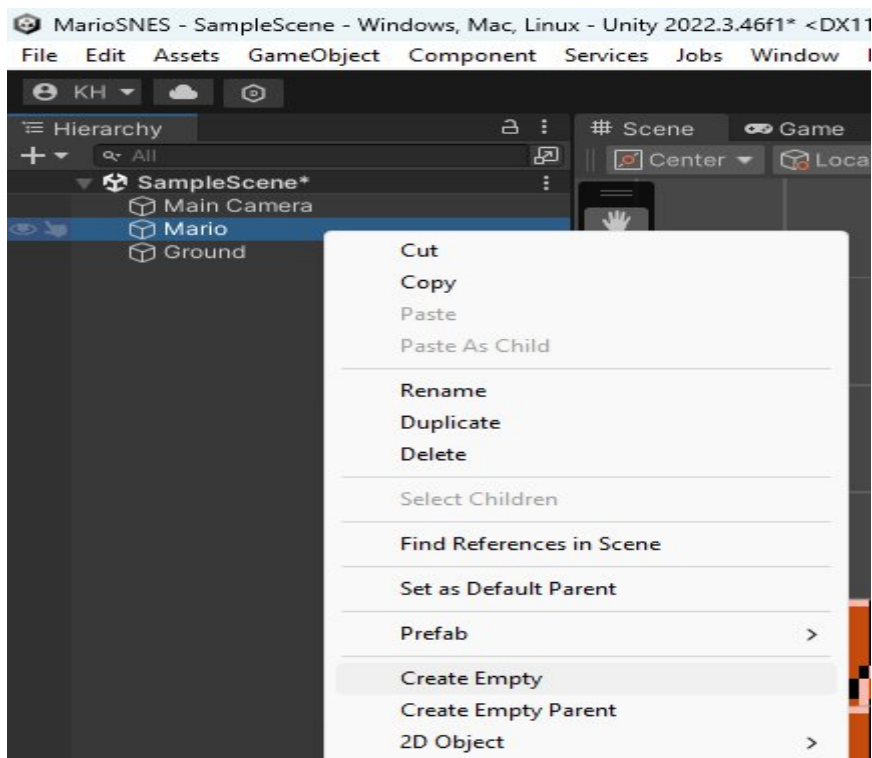
Ahora seleccionamos nuestro Ground en la jerarquía y en Layer Seleccionamos Ground



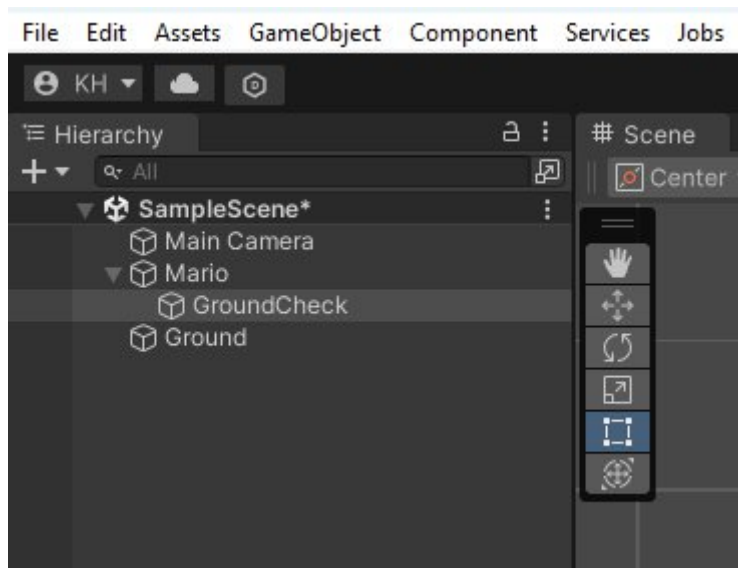
También agregamos un Collider para Mario



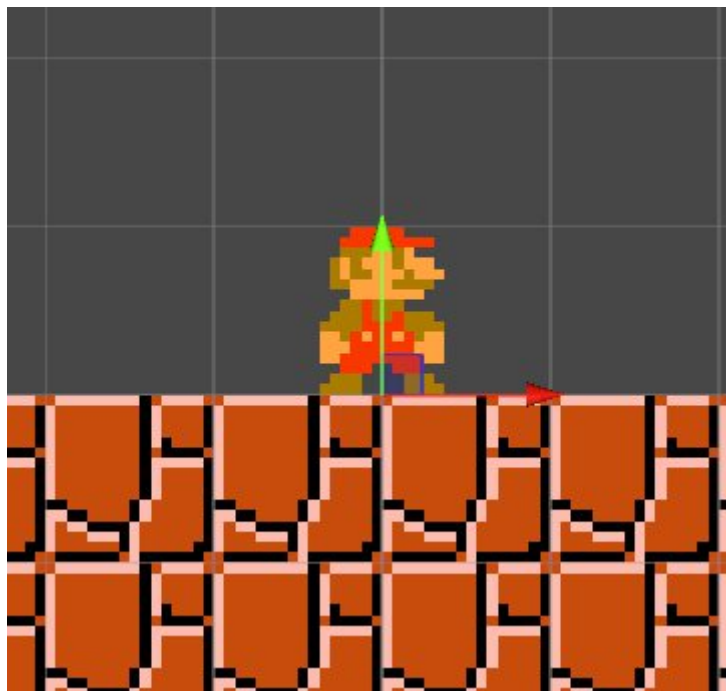
Agregamos un nuevo objeto hijo para nuestro GameObject Mario



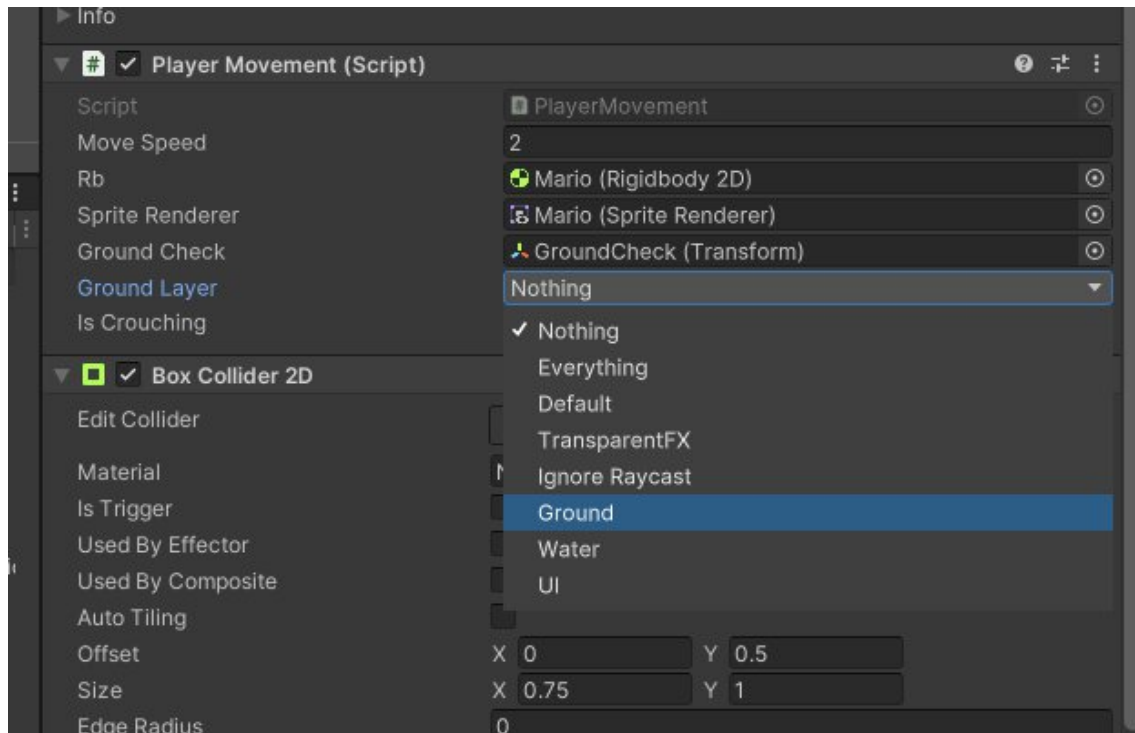
Y renombamos como GroundCheck



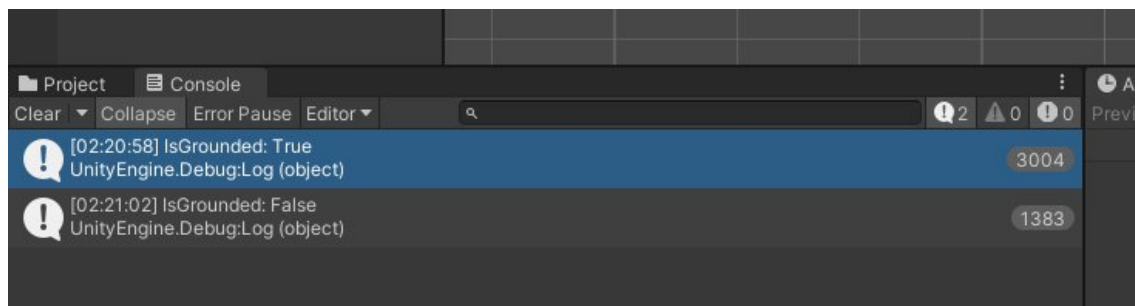
El objeto GroundCheck estará posicionado en los pies de Mario el cual usaremos para detectar la colisión con el Ground del escenario



En nuestro Objeto Mario seleccionamos el Ground Layer a Ground



Ahora podemos saber cada vez que Mario este interactuando con el suelo de nuestro escenario



Con esto tendríamos configurado las acciones de moverse izquierda y derecha además del salto

Nuestro Script C# PlayerMovement quedaría así:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f;      // Velocidad de movimiento
    ajustable
    public float jumpForce = 10f;     // Fuerza de salto ajustable
    public Rigidbody2D rb;            // Referencia al componente
    Rigidbody2D
    public SpriteRenderer spriteRenderer; // Para cambiar la dirección
    del sprite
    public Transform groundCheck;     // Verificación de si Mario está en
    el suelo
    public LayerMask groundLayer;     // Capa que define el suelo
    private Vector2 movement;         // Vector para almacenar el
    movimiento del jugador
    public bool isCrouching = false;  // Verificar si Mario está agachado
    private bool isJumping = false;   // Verificar si Mario está saltando

    void Update()
    {
        // Solo permitir movimiento en el eje horizontal
        movement.x = Input.GetAxis("Horizontal");

        // Cambiar la dirección del sprite según el movimiento
        if (movement.x > 0) // Si Mario se mueve a la derecha
        {
            spriteRenderer.flipX = false; // No voltear el sprite
        }
        else if (movement.x < 0) // Si Mario se mueve a la izquierda
        {
            spriteRenderer.flipX = true; // Voltear el sprite
        }

        // Si está en el suelo y el usuario presiona hacia abajo, Mario
        se agacha
        if (IsGrounded() && Input.GetAxis("Vertical") < 0)
        {
            isCrouching = true;
        }
        else
        {
            isCrouching = false;
        }

        // Detectar si el jugador quiere saltar y está en el suelo
        if (Input.GetButtonDown("Jump") && IsGrounded())
```

```

        {
            isJumping = true;
        }
    }

    void FixedUpdate()
    {
        // Mover el personaje solo si no está agachado
        if (!isCrouching)
        {
            rb.velocity = new Vector2(movement.x * moveSpeed,
rb.velocity.y);
        }

        // Aplicar el salto si se ha detectado el input y el personaje
está en el suelo
        if (isJumping)
        {
            rb.velocity = new Vector2(rb.velocity.x, jumpForce); //
Aplicar el salto directamente a la velocidad
            isJumping = false; // Evitar que salte de nuevo hasta que
vuelva a tocar el suelo
        }
    }

    // Verificar si Mario está en el suelo
    private bool IsGrounded()
    {
        // Verificar si hay colisión en el área de GroundCheck
        bool grounded = Physics2D.OverlapCircle(groundCheck.position,
0.2f, groundLayer);

        // Mostrar en la consola si Mario está en el suelo o no
        Debug.Log("IsGrounded: " + grounded);

        return grounded;
    }
}

```

Fin primera parte...

PARTE 2

Para controlar los eventos de mario necesitamos saber cuándo está o no tocando el suelo para lo cual añadimos al Script una variable la cual tomara el valor de nuestra función `IsGrounded()`;

```
1 reference
public float moveSpeed = 5f; // Velocidad de movimiento ajustable
1 reference
public float jumpForce = 10f; // Fuerza de salto ajustable
4 references
public Rigidbody2D rb; // Referencia al componente Rigidbody2D
2 references
public SpriteRenderer spriteRenderer; // Para cambiar la dirección del sprite
1 reference
public Transform groundCheck; // Verificación de si Mario está en el suelo
1 reference
public LayerMask groundLayer; // Capa que define el suelo
4 references
private Vector2 movement; // Vector para almacenar el movimiento del jugador
1 reference
[SerializeField] private bool isGrounded; // Variable para ver el estado si mario esta tocando o no el suelo
3 references
public bool isCrouching = false; // Verificar si Mario está agachado
3 references
private bool isJumping = false; // Verificar si Mario está saltando
0 references
```

```
0 references
void Update()
{
    // Solo permitir movimiento en el eje horizontal
    movement.x = Input.GetAxis("Horizontal");

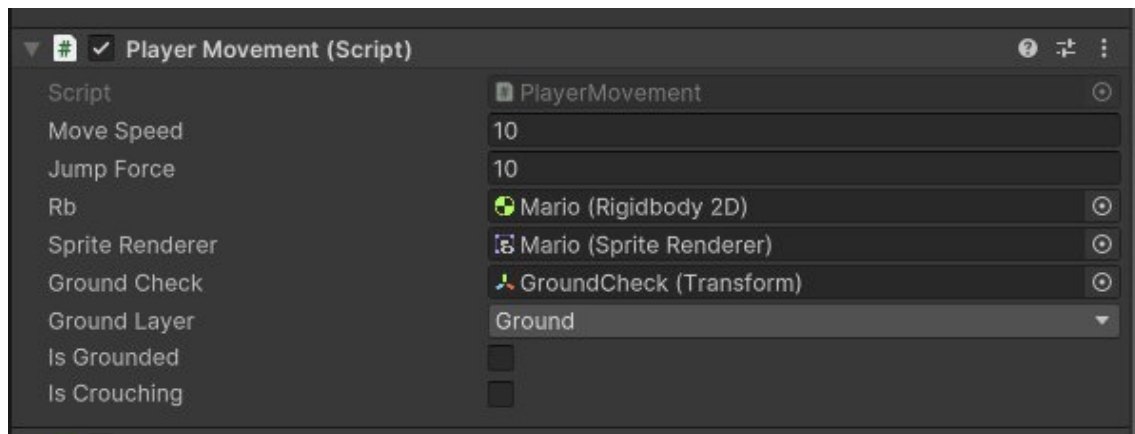
    // Cambiar la dirección del sprite según el movimiento
    if (movement.x > 0) // Si Mario se mueve a la derecha
    {
        spriteRenderer.flipX = false; // No voltear el sprite
    }
    else if (movement.x < 0) // Si Mario se mueve a la izquierda
    {
        spriteRenderer.flipX = true; // Voltear el sprite
    }

    // Si está en el suelo y el usuario presiona hacia abajo, Mario se agacha
    if (IsGrounded() && Input.GetAxis("Vertical") < 0)
    {
        isCrouching = true;
    }
    else
    {
        isCrouching = false;
    }

    // Detectar si el jugador quiere saltar y está en el suelo
    if (Input.GetButtonDown("Jump") && IsGrounded())
    {
        isJumping = true;
    }

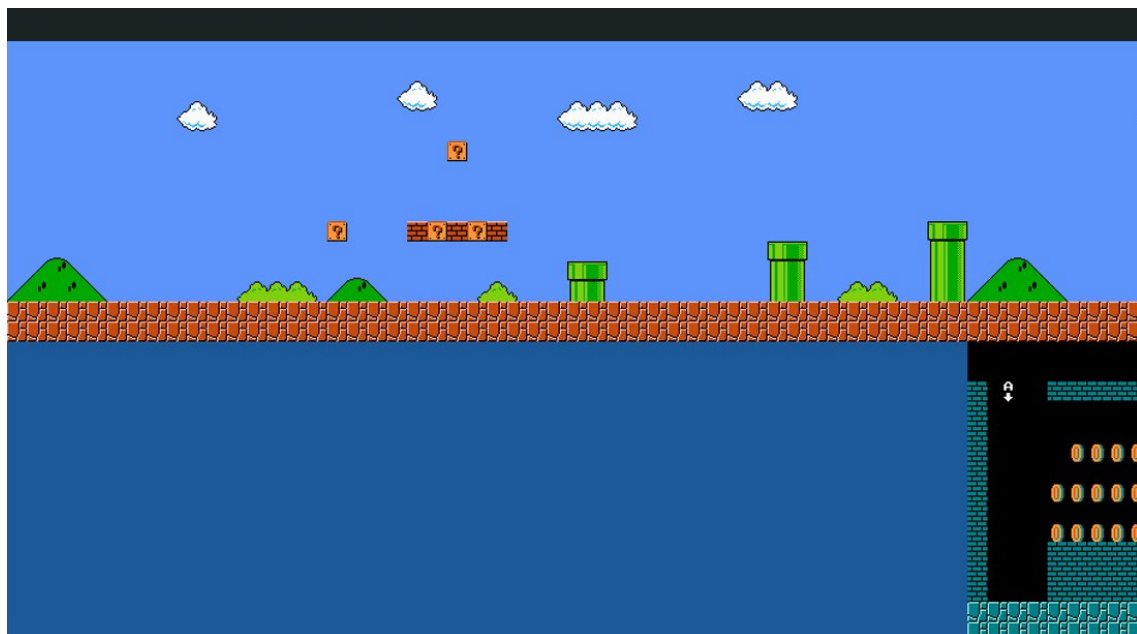
    isGrounded = IsGrounded();
}
```

Con estos cambios se agrega al inspector en el Script "Is Grounded"

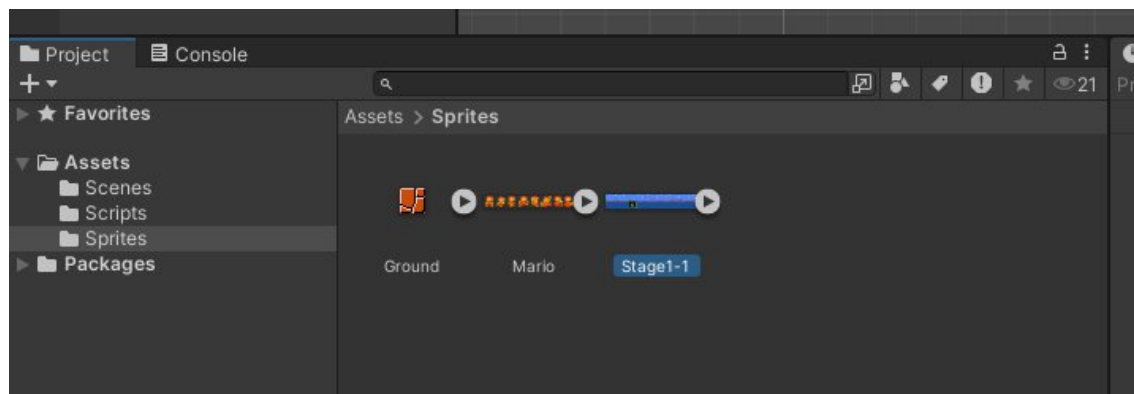


Descargar el siguiente Sprite que será el Stage 1-1 del juego

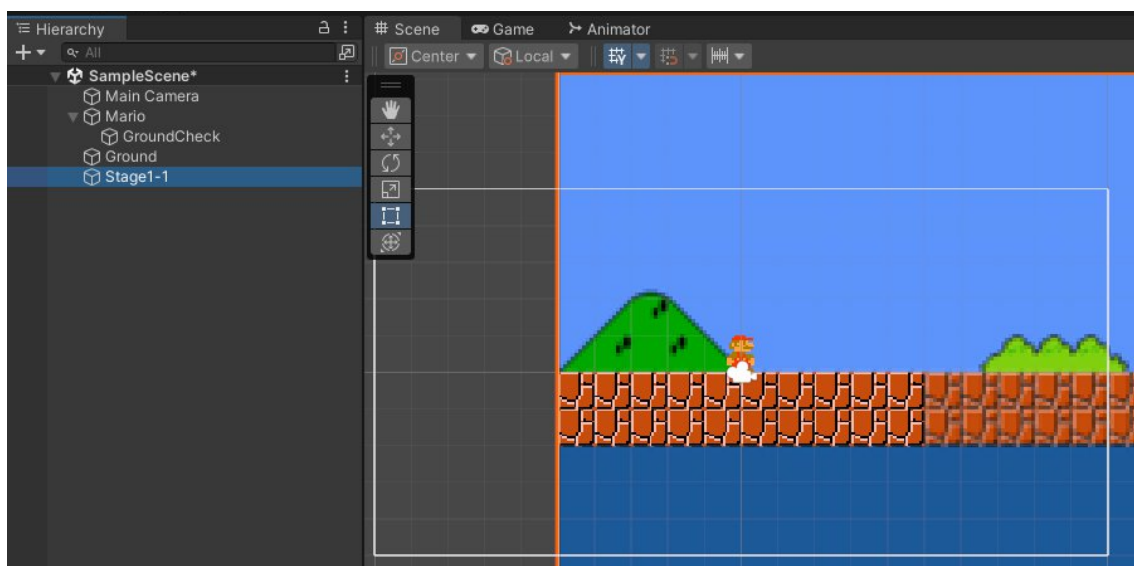
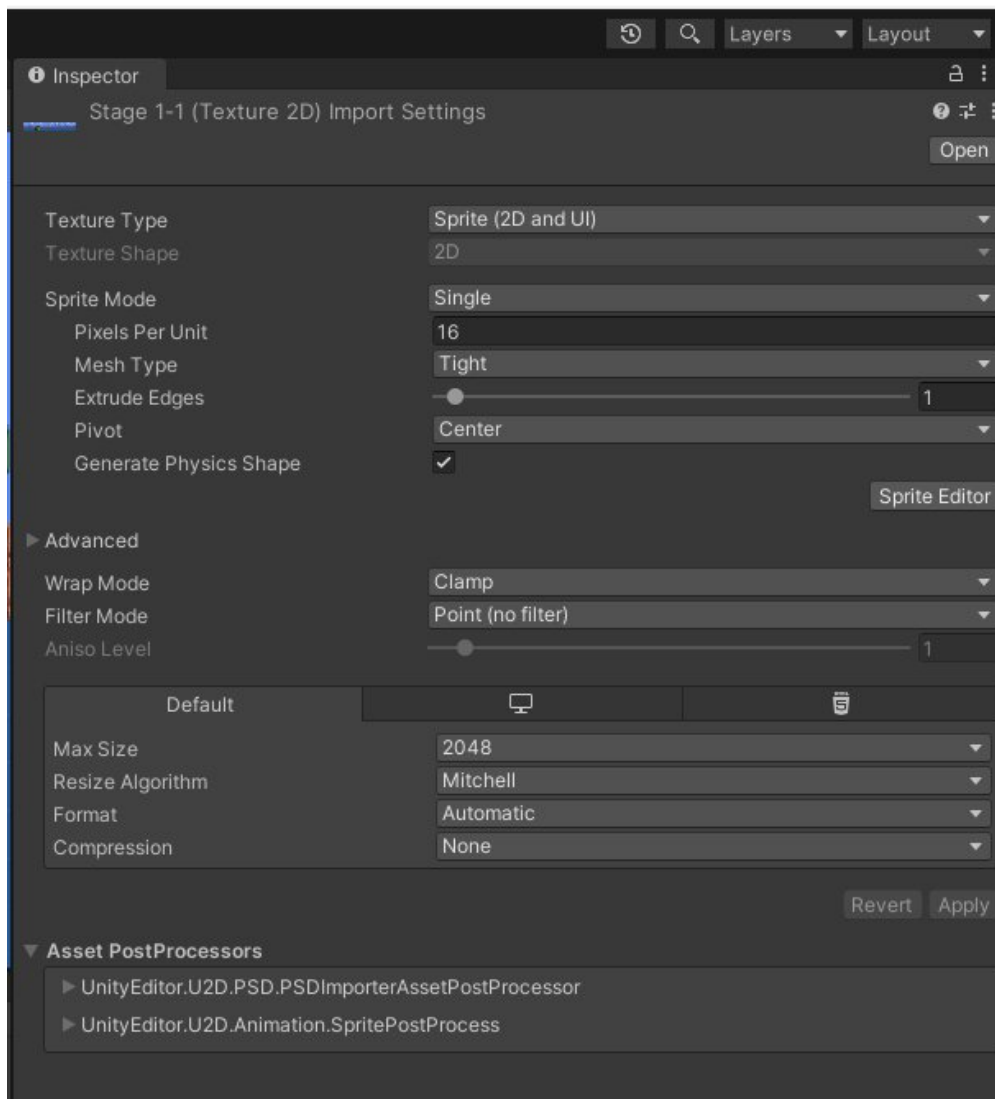
https://drive.google.com/file/d/1053t7bzQS2UAg2_zErZ7fd6gdNG3-FJQ/view?usp=sharing



Agregamos el sprite a nuestra carpeta del proyecto

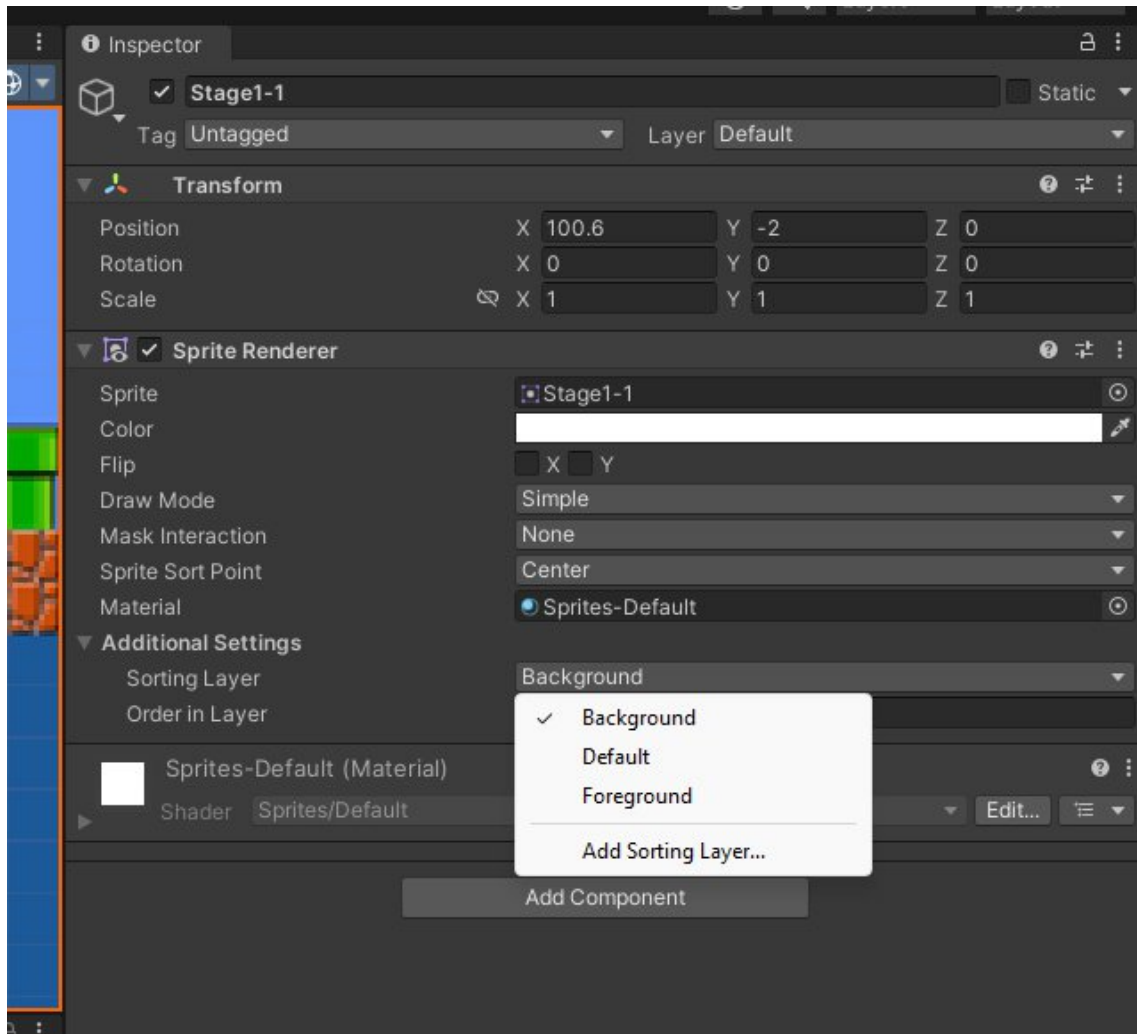


Arrastramos el escenario hacia la jerarquía de UNITY y configuramos

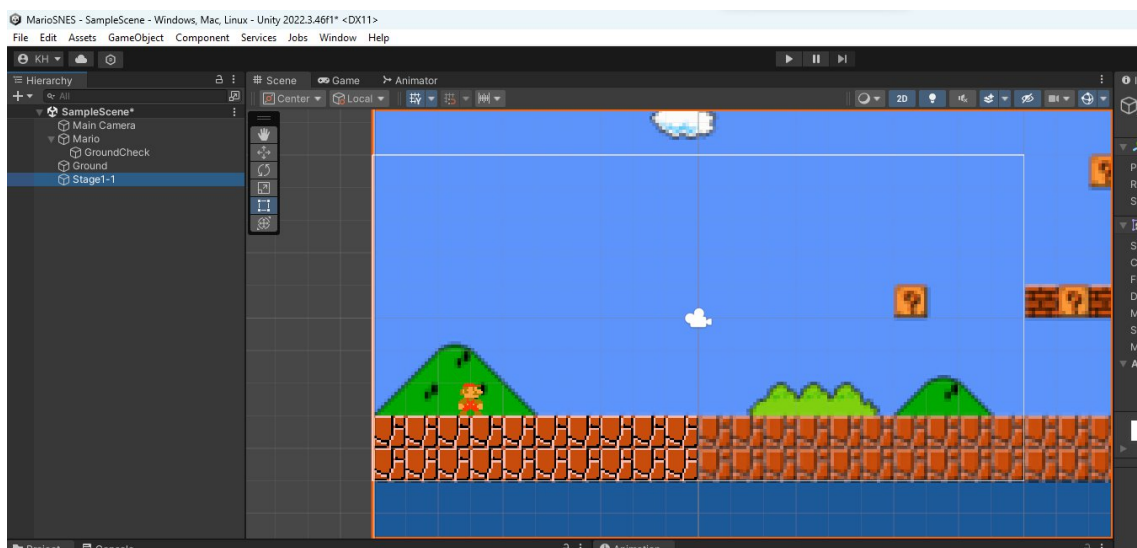


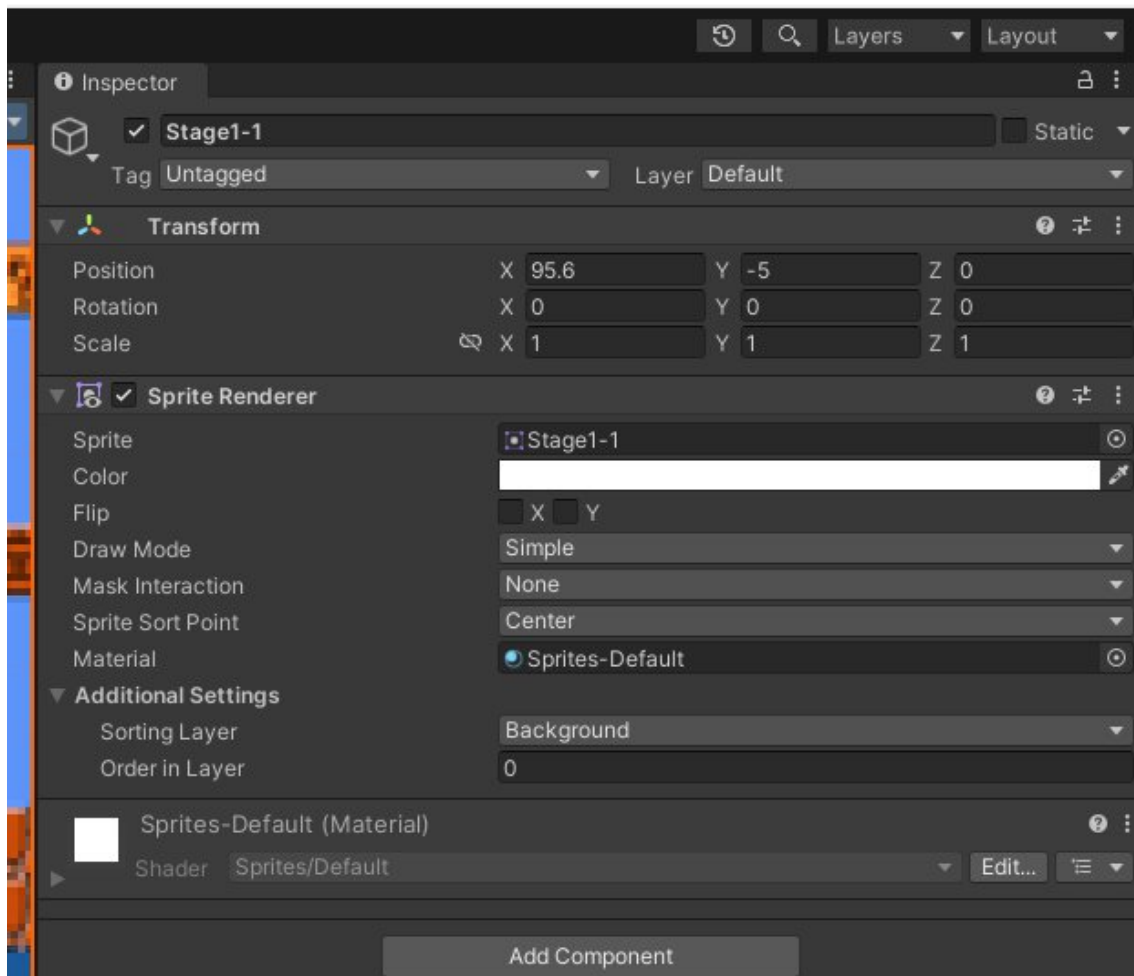
Asegúrate de que el sprite del Stage 1-1 esté en una capa que no interfiera con los objetos del nivel. Para esto seleccionamos el stage en la jerarquía y en el inspector :

SET Sorting Layer to Background ó **Edit > Project Settings > Tags and Layers**(Agregar si no lo creaste)



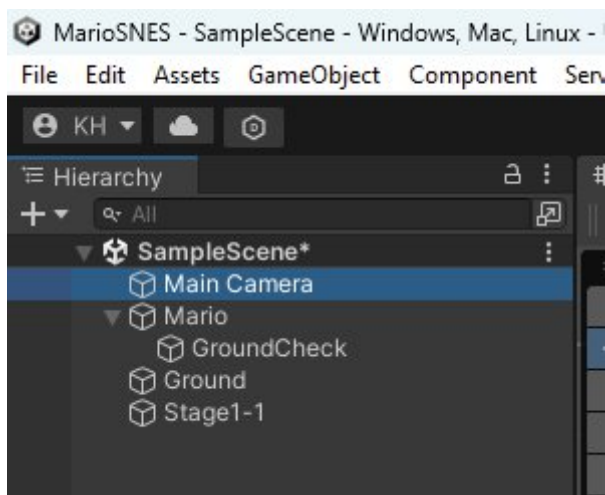
Cambiamos la posición del stage y de mario para comenzar a recrear el nivel





Ahora configuramos el tamaño de la cámara para que se muestre como en el juego original MARIO SNES 1985

Seleccionamos Main cámara en la jerarquía

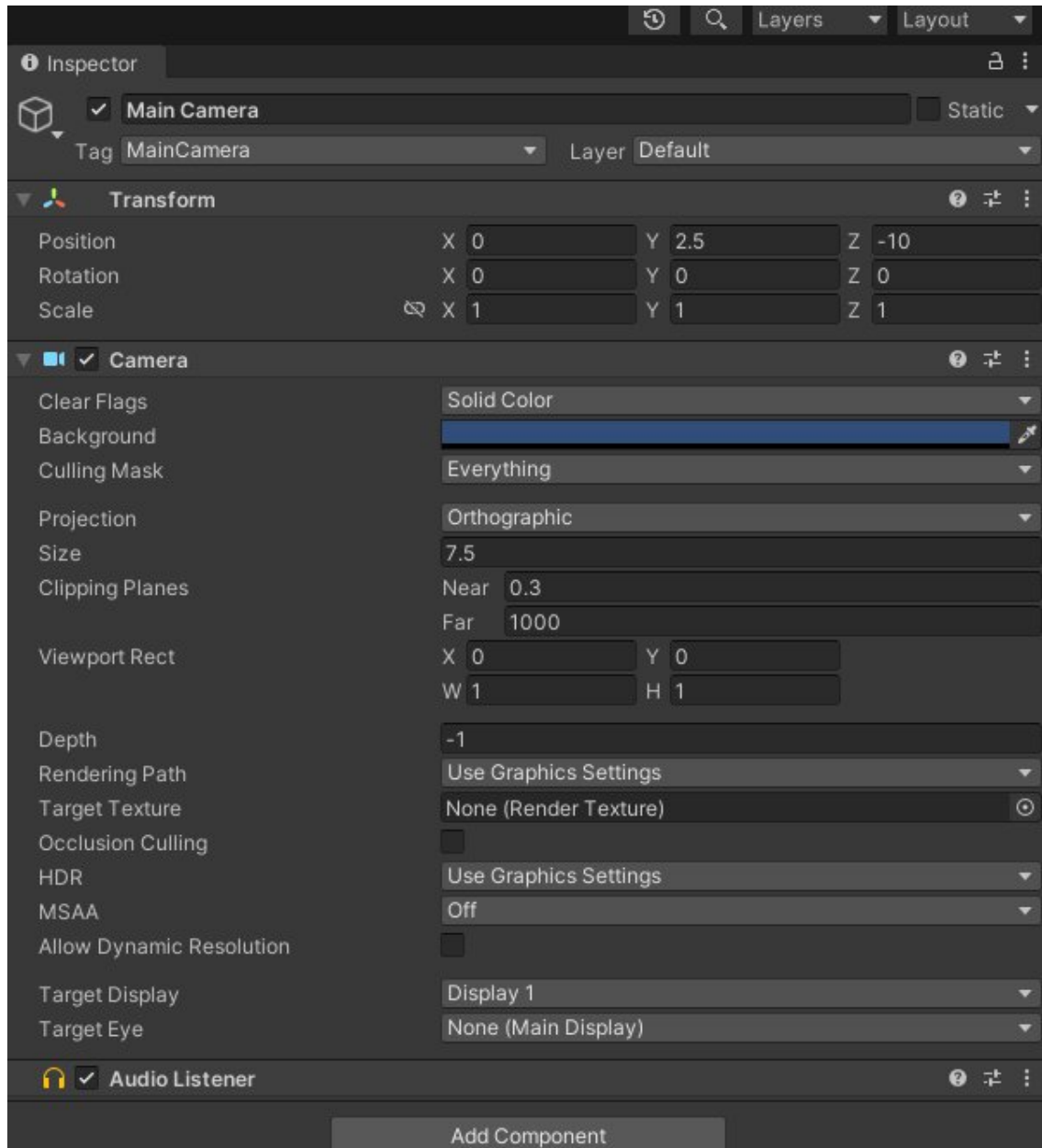


Ya que estamos utilizando 16 pixeles por UNIDAD en nuestro juego necesitamos calcular el tamaño de nuestra cámara

$$\text{Tamaño de la cámara} = \frac{\text{Altura en píxeles}}{\text{Píxeles por unidad}} = \frac{240}{16} = 15$$

¿Por qué se calcula $240 / 16 = 15$ para el tamaño de la cámara?

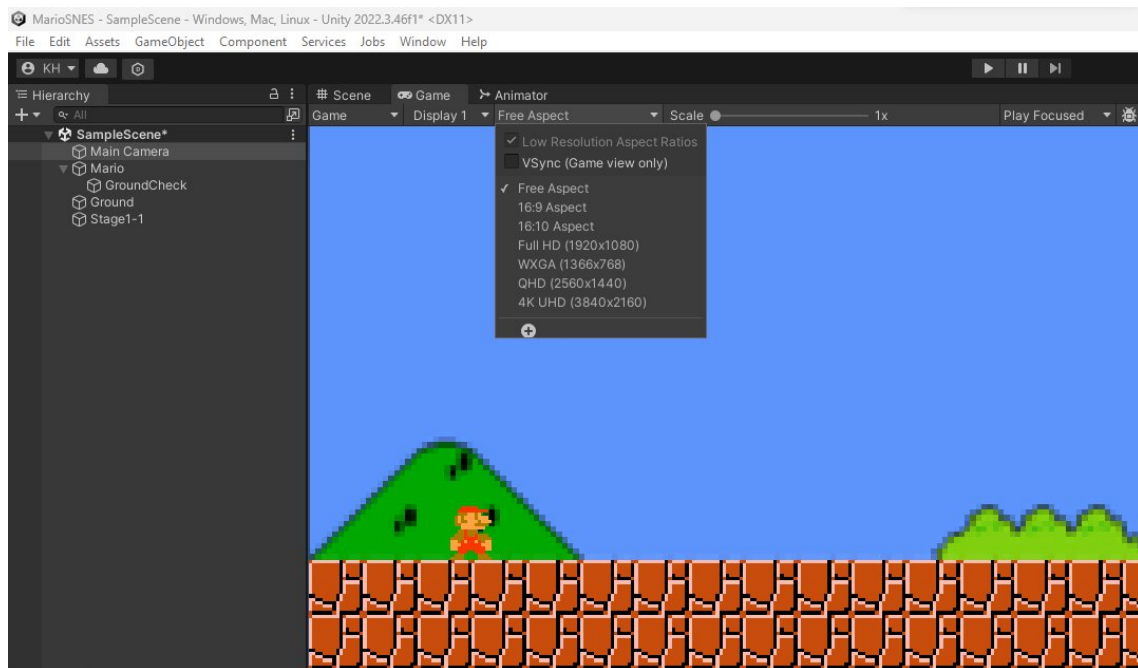
La resolución estándar de un juego en **4:3** que coincide con el estilo de los juegos retro es de **320x240** píxeles. Esto significa que si deseas que tu cámara en Unity represente esa resolución, debes hacer coincidir la altura de la cámara con la altura en píxeles que se está utilizando en el juego.



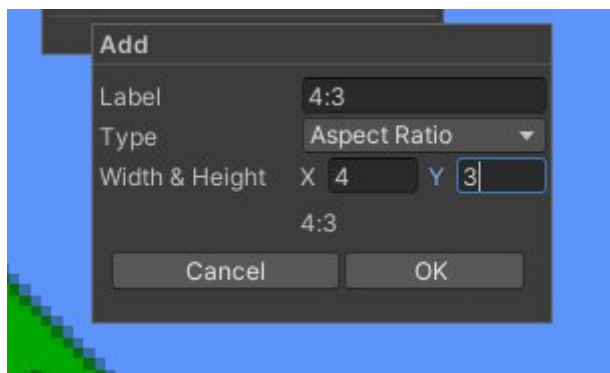
Cuando estableces el tamaño de la cámara en **Unity**, el valor que ingresas se refiere a la mitad de la altura visible de la cámara en el modo **Orthographic**. Por lo tanto, cuando trabajas con una cámara ortográfica, el **Size** de la cámara se

refiere a la distancia desde el centro de la cámara hasta la parte superior de la vista.

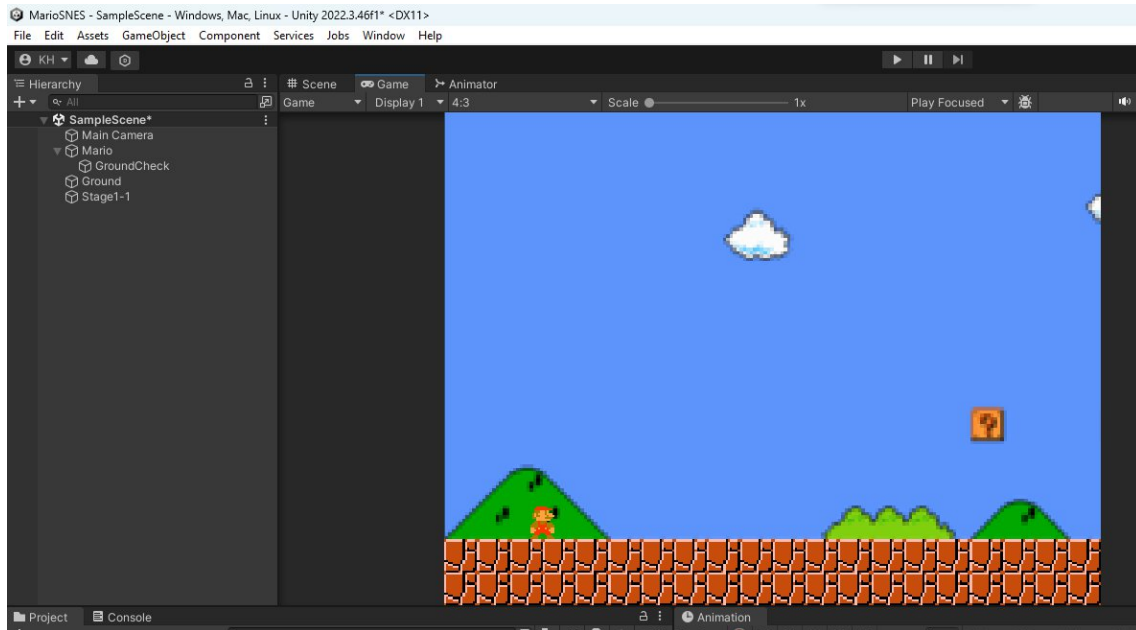
Ahora mientras tenemos seleccionado la main camera, vamos a la pestaña Game y cambiaremos el Game View del juego, clic en +



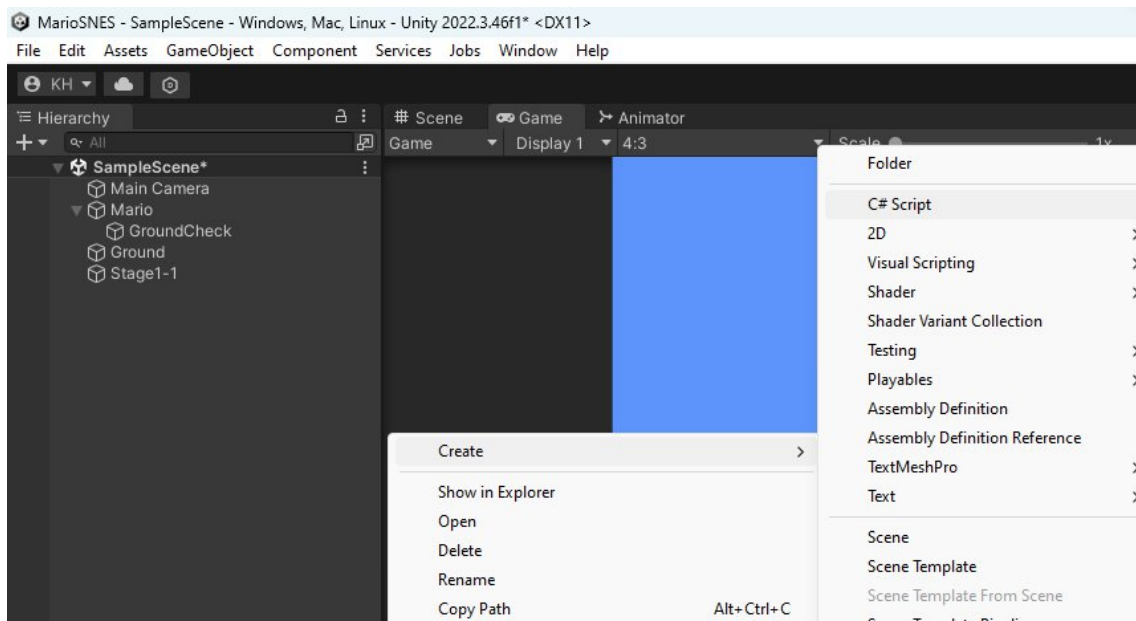
Añadimos el Aspecto 4:3



Con esto tenemos configurado nuestra cámara, ahora le añadiremos movimiento



Creamos un nuevo Script en nuestra carpeta y nombramos CameraFollow.cs



En el script especificaremos las acciones que tomara la cámara cuando Mario cruce el 50% del largo la cámara comenzara a avanzar

```
using UnityEngine;

public class CameraFollow : MonoBehaviour
{
    public Transform player; // Referencia al jugador (Mario)
    public float followSpeed = 0.1f; // Velocidad de seguimiento de la
    cámara

    private float cameraHalfWidth; // Mitad del ancho de la cámara en
    unidades
```

```

void Start()
{
    // Calcular la mitad del ancho de la cámara en unidades
    cameraHalfWidth = Camera.main.orthographicSize *
Camera.main.aspect;
}

void LateUpdate()
{
    // Obtener la posición de Mario
    Vector3 playerPosition = player.position;

    // Calcular el punto medio de la cámara
    float cameraMidPointX = transform.position.x;

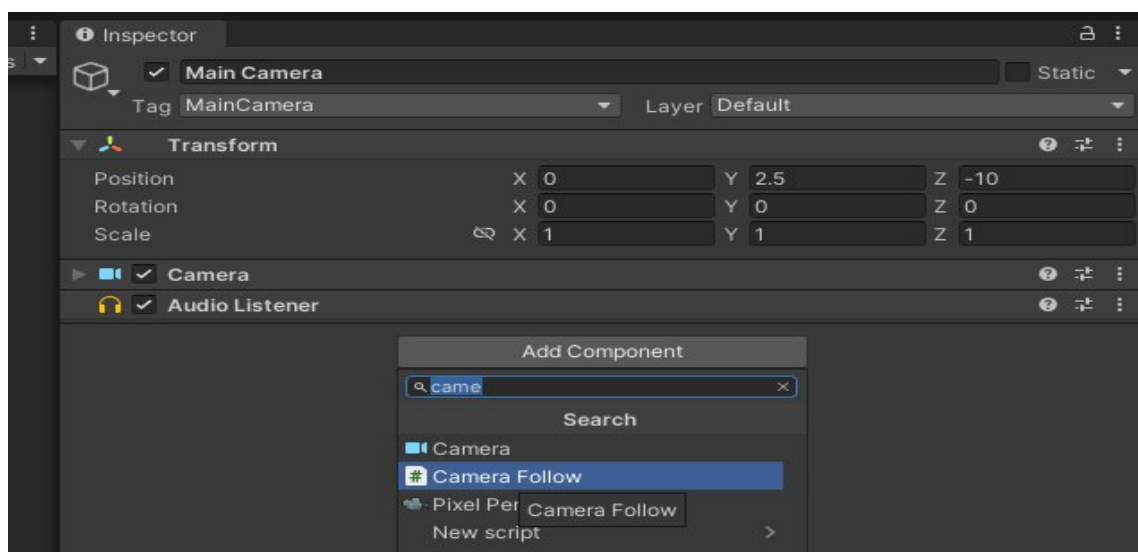
    // Si Mario está a la derecha del punto medio de la cámara
    if (playerPosition.x > cameraMidPointX)
    {

        // Mover la cámara para que siga a Mario
        Vector3 newCameraPosition = transform.position;
        newCameraPosition.x = Mathf.Lerp(transform.position.x,
playerPosition.x, followSpeed);

        // Aplicar la nueva posición de la cámara
        transform.position = newCameraPosition;
    }
}
}

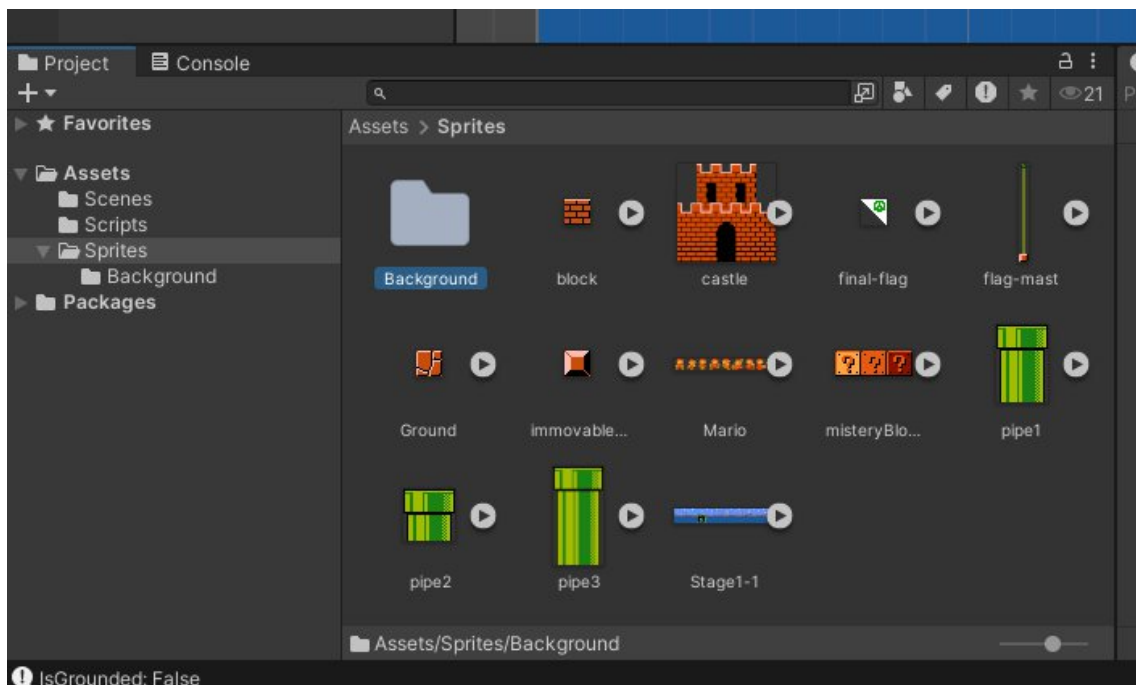
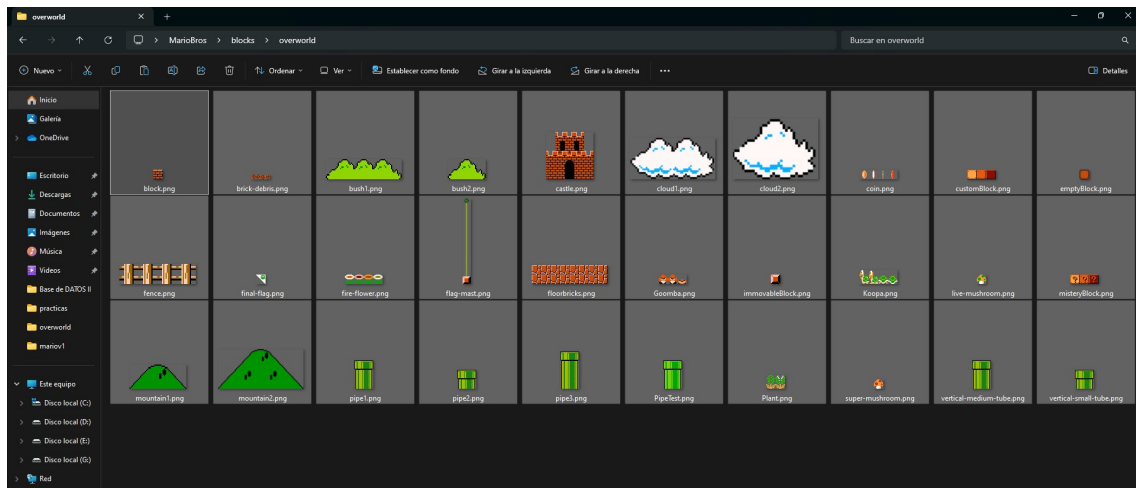
```

Guardamos y Seleccionamos la cámara en la jerarquía y añadimos el Script creado



Una vez tenemos esto configurado tenemos que añadir los objetos al juego siguiendo la plantilla del nivel 1 como modelo

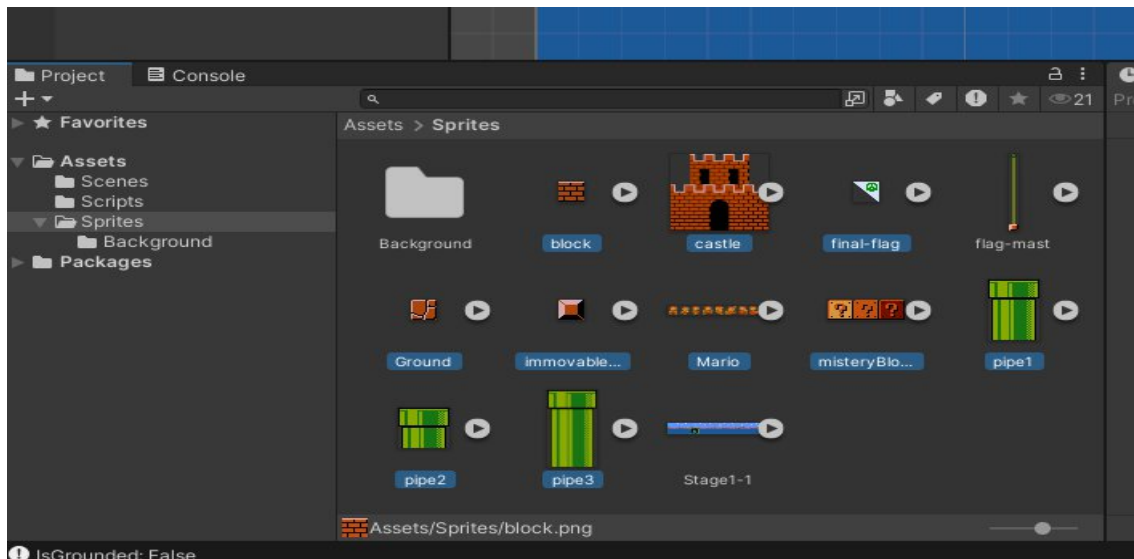
Selecciono los recursos de los Asset descargados y añado lo que necesito a la carpeta Sprite de nuestro proyecto



Separo los sprite en dos categorías.

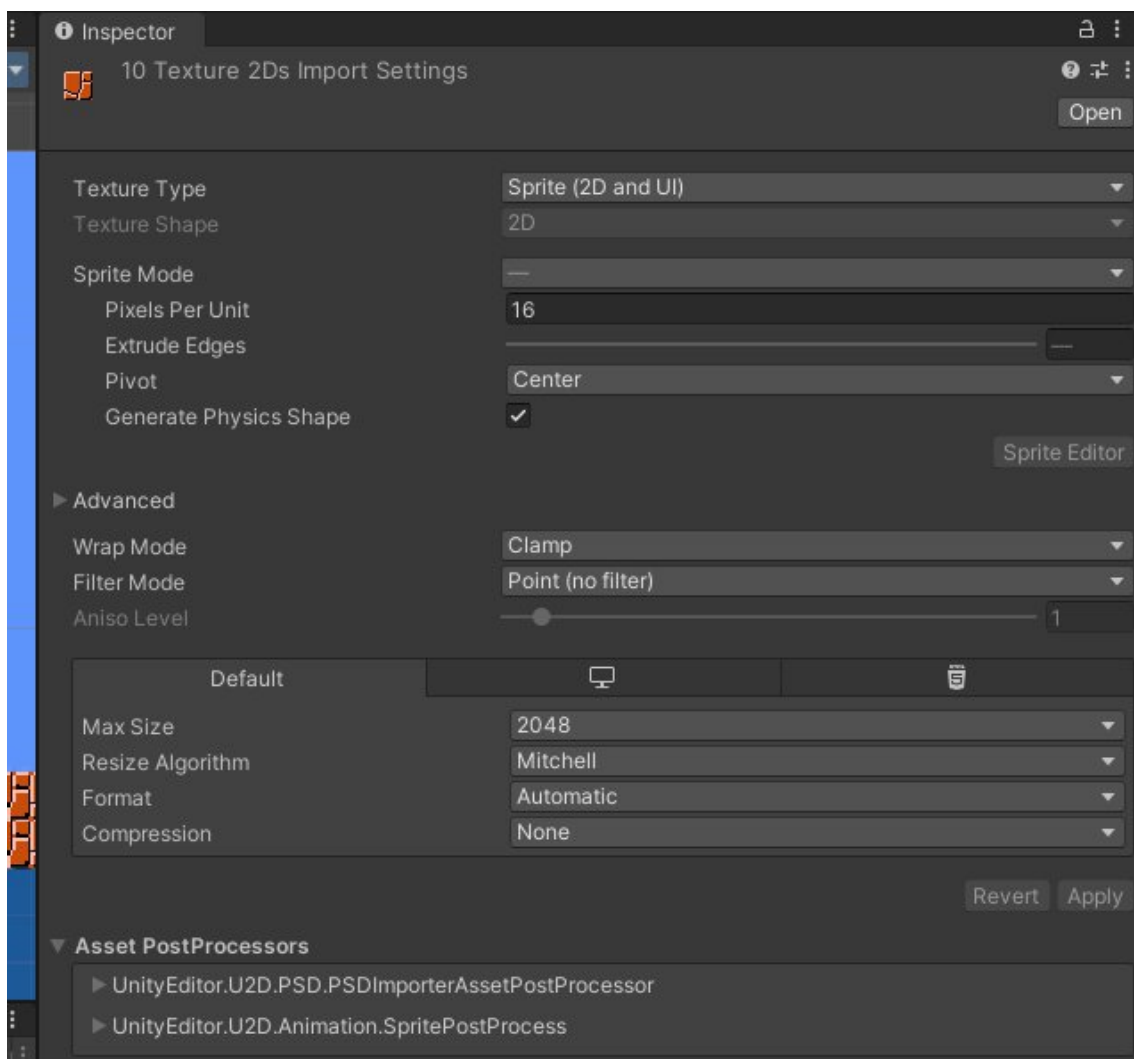
- Los sprite que son parte del fondo con los cuales no tendremos interaccion(Nubes, montañas, Arbustos
- Los sprite que son parte de los objetos con los cuales mario interactuara (enemigos, tubos, hongos, bloques,etc)

Seleciono todos mis sprites y Configuro en el inspector

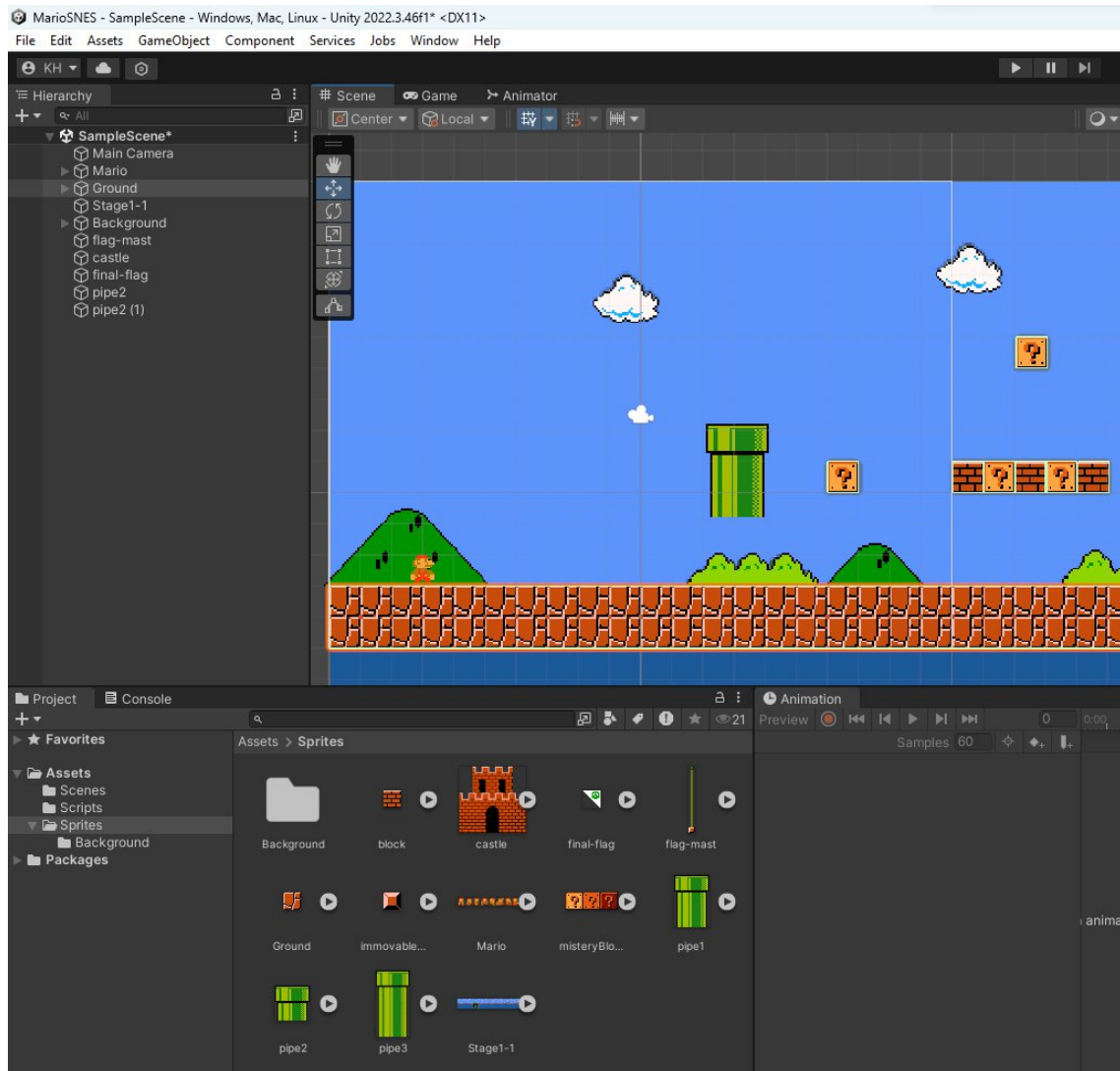


Tenemos que normalizar el tamaño de todos los sprite a nuestra escala, 16 pixeles por unidad

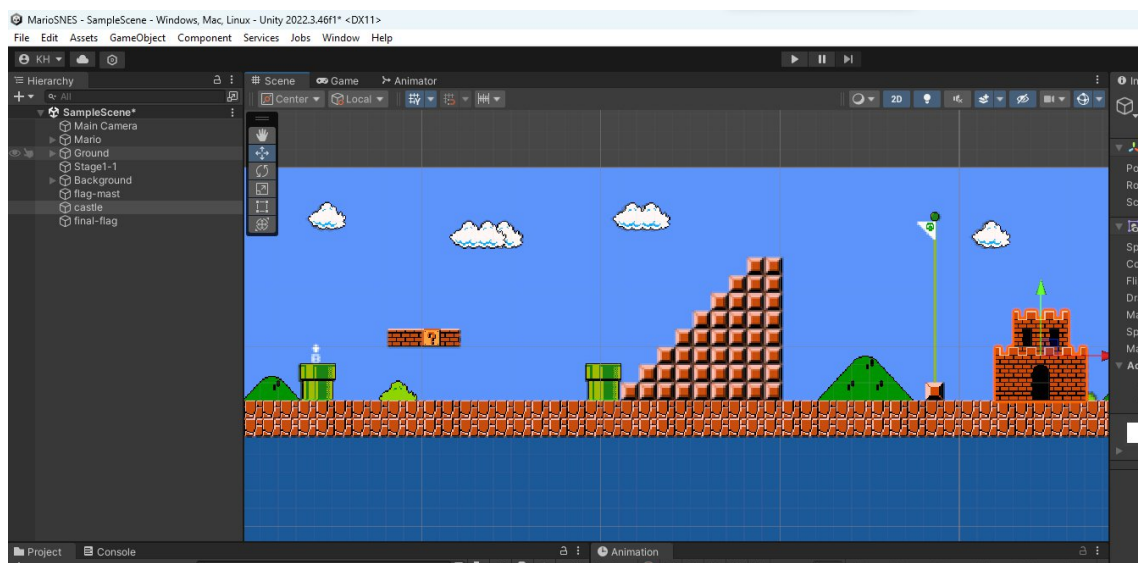
Filter Mode > Point (no filter), Compresion > None

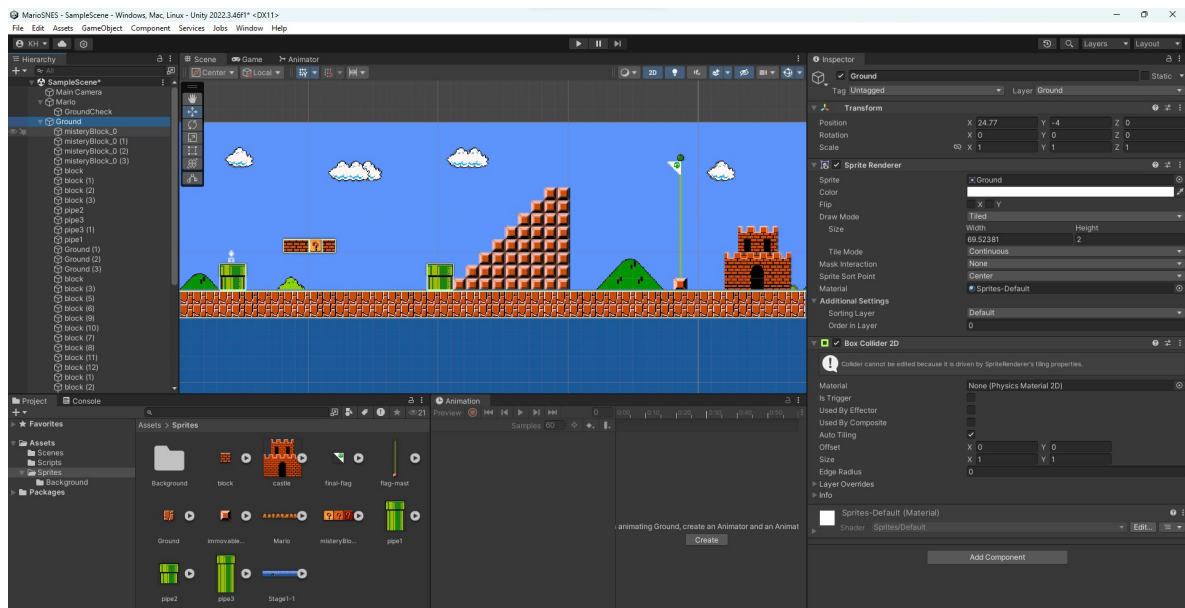


Arrastramos nuestros sprite al Escenario y luego colocamos donde debería ir según la plantilla

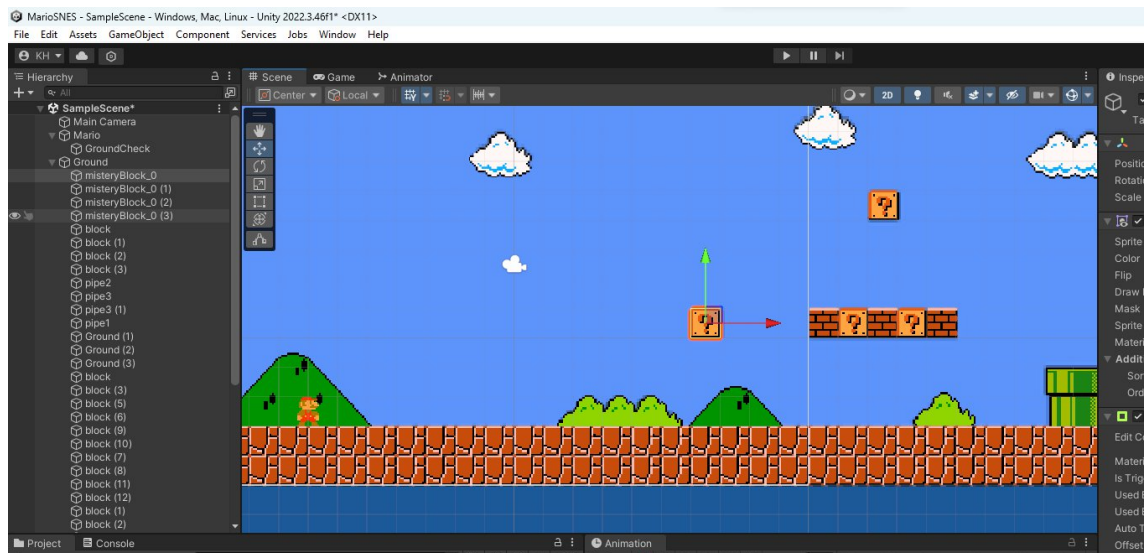


Añaden los sprite necesarios para que creen todo el nivel 1-1

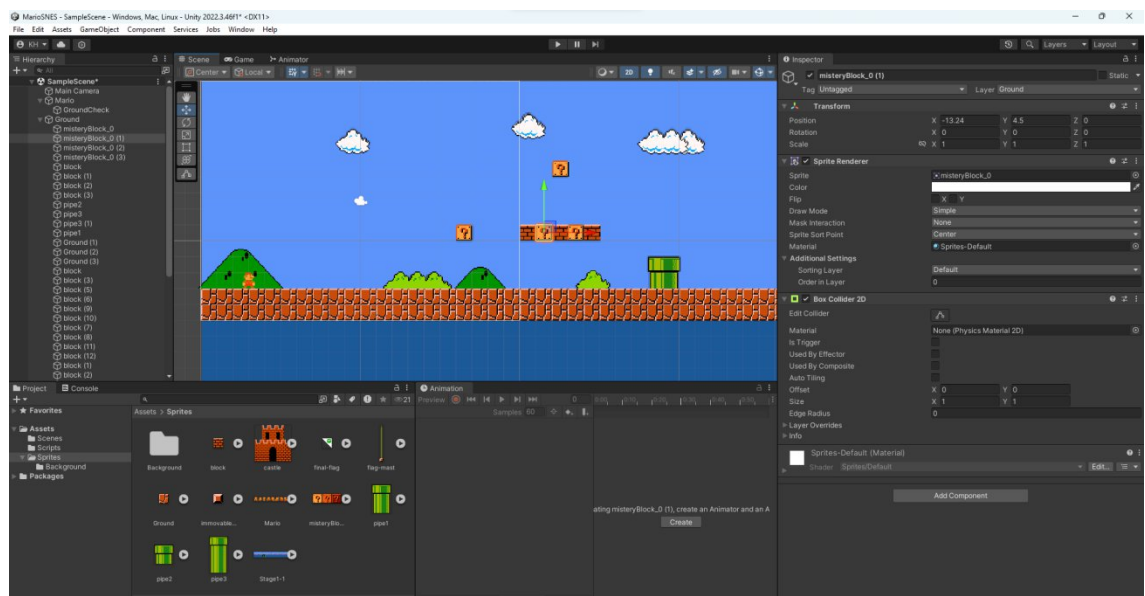




Cuando añades un sprite le tenemos que asignar un Box Collider 2D,



Luego puedes duplicar el objeto que seleccionaste con CTR+D, luego arrastras el objeto donde lo necesites, esto también copia los componentes como los Box collider2D, Ahorrando tiempo



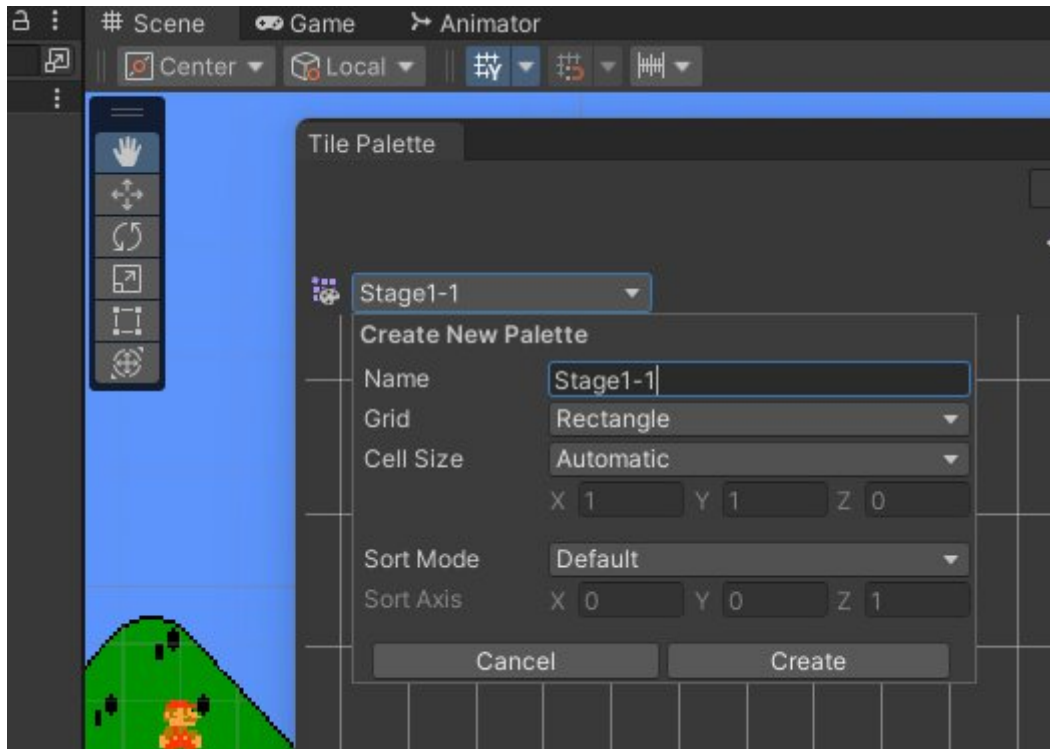
Para evitar usar muchos GameObject en nuestro juego hacemos uso de los Tilemaps

Creamos un nuevo folder con nombre **Tilemaps Assets**

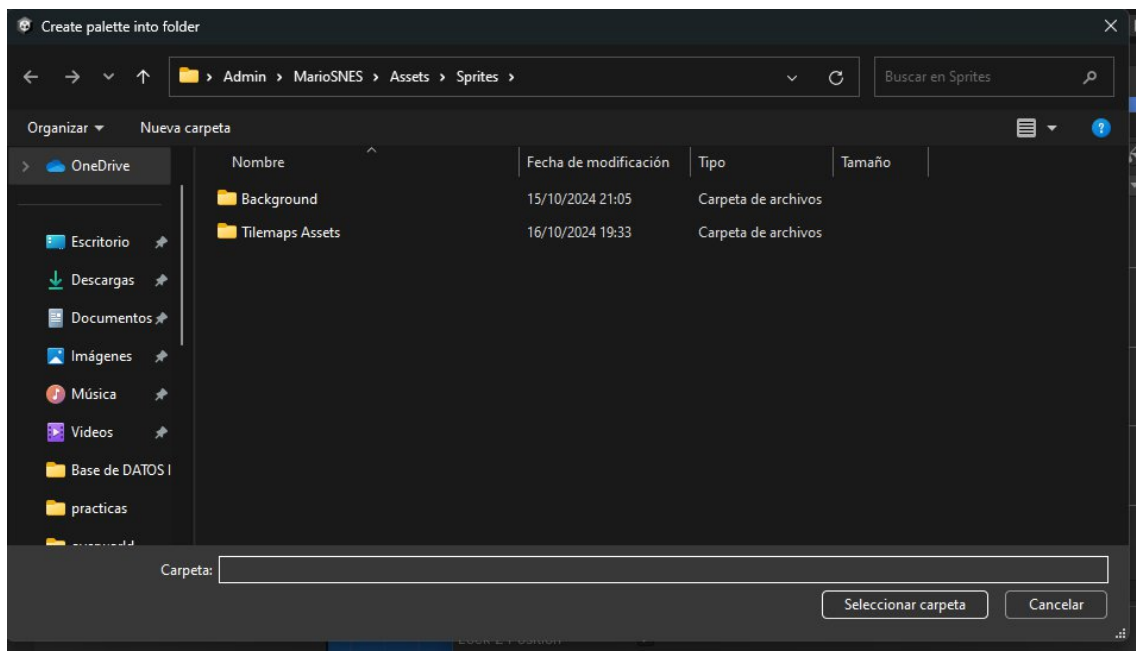
Crear una Tile Palette:

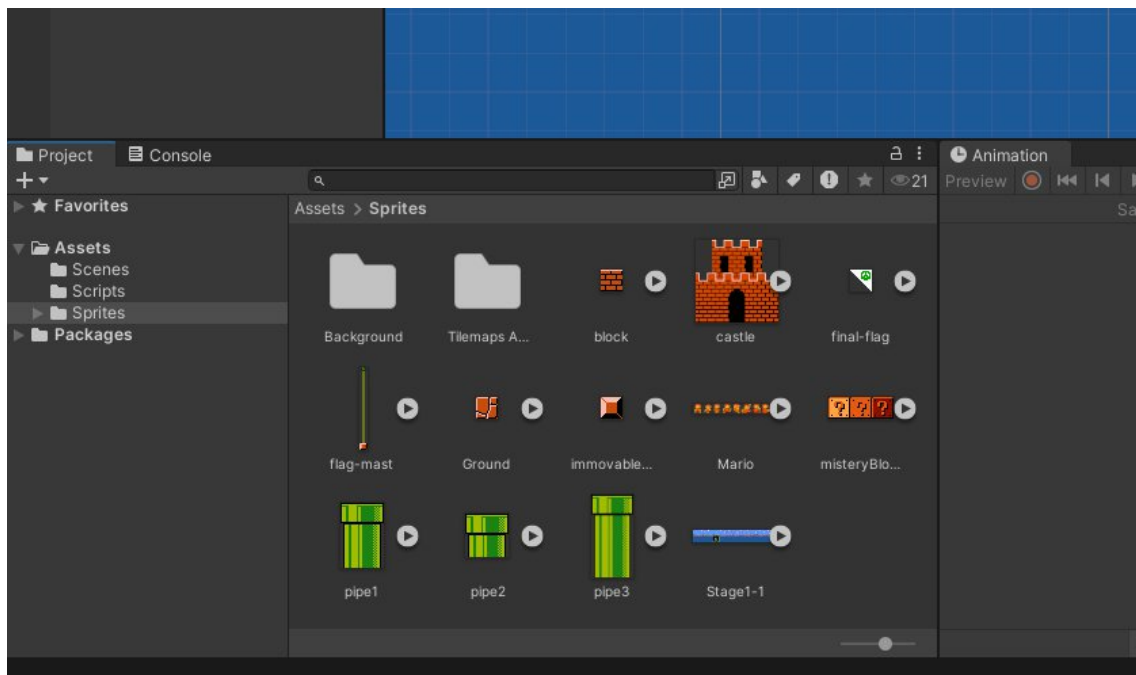
Ve al menú Window > 2D > Tile Palette.

En la ventana de Tile Palette, crea una nueva paleta (Create New Palette),Nombre: Stage1-1

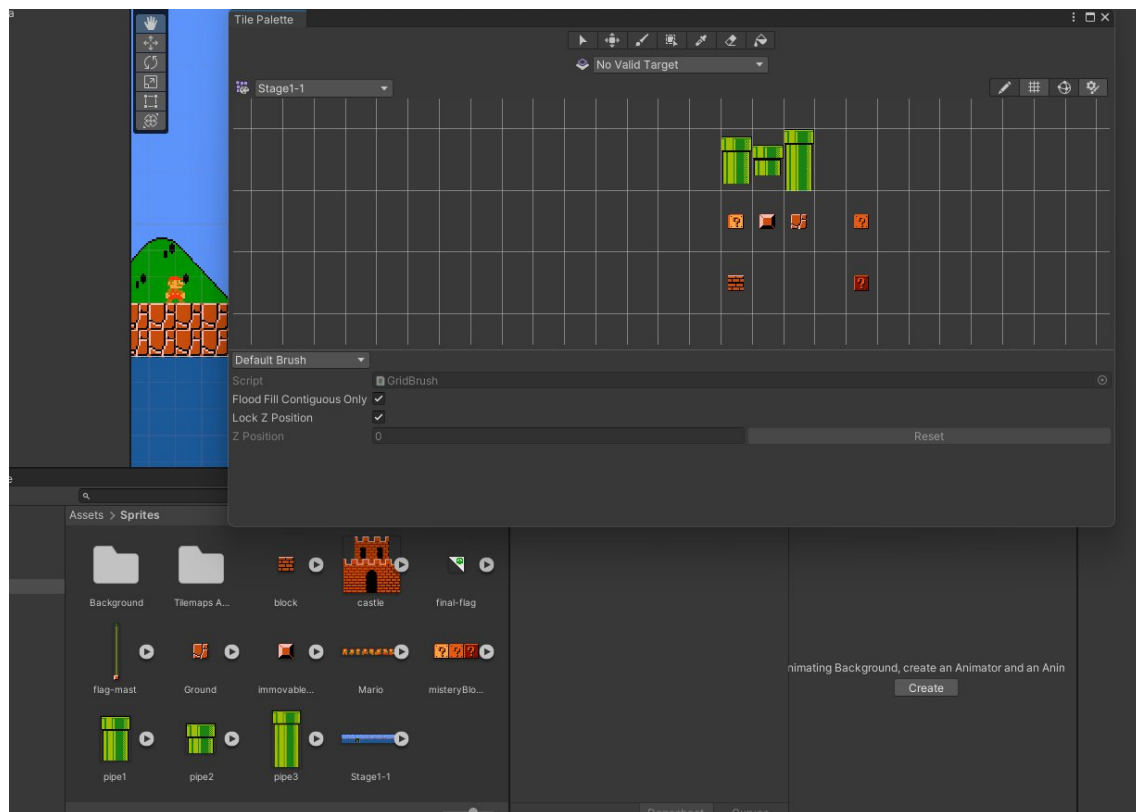


Seleccionamos la carpeta que creamos, esta carpeta contendrá los Tile Palette

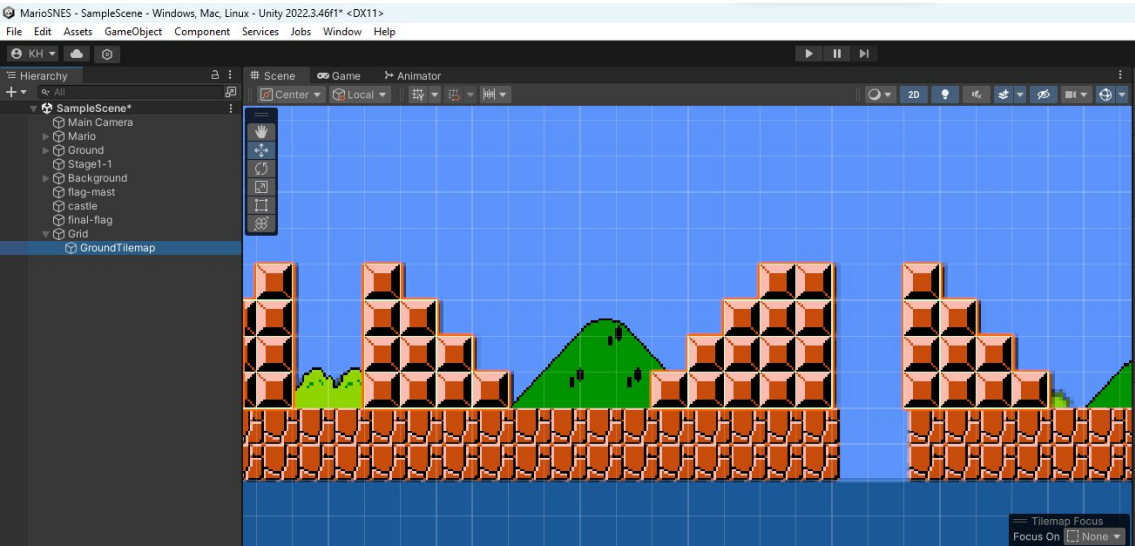
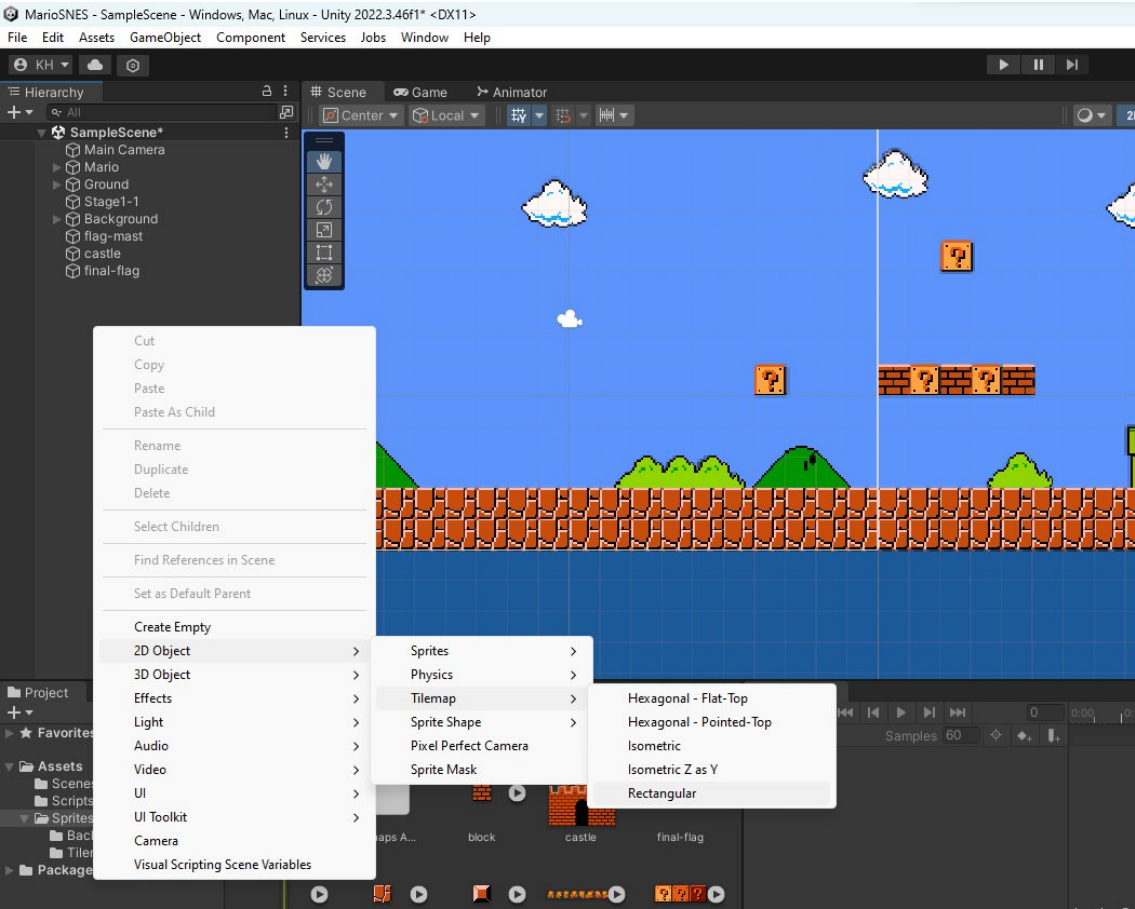




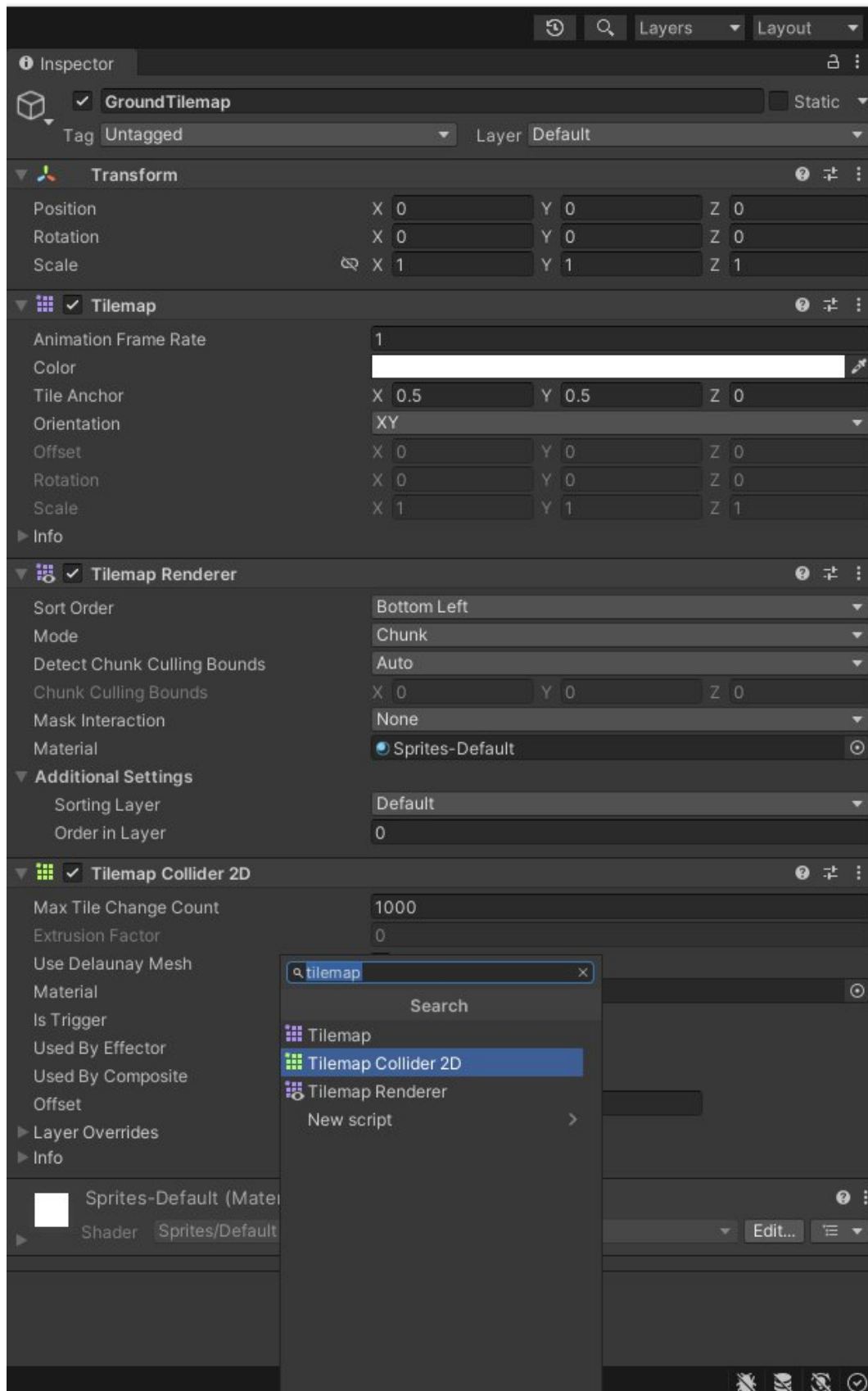
Ahora, simplemente arrastra todos los sprites desde tu carpeta al área de la paleta para crear automáticamente los tiles correspondientes.



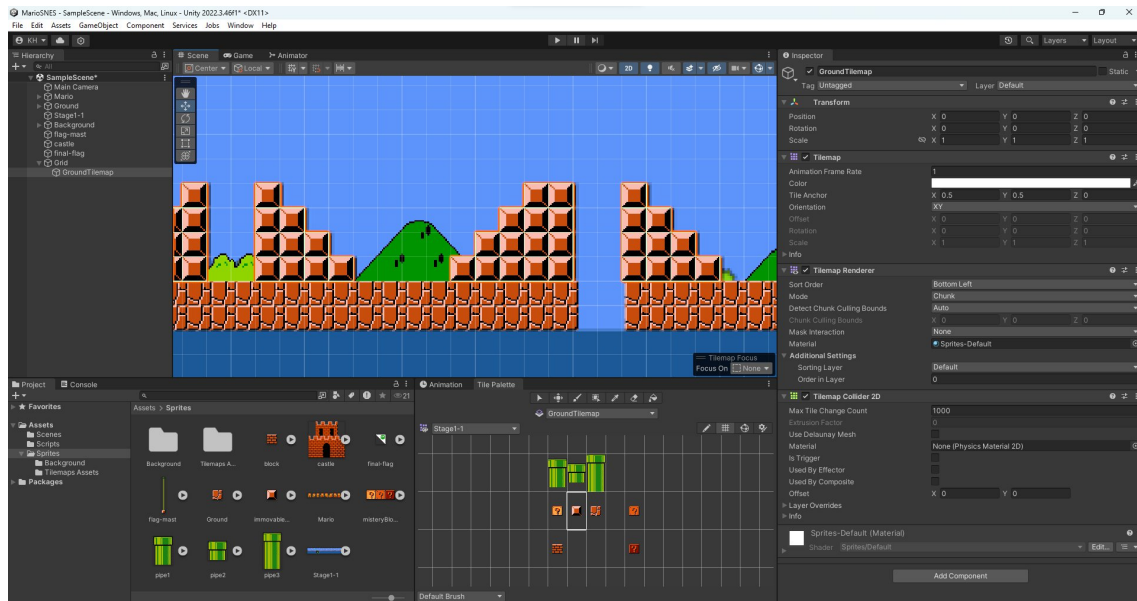
Añadimos un Tilemap Rectangular en nuestra jerarquia



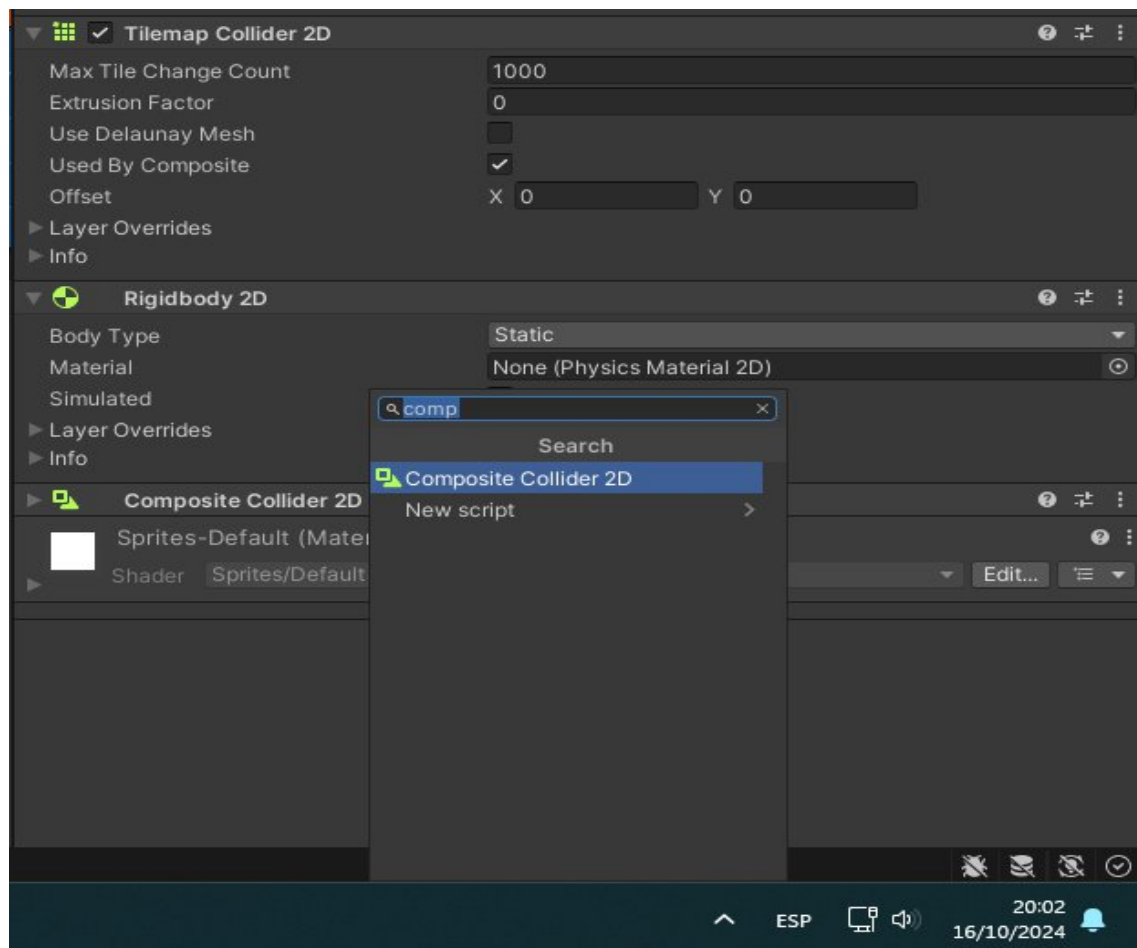
Mientras seleccionamos El GroundTilemap agregamos un Tilemap Collider 2D



Esto hara posible que mario pueda interactuar con los objetos que creamos a partir de nuestro Tilemap

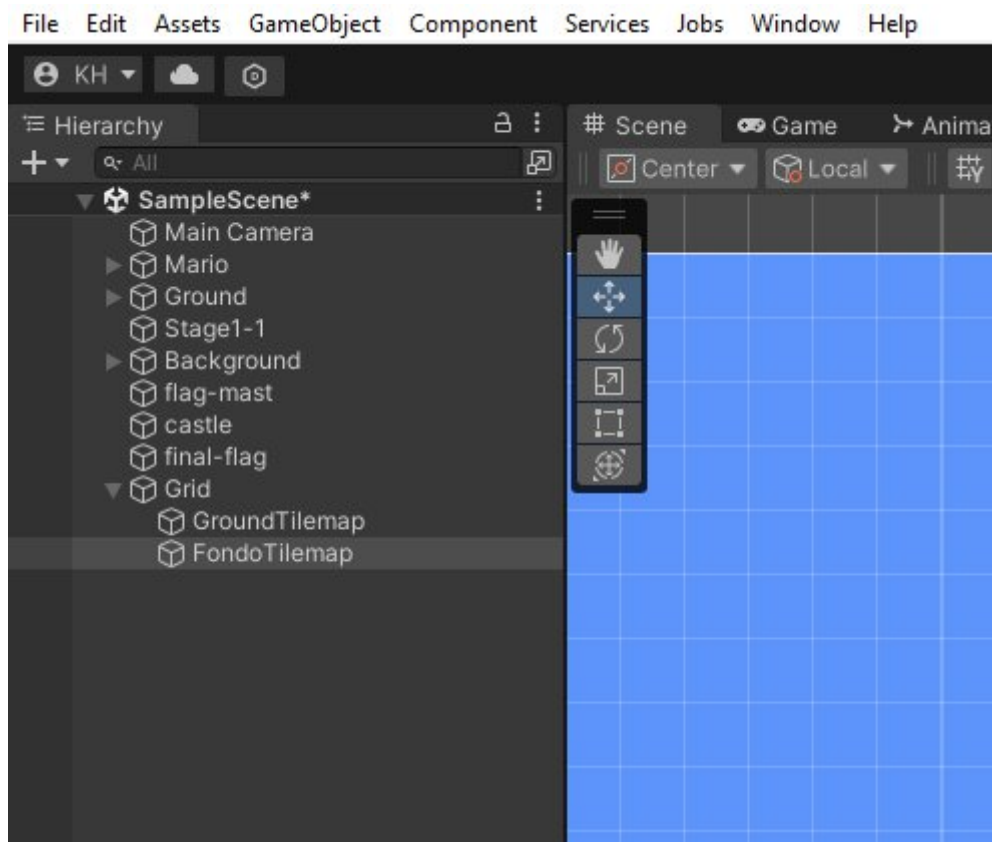


Agregamos Tambien un **Composite Collider 2D**, esto agregara automáticamente un **rididbody2D** el cual poner **Body Type** a **Static** y en **Tilemap Collider2D** marcamos **Used by composite**

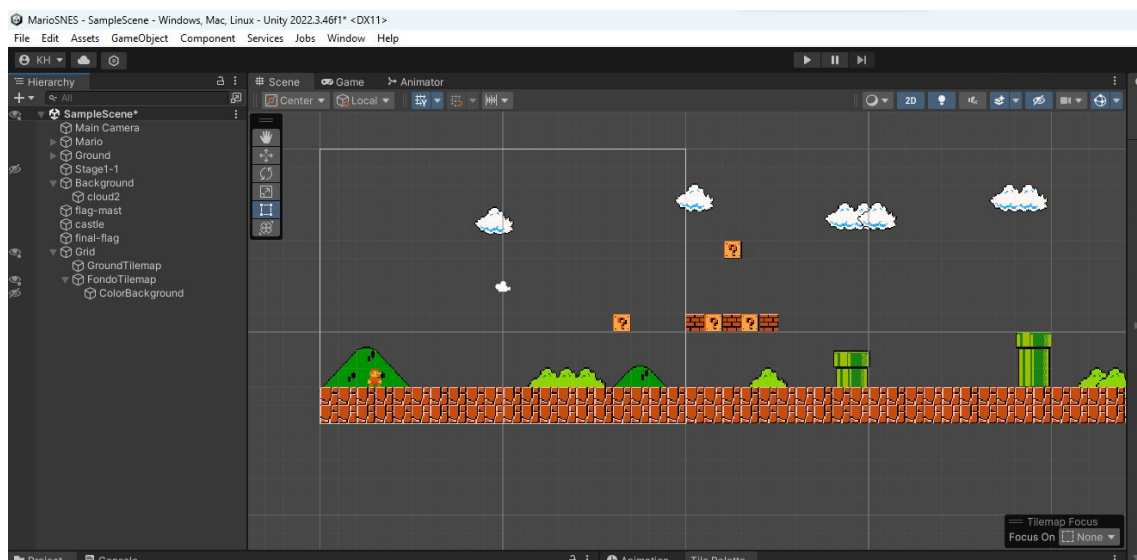


Con esto tendríamos configurado la Tile Palette que van a interactuar con mario, ahora tenemos que agregar otro tilemap para graficar los objetos que no interactúan con el jugador pero son parte del escenario

Creamos un nuevo tilemap llamado FondoTilemap

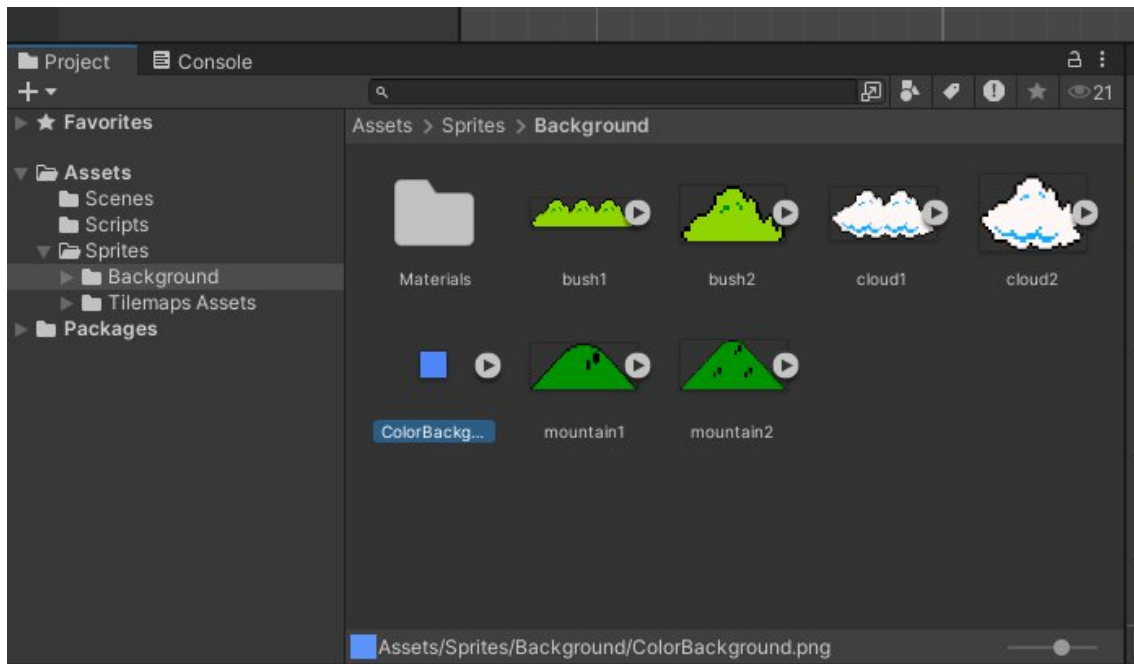


Desactivamos la vista de Stage1-1 para ver como esta nuestro nivel



Ya teniendo todos los objetos dibujados en el escenario solo faltaría agregar el color de fondo

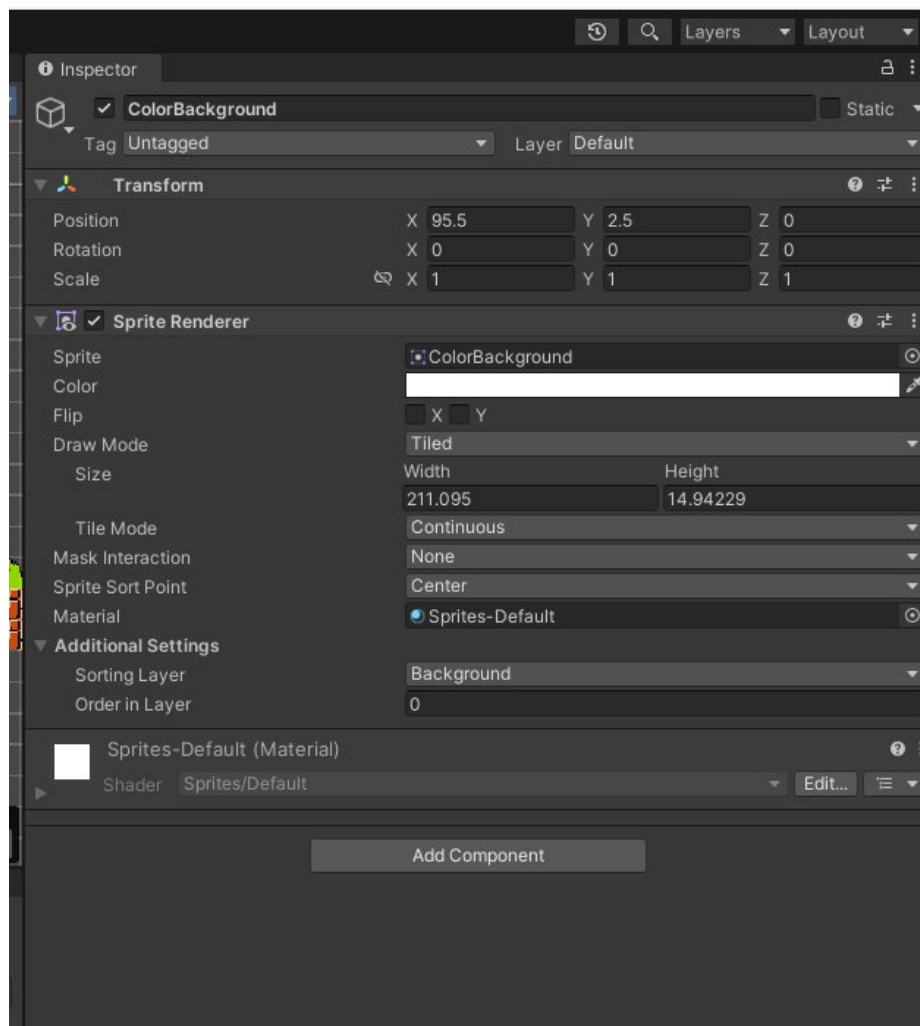
Agregamos un Sprite del mismo color que el fondo original de mario Bros 1985



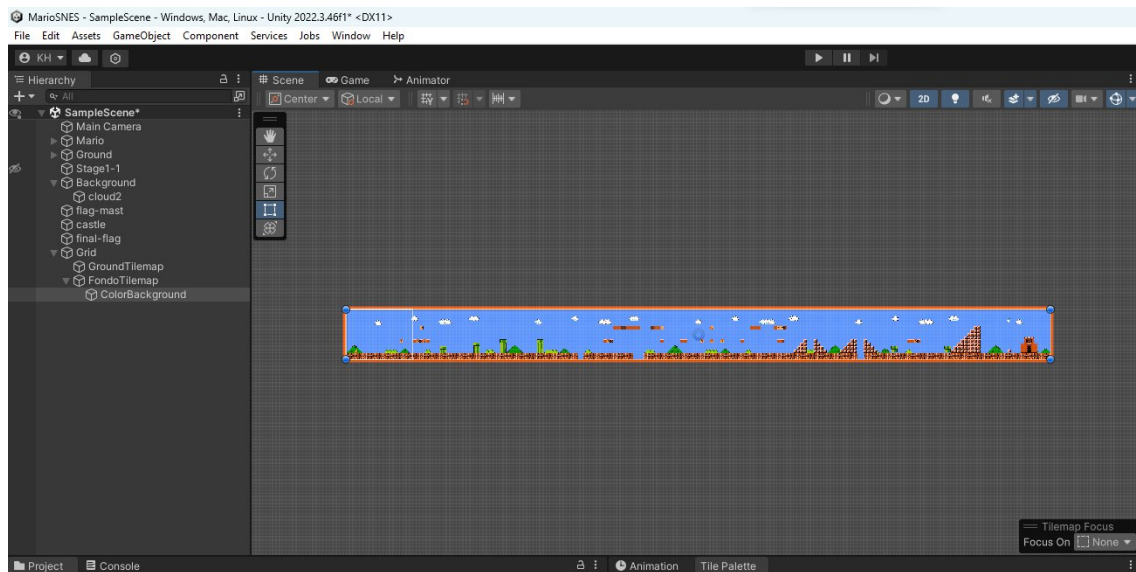
Luego lo añadimos al escenario y configuramos como 16 en pixeles por unidad

Mesh type Full rect

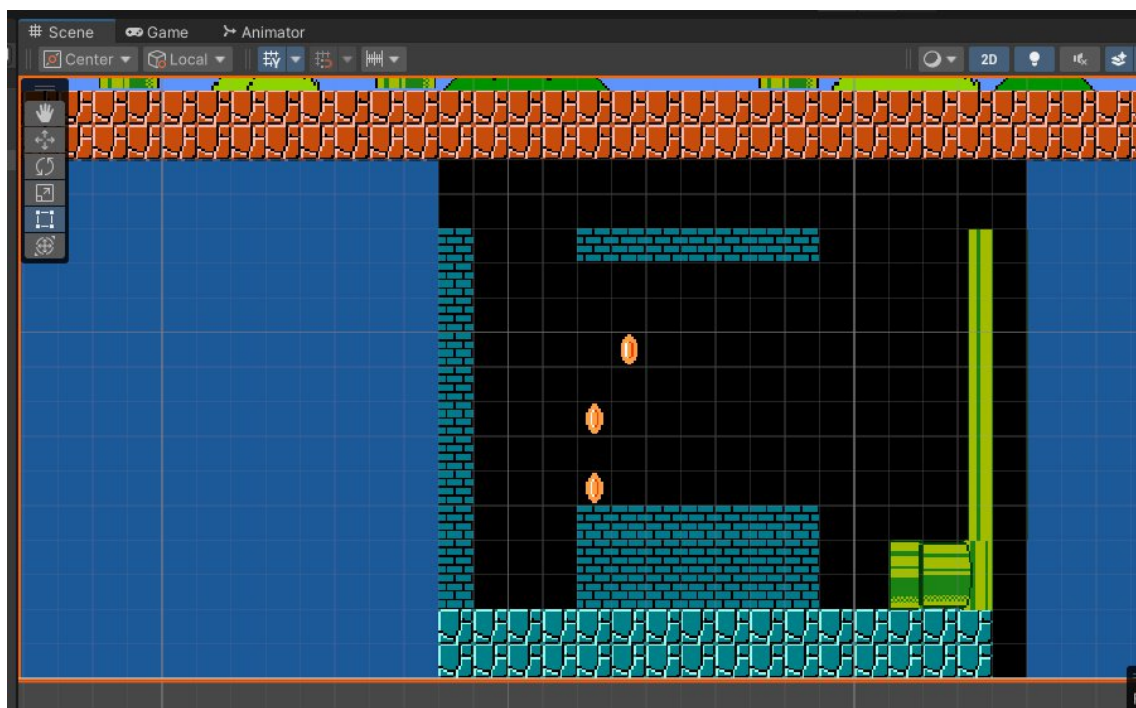
En el inspector del objeto añadido configuramos Drawn Mode en Tiled y Tile Mode en Continuo



Ahora Seleccionamos el sprite del fondo y Expandimos hasta cubrir todo el escenario Tambien cambiamos su Sorting Layer a Background para que se dibuje al final de todo



Ahora recreamos el mundo Underworld cuando mario baja por la tubería



Luego de completado los sprite del primer Stage de mario bros, nos quedaría a gregar enemigos, colisiones con enemigos, colocar triggers para cuando mario caiga al vacio y animar a mario con los sprites

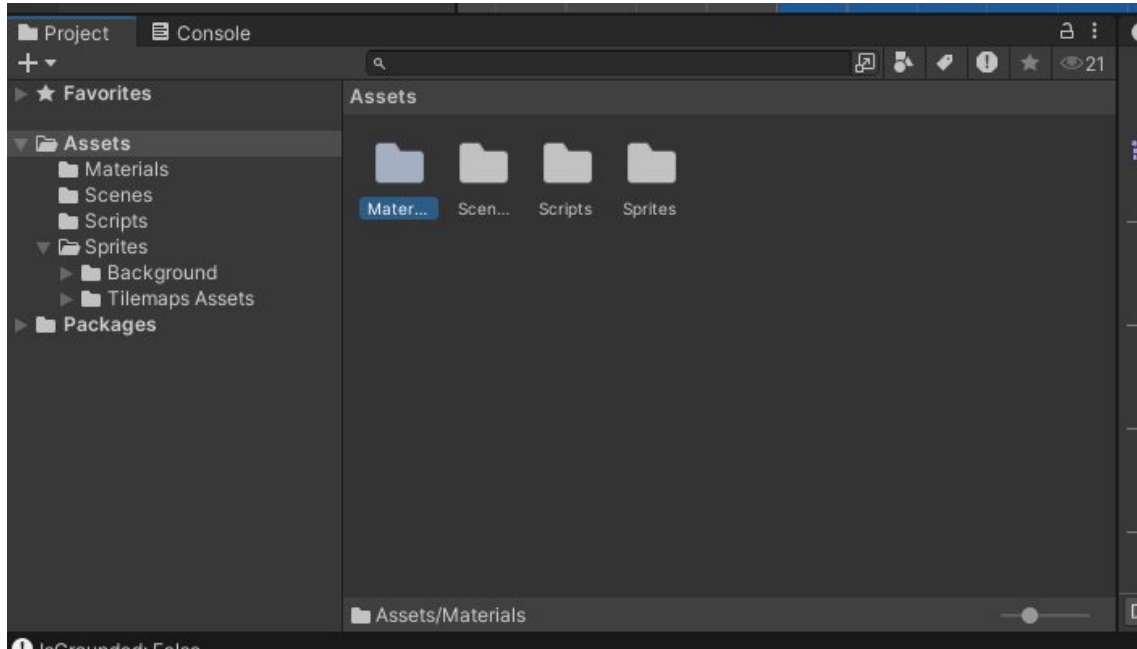
Siguiendo la primera guía de videojuegos añadimos las animaciones de caminar para mario

FIN PARTE 2...

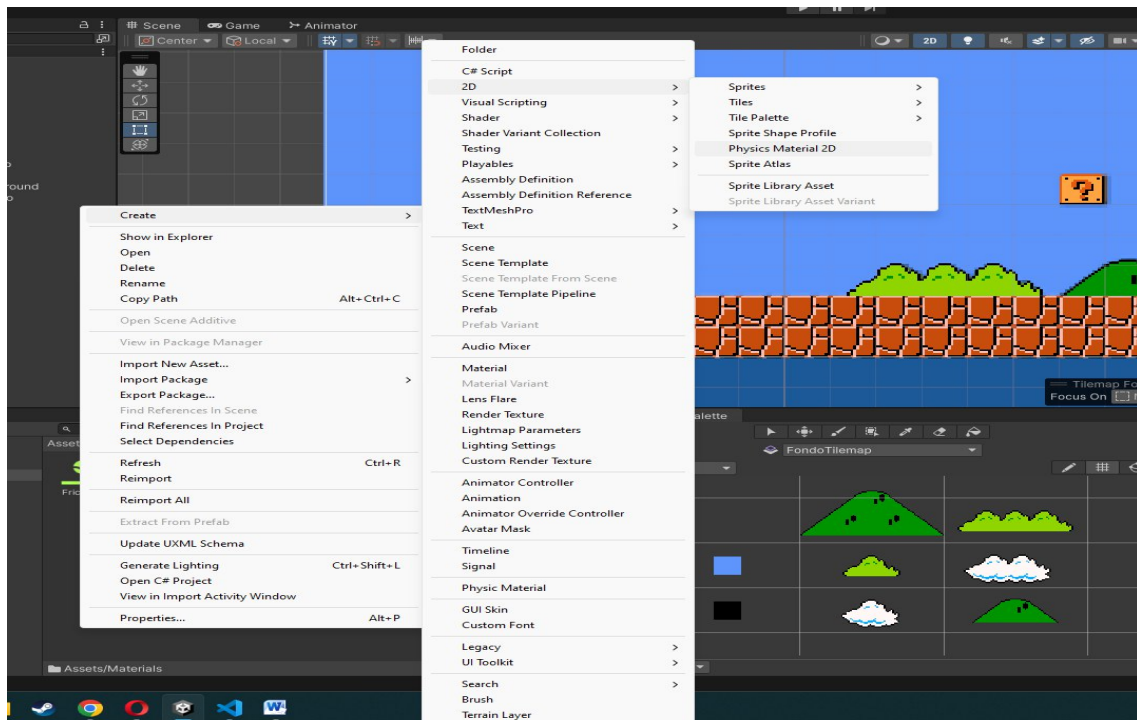
PARTE 3

Asignaremos Fisicas a nuestros materiales

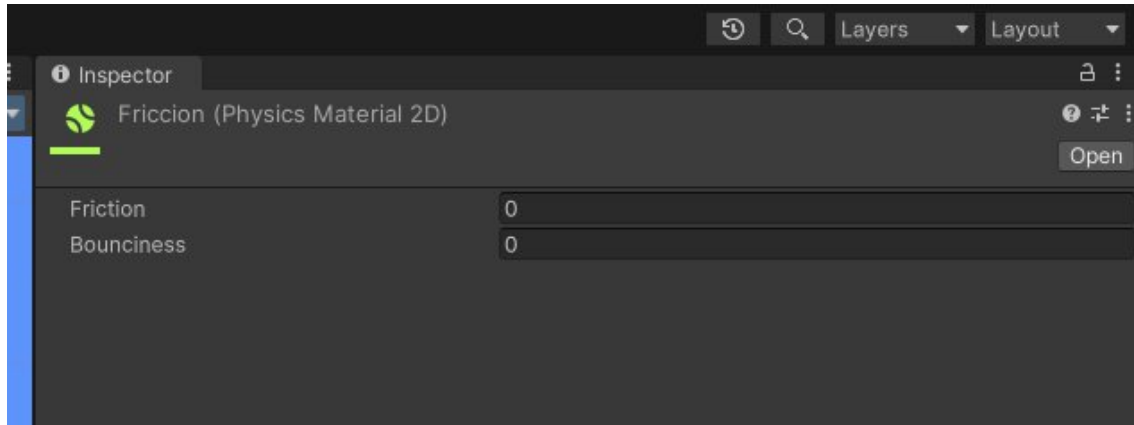
Creamos un nuevo Folder llamado Materials a nuestro proyecto



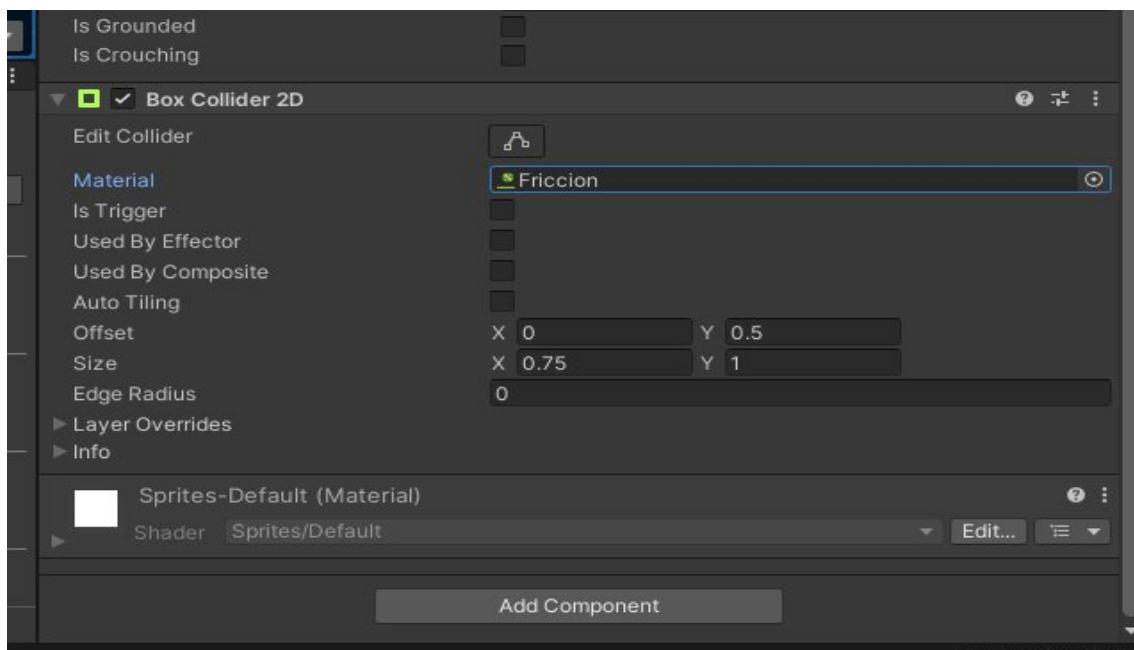
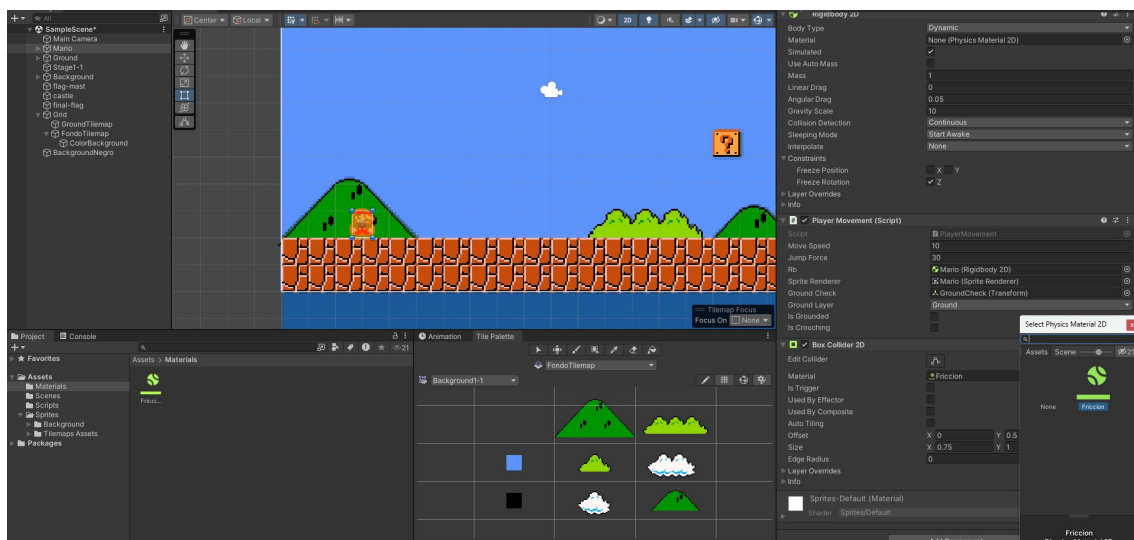
Clic derecho Create>2D>Physics Material 2D



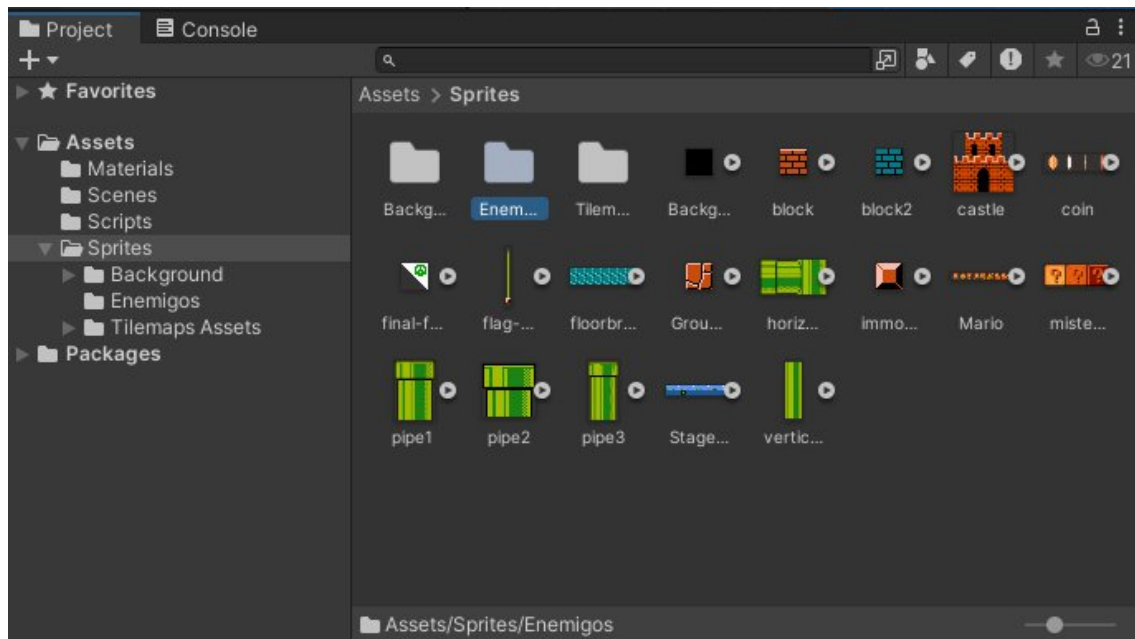
Nombramos como **Friccion** y en Friction set 0



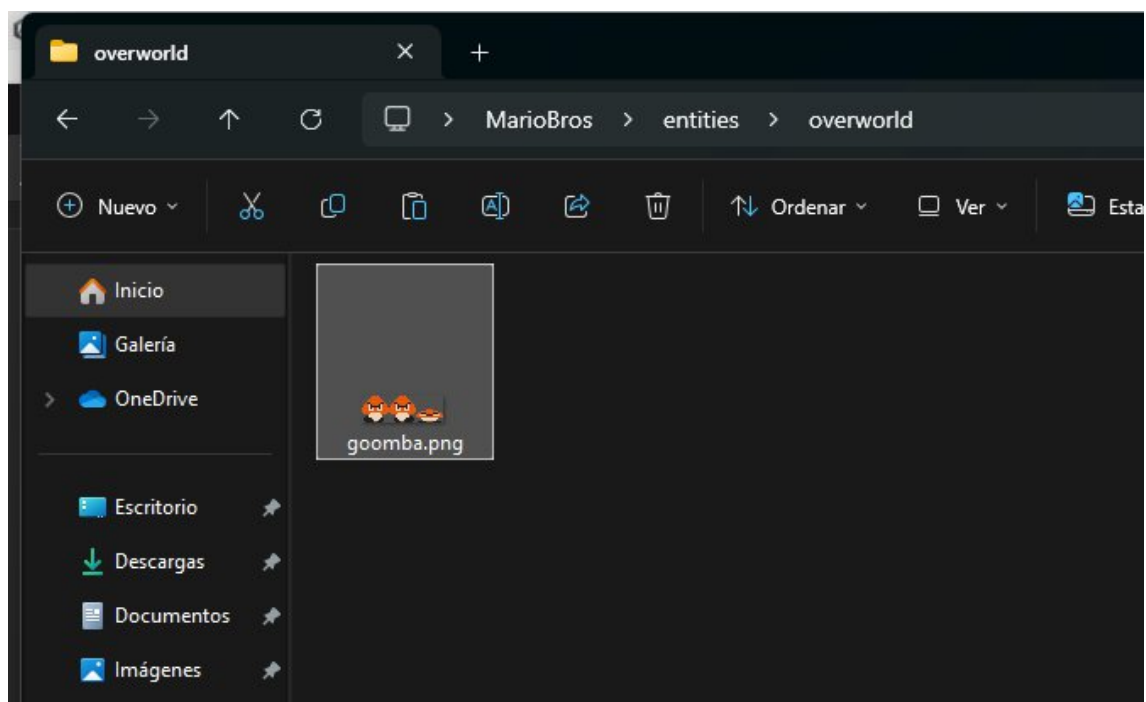
Mientras seleccionan a mario en la jerarquía, en Box Collider 2D Material y seleccionan Friccion



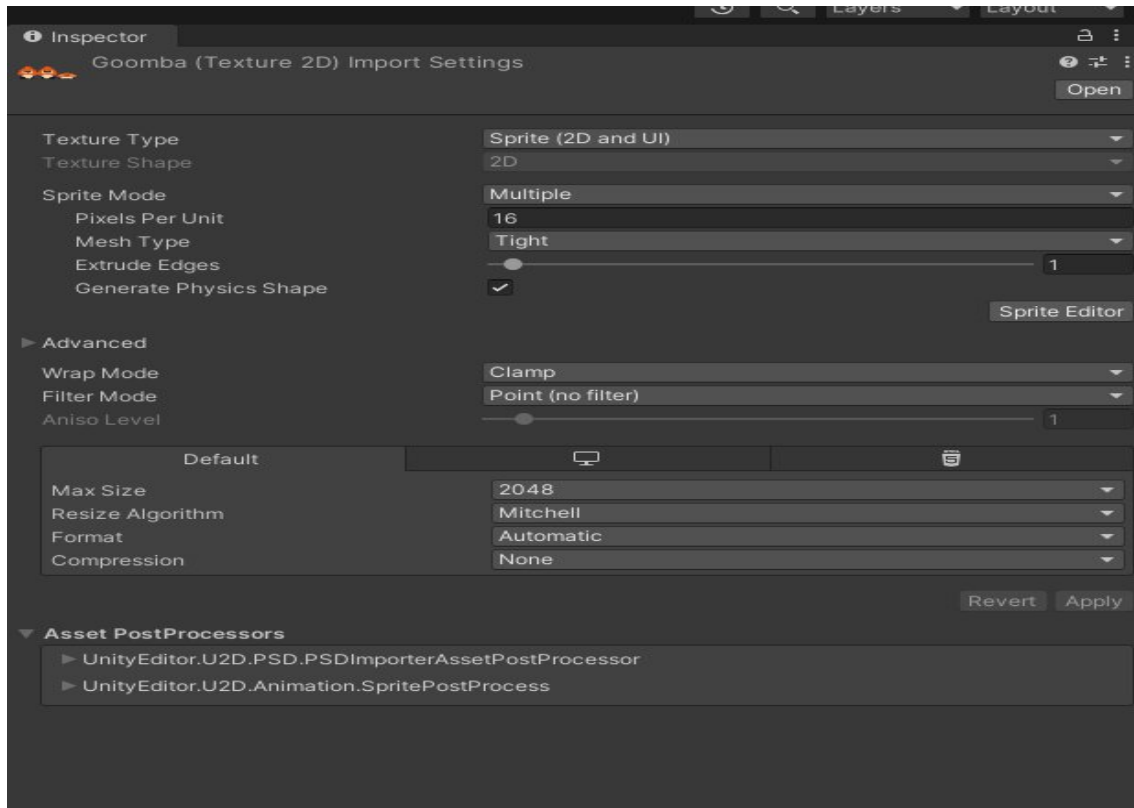
Creamos una carpeta con nombre Enemigos



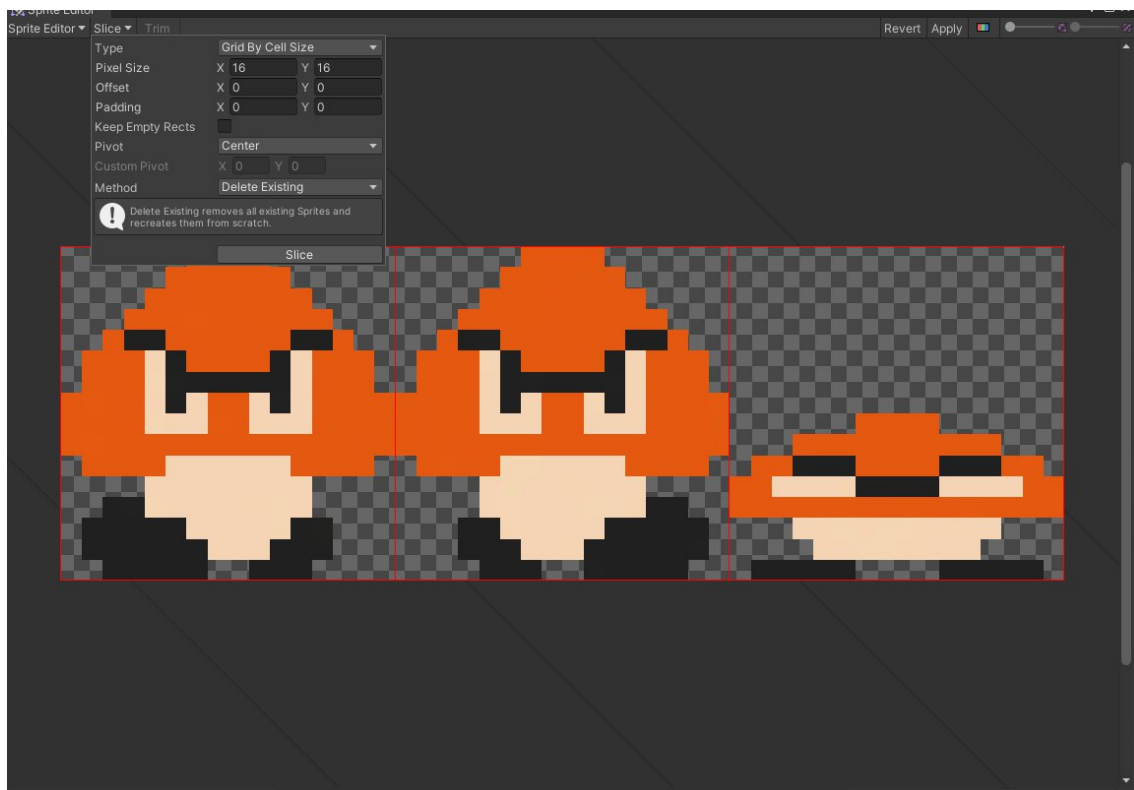
En la carpeta de nuestros recursos Entramos en Entities>overlord y agregamos el goomba a nuestra carpeta Enemigos



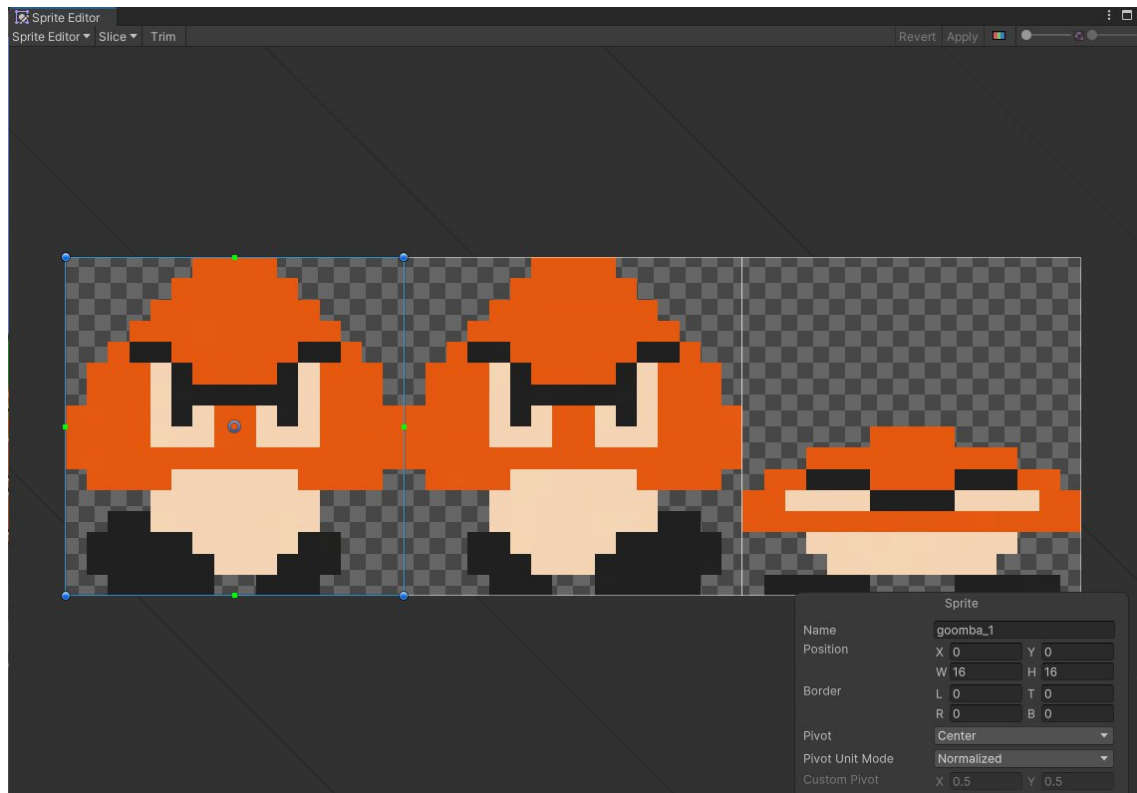
En el inspector cambiamos sus valores pixel per unit a 16, multiple, compresión none y filter mod > point no filter



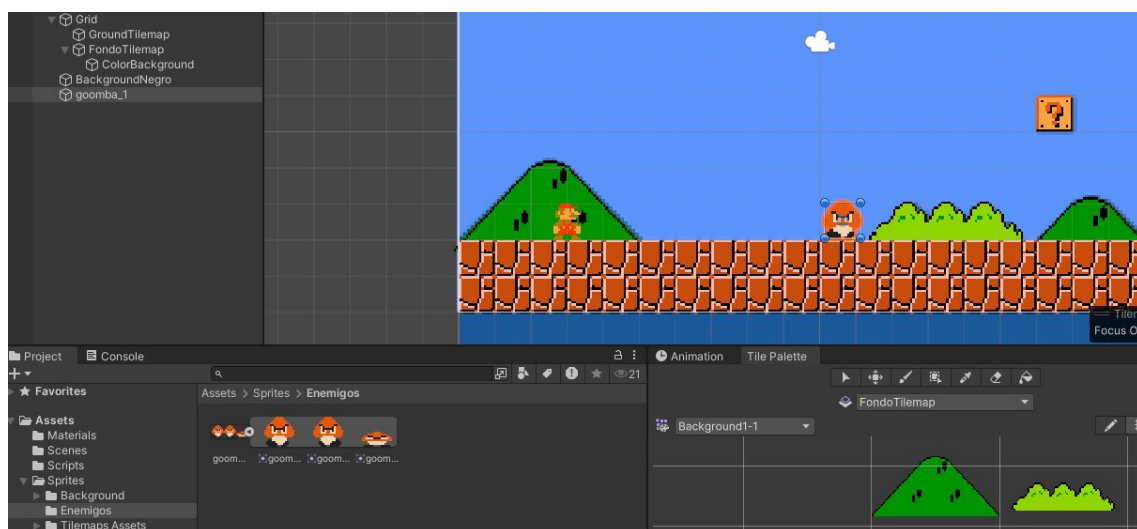
Entramos a sprite editor y cortamos 16x16



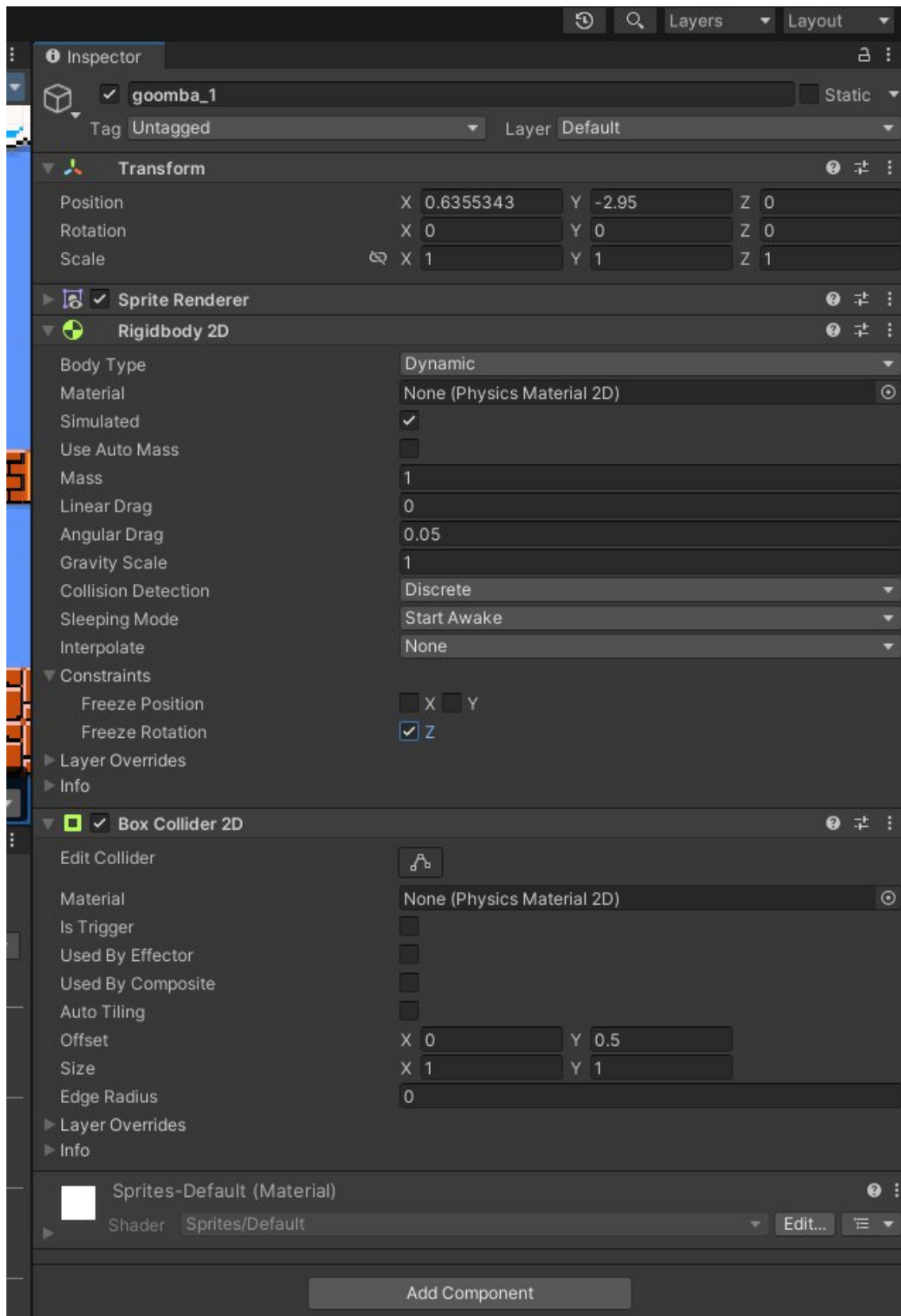
Editamos los nombres a goomba_1,2 y die, pivot en Center



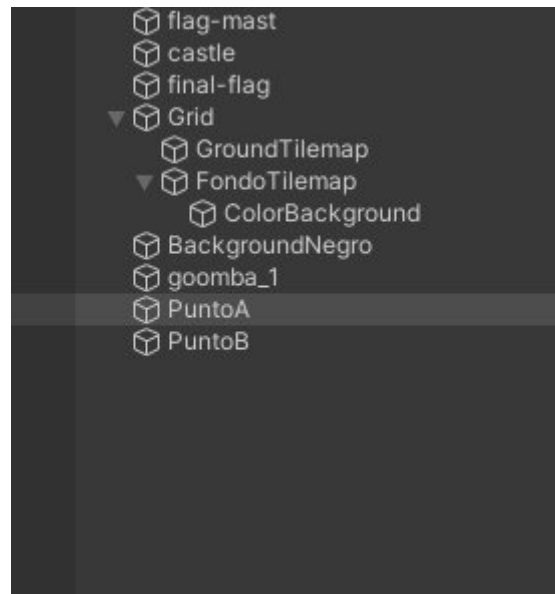
Agregamos el enemigo a nuestra escena



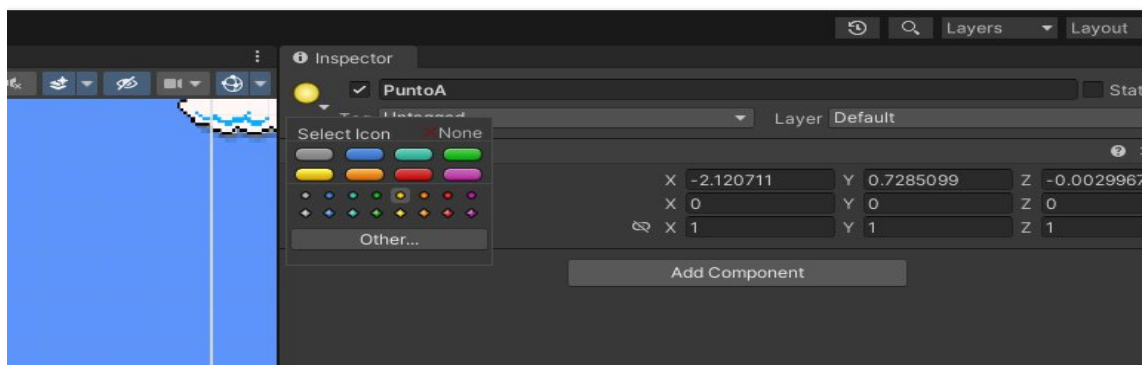
Agregamos un Rigidbody2d y un collider 2d, Freeze Rotattion check



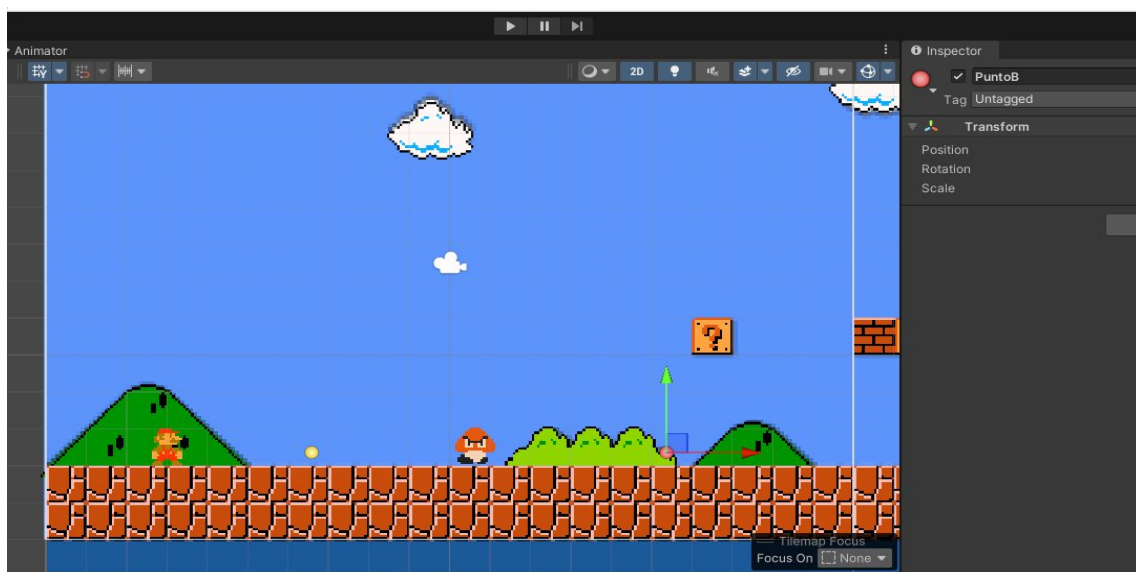
Agregamos dos objetos vacios a la jerarquía



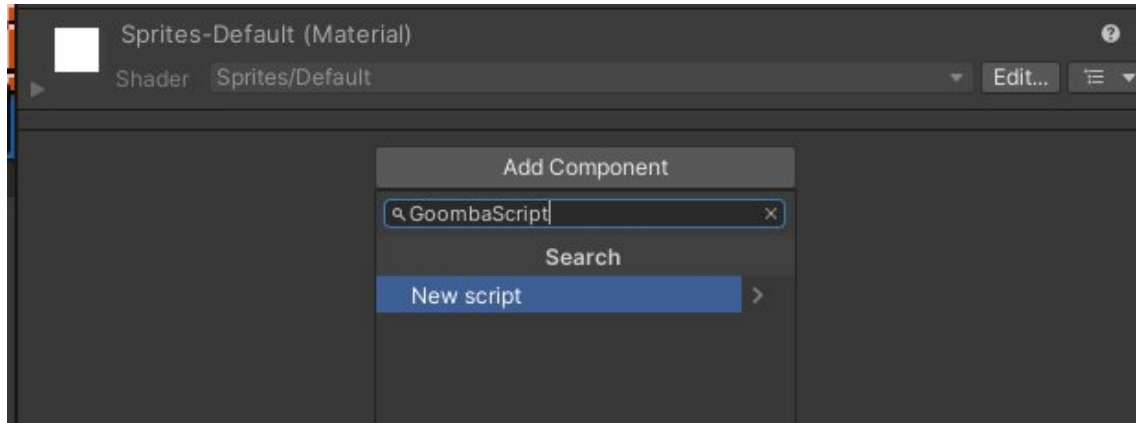
En el objeto PuntoA le agregamos un Icono punto verde



Agregamos un icono rojo para el punto B y situamos los puntos como en la imagen



Agregamos un nuevo Script llamado GoombaScript



Dentro del Script agregamos el siguiente código:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class GoombaScript : MonoBehaviour
{
    public Transform PuntoA;
    public Transform PuntoB;
    public float Speed;

    public bool MoveToA = false;
    public bool MoveToB = false;

    private Rigidbody2D MyRb;

    // Start is called before the first frame update
    void Start()
    {
        MyRb = GetComponent<Rigidbody2D>();
        MoveToA=true;
    }

    // Update is called once per frame
    void Update()
    {
        if (MoveToA)
        {
            MyRb.transform.position =
            Vector2.MoveTowards(transform.position, PuntoA.position, Speed *
            Time.deltaTime);

            if (transform.position == PuntoA.position)
            {
                MoveToA = false;
                MoveToB = true;
            }
        }
    }
}
```

```

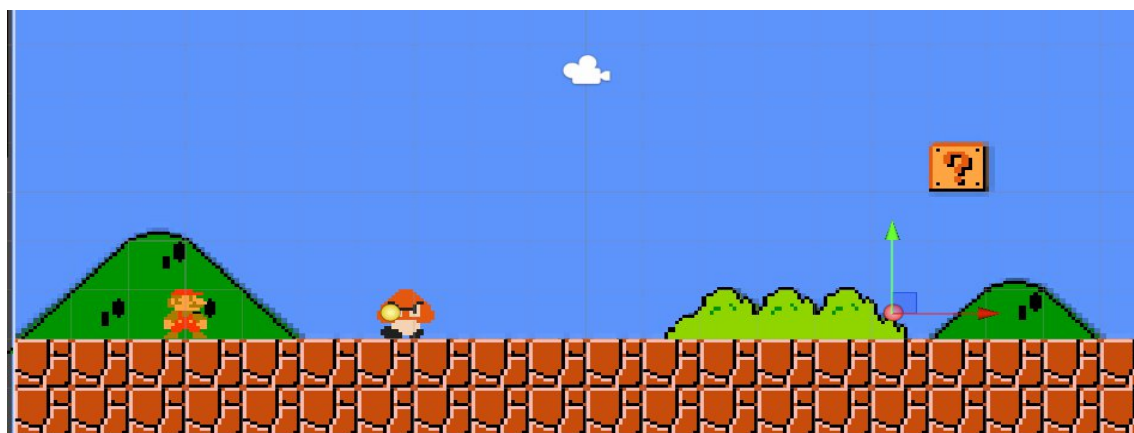
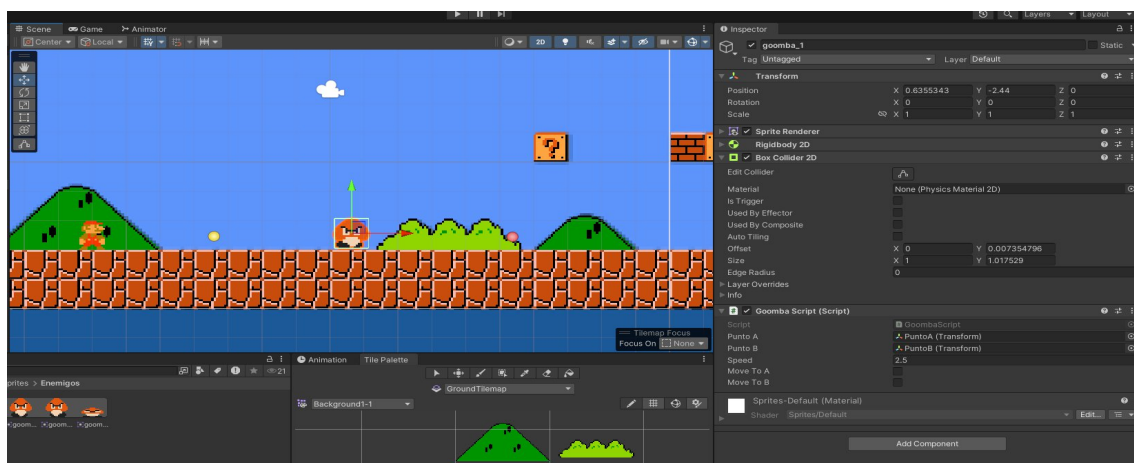
    }
}

if (MoveToB)
{
    MyRb.transform.position =
Vector2.MoveTowards(transform.position, PuntoB.position, Speed *
Time.deltaTime);

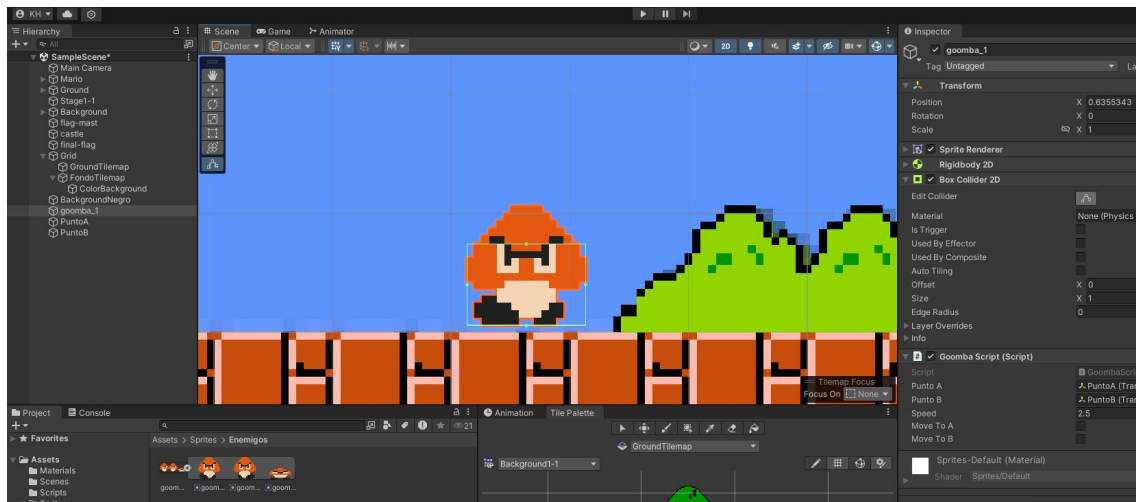
    if (transform.position == PuntoB.position)
    {
        MoveToA = true;
        MoveToB = false;
    }
}
}
}

```

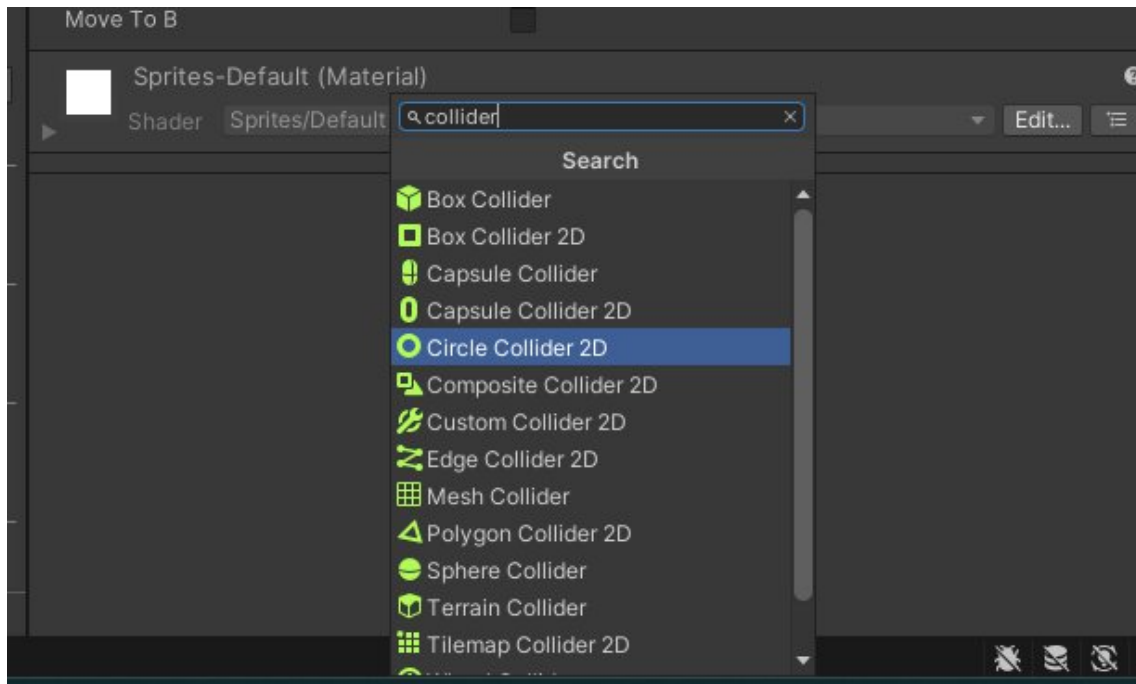
En el inspector ponemos el valor de velocidad a 2.5 y ejecutamos el juego para comprobar el movimiento



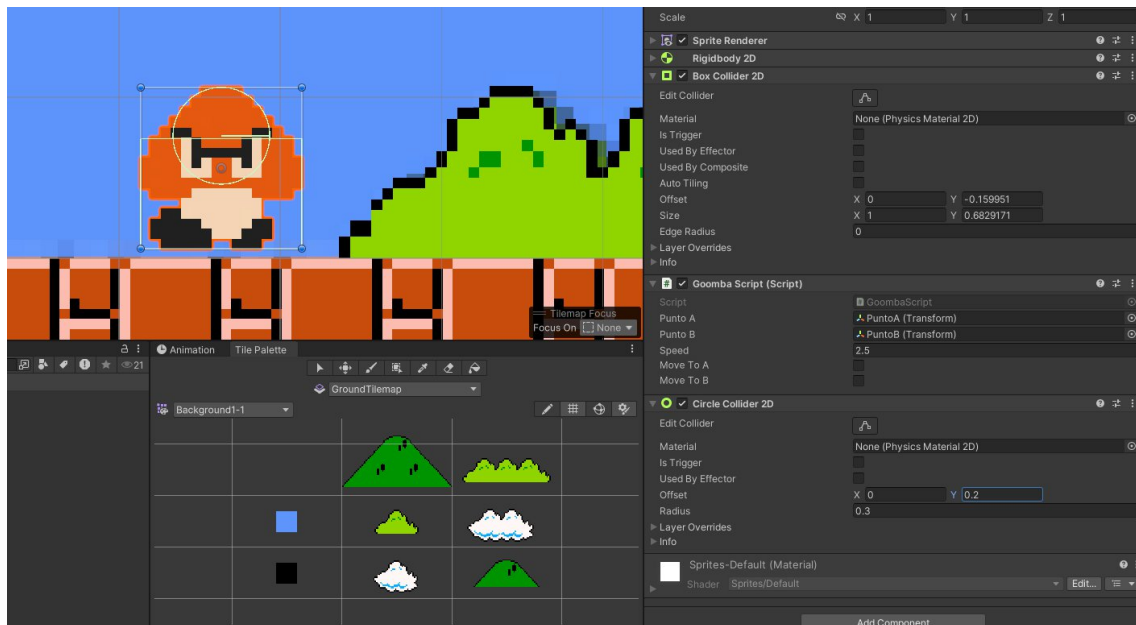
En nuestro objeto goomba_1 editamos su collider box dándole clic al botón y lo redimensionamos



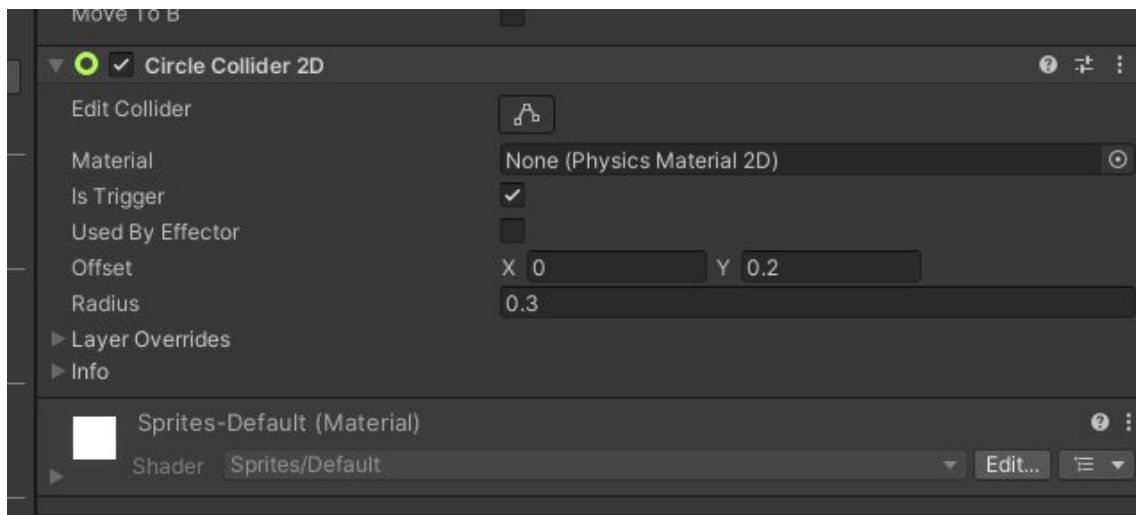
Agregamos un circle collider



Editamos el Circle Collider, Radio a 0.3 y el con el offset lo posicionamos en la cabeza del Goomba



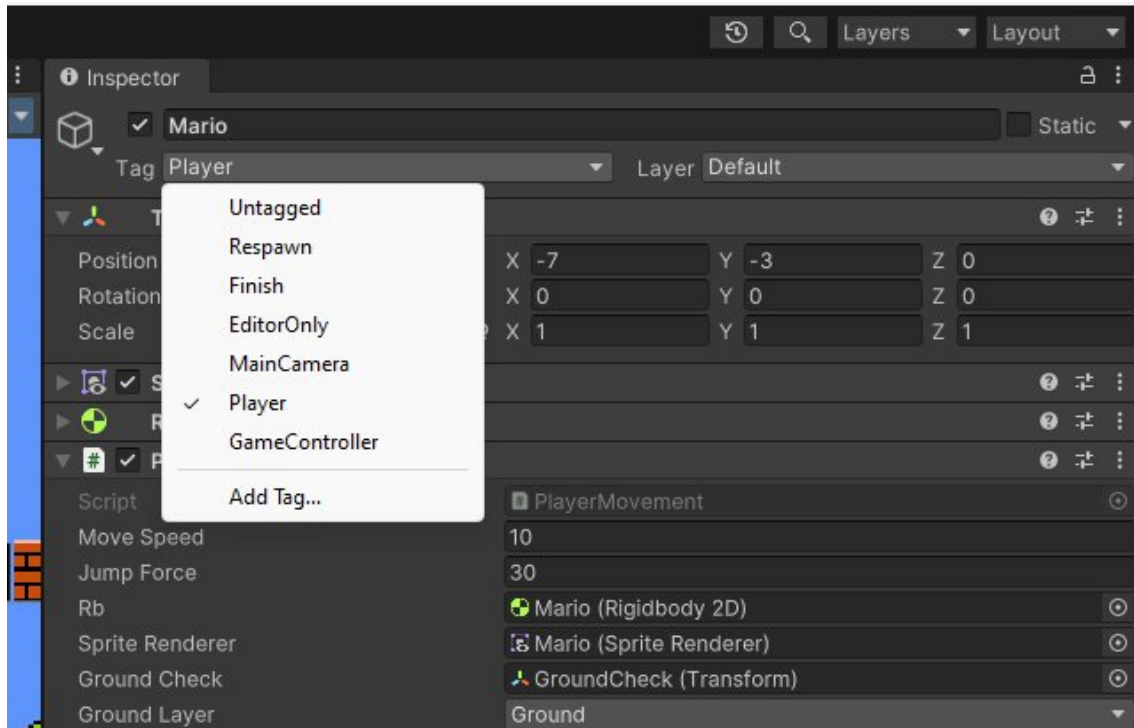
Check en Is trigger



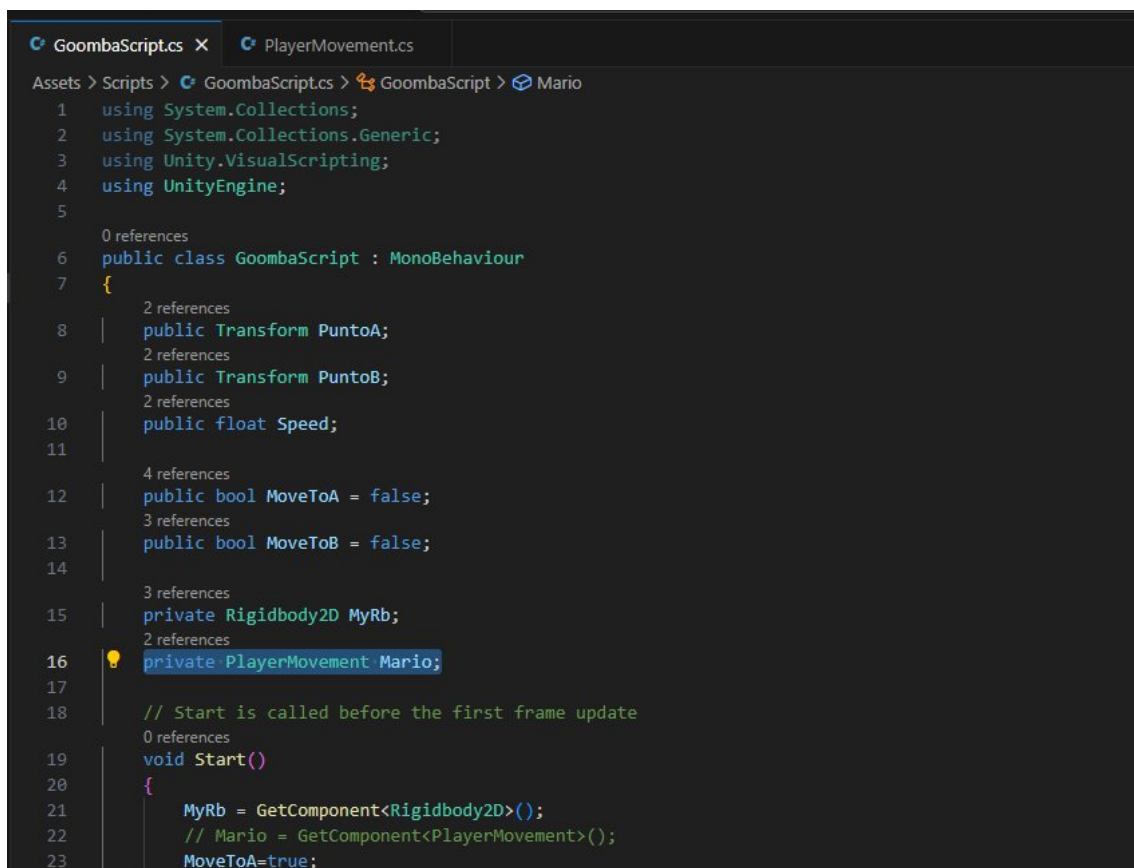
En nuestro Script GoombaScript agregamos el método ontrigger:

```
private void OnTriggerEnter2D(Collider2D collision) // Unity invoca automáticamente cuando
un objeto con un collider entra en contacto con otro
{
    if(collision.gameObject.tag == "Player") // Si la colision que entra al trigger es
player entonces
    {
        Destroy(gameObject); // se destruye el gameObject
    }
}
```

Seleccionando a mario en la jerarquía cambiamos el Tag a Player



Ahora tenemos que agregar que cuando Mario colisione con la cabeza del Goomba este haga un salto de rebote, dentro de la condicional obtenemos el objeto de PlayerMovement para luego hacer True el método Hit de Mario provocando que salte



Hacemos uso del método `OnTriggerEnter2D` que unity invoca automáticamente cuando un collider entra en contacto con otro

```
GoombaScript.cs | PlayerMovement.cs
Assets > Scripts > GoombaScriptcs > GoombaScript
6 public class GoombaScript : MonoBehaviour
19 void Start()
22 // Mario = GetComponent<PlayerMovement>();
23 MoveToA = true;
24 }
25
26 // Update is called once per frame
27 void Update()
28 {
29     if (MoveToA)
30     {
31         MyRb.transform.position = Vector2.MoveTowards(transform.position, PuntoA.position, Speed * Time.deltaTime);
32
33         if (transform.position == PuntoA.position)
34         {
35             MoveToA = false;
36             MoveToB = true;
37         }
38     }
39
40     if (MoveToB)
41     {
42         MyRb.transform.position = Vector2.MoveTowards(transform.position, PuntoB.position, Speed * Time.deltaTime);
43
44         if (transform.position == PuntoB.position)
45         {
46             MoveToA = true;
47             MoveToB = false;
48         }
49     }
50 }
51
52 // References
53 private void OnTriggerEnter2D(Collider2D collision) // Unity invoca automáticamente cuando un objeto con un collider entra en contacto con otro
54 {
55     if (collision.gameObject.tag == "Player") // Si la colision que entra al trigger es player entonces
56     {
57         Mario = collision.gameObject.GetComponent<PlayerMovement>(); // Obtener el componente PlayerMovement desde el objeto que colisiona (Mario)
58         Mario.Hit = true; // Hacer que Mario rebote al caer sobre el Goomba
59         Destroy(gameObject); // se destruye el gameObject
60     }
61 }
62
```

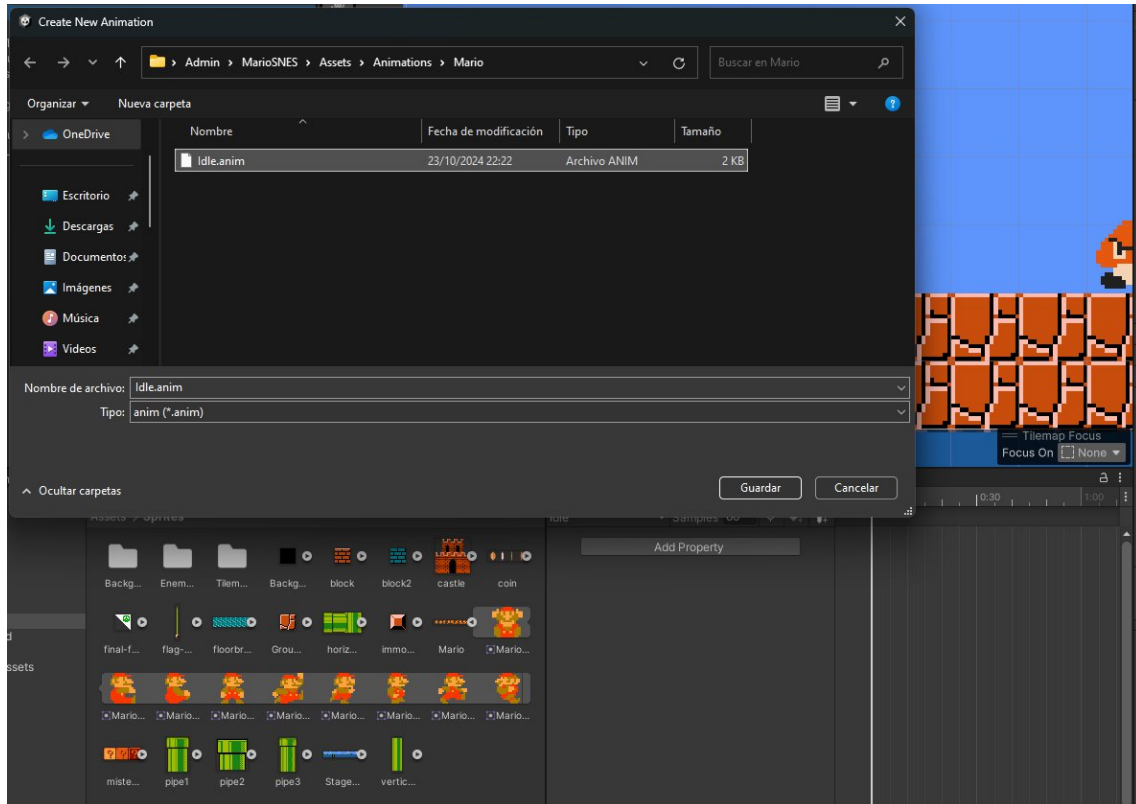
```
private void OnTriggerEnter2D(Collider2D collision) // Unity invoca automáticamente cuando un
objeto con un collider entra en contacto con otro
{
    if(collision.gameObject.tag == "Player") // Si la colision que entra al trigger es
player entonces
    {
        Mario = collision.gameObject.GetComponent<PlayerMovement>(); // Obtener el
componente PlayerMovement desde el objeto que colisiona (Mario)
        Mario.Hit = true; // Hacer que Mario rebote al caer sobre el Goomba
        Destroy(gameObject); // se destruye el gameObject
    }
}
```

En nuestro Script PlayerMovement agregamos **public bool Hit=false;**

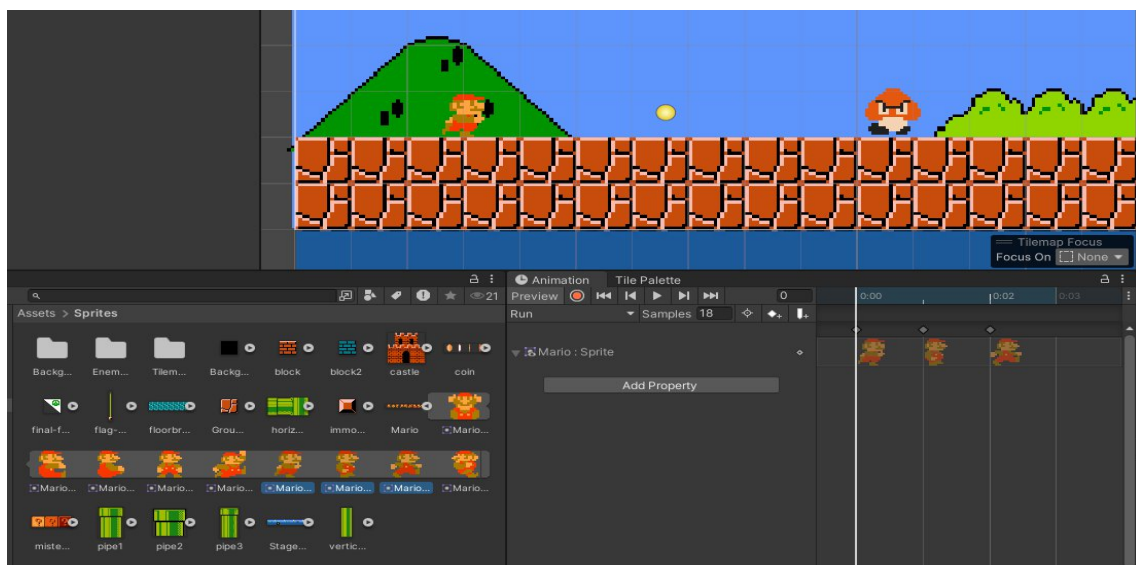
```
Assets > Scripts > PlayerMovement.cs > PlayerMovement > Hit
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  2 references
6  public class PlayerMovement : MonoBehaviour
7  {
8      1 reference
9      public float moveSpeed = 5f;    // Velocidad de movimiento ajustable
10     2 references
11     public float jumpForce = 10f;    // Fuerza de salto ajustable
12     6 references
13     public Rigidbody2D rb;            // Referencia al componente Rigidbody2D
14     2 references
15     public SpriteRenderer spriteRenderer; // Para cambiar la dirección del sprite
16     1 reference
17     public Transform groundCheck;     // Verificación de si Mario está en el suelo
18     1 reference
19     public LayerMask groundLayer;     // Capa que define el suelo
20     4 references
21     private Vector2 movement;         // Vector para almacenar el movimiento del jugador
22     1 reference
23     [SerializeField] private bool isGrounded; // Variable para ver el estado si mario esta tocando o no el suelo
24     3 references
25     public bool isCrouching = false;  // Verificar si Mario está agachado
26     3 references
27     private bool isJumping = false;   // Verificar si Mario está saltando
28     3 references
29     public bool Hit=false;
30
31     0 references
32     void Update()
33     {
34         // Solo permitir movimiento en el eje horizontal
35         movement.x = Input.GetAxis("Horizontal");
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622

```

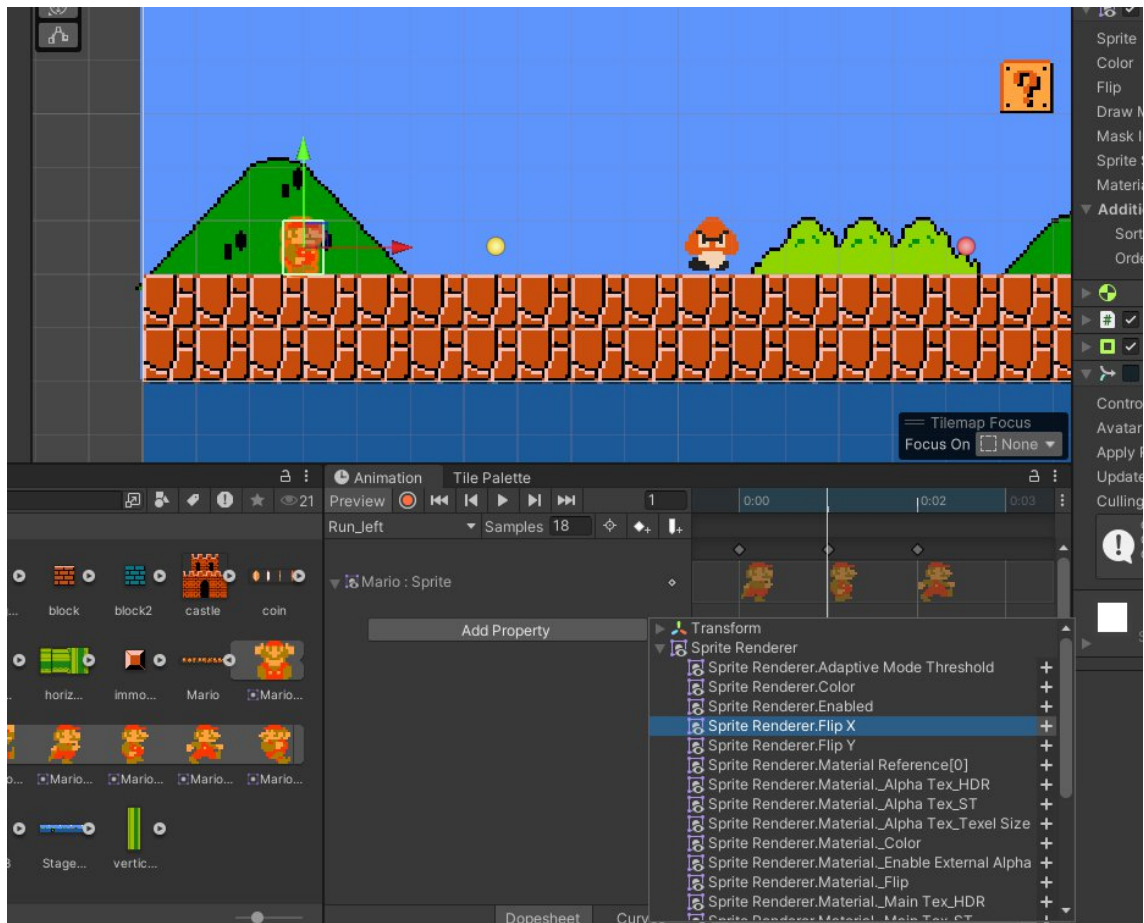
Creamos una nueva animación mientras seleccionamos a mario en la jerarquía



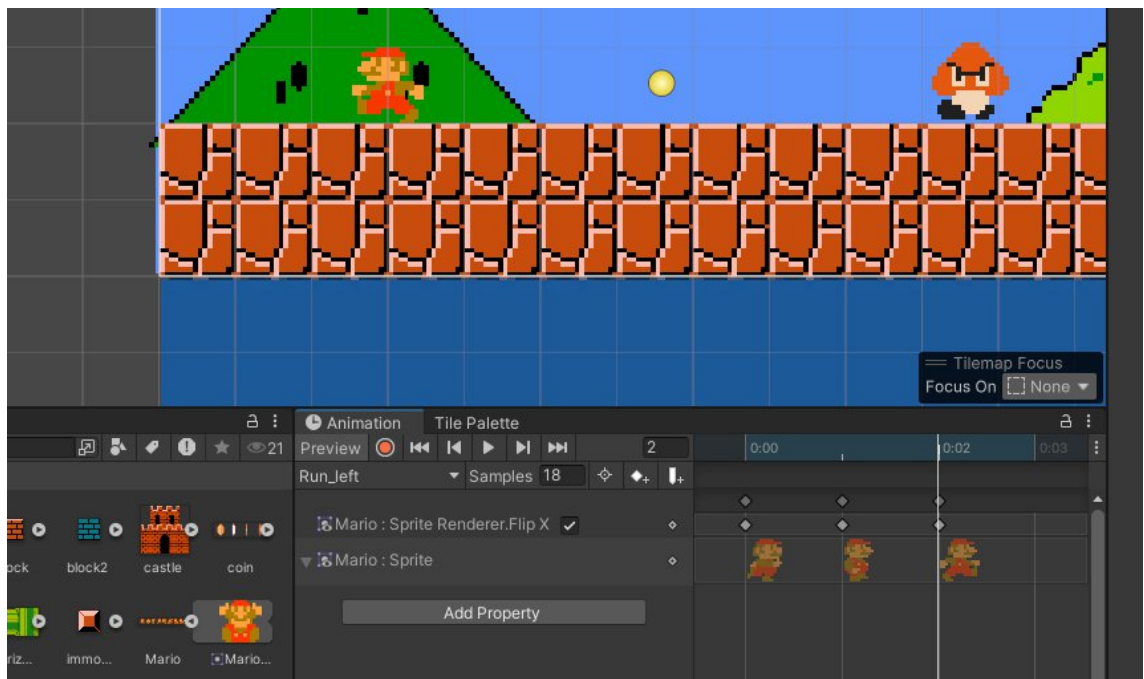
Añadimos otra animación con nombre "Run" y agregamos mario1,2 y 3 arrastrandolos al animator



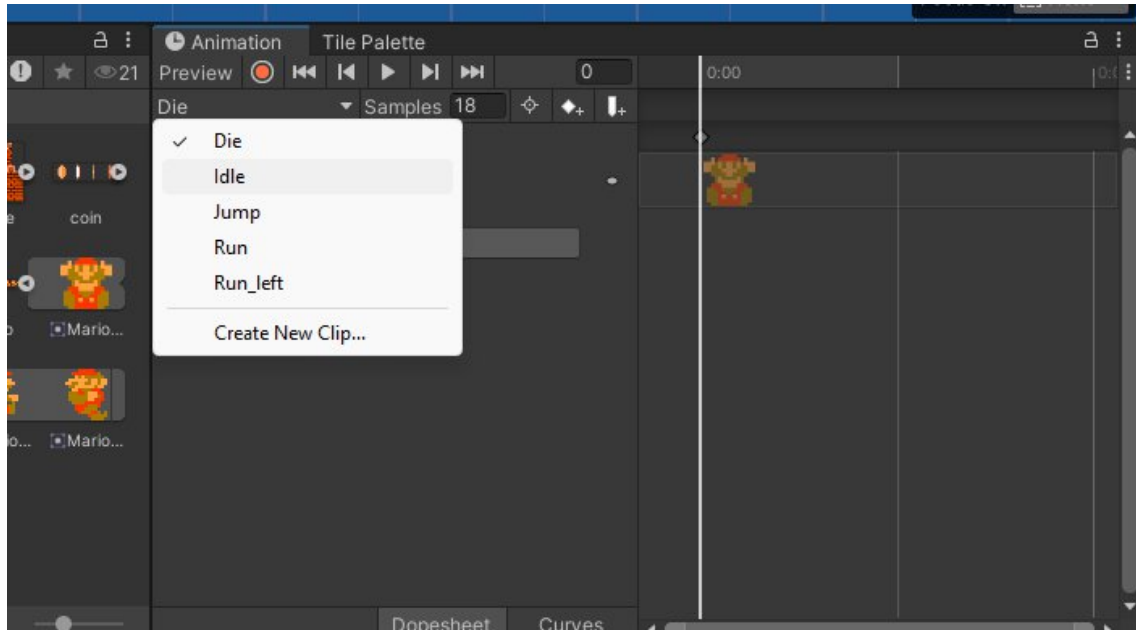
Agregamos un Sprite Renderer.Flíp X en la animación de Run_left (correr izquierda)



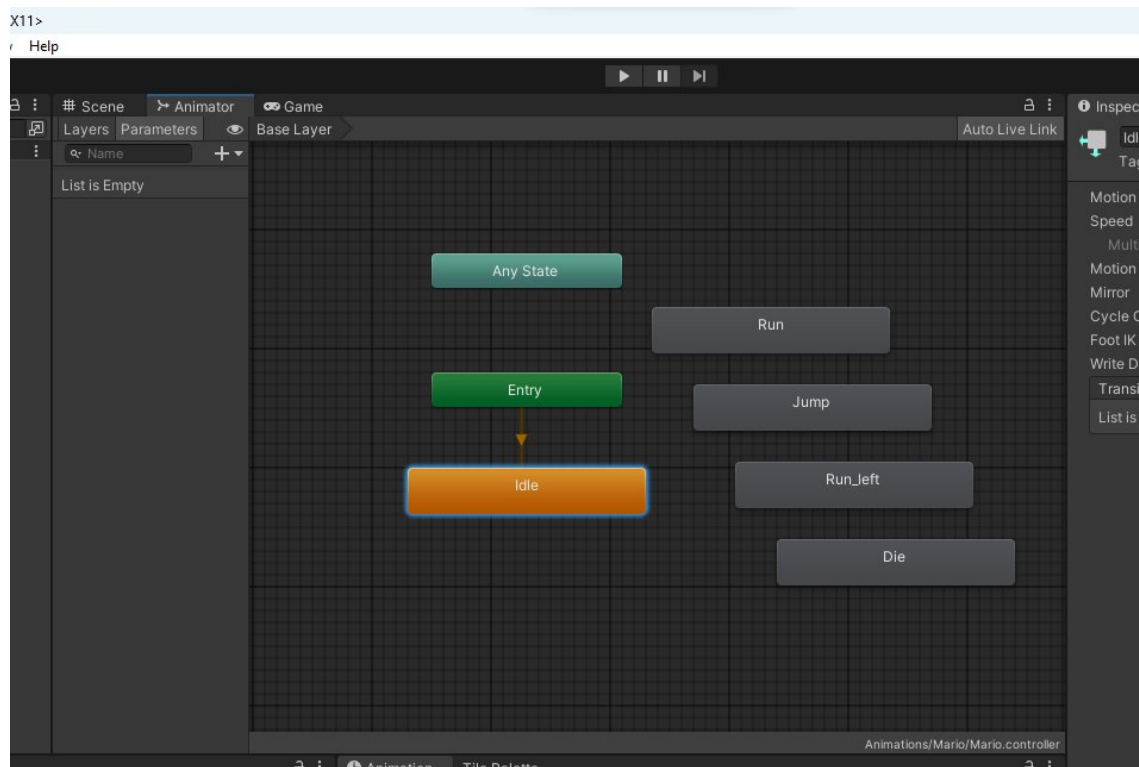
Damos Check en sprite renderer para que invierta el sprite cuando corre hacia la izquierda



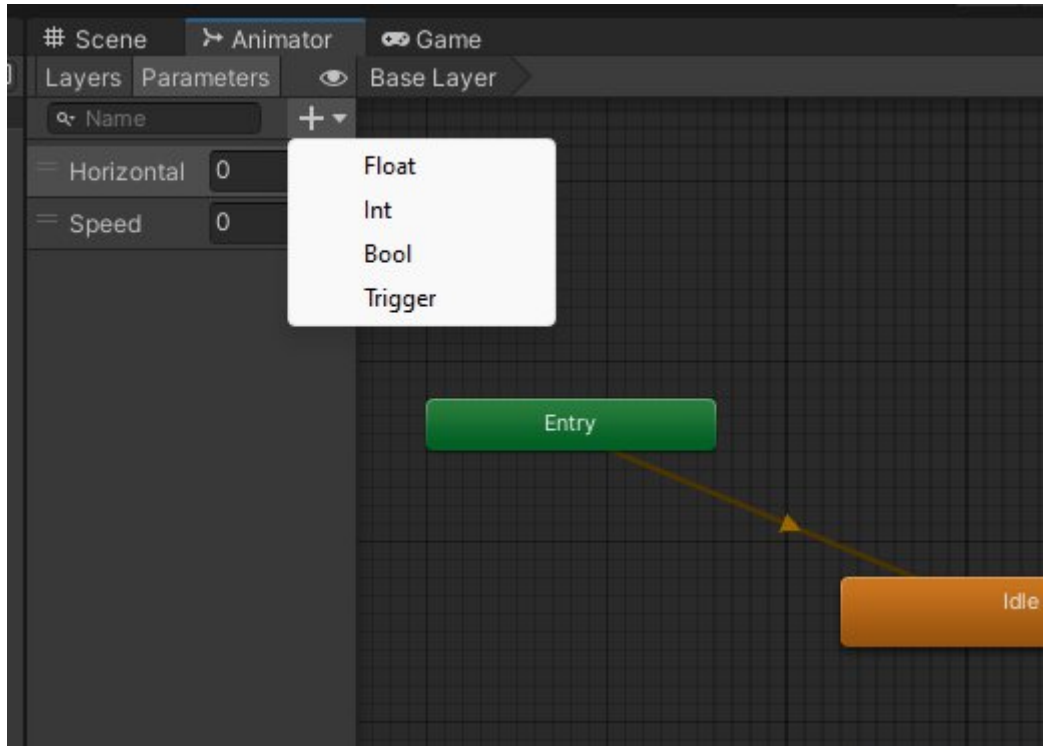
Tendríamos las siguientes animaciones creadas con sus respectivos sprites



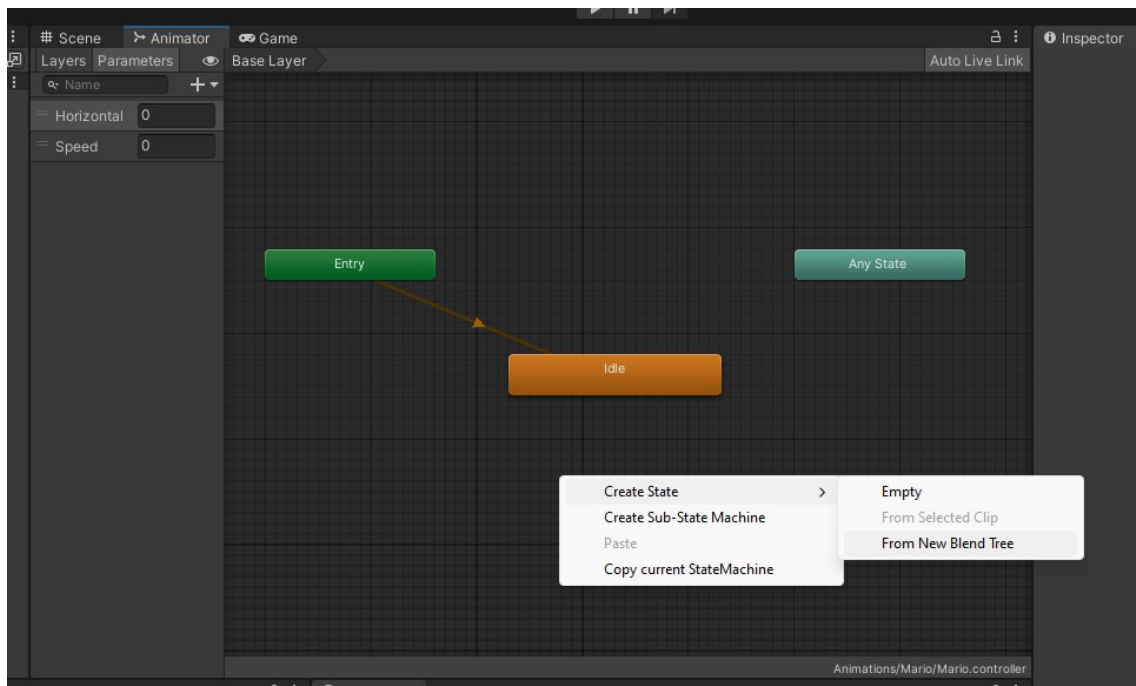
Seleccionamos la pestaña “Animator” y encontraremos todas las animaciones que creamos



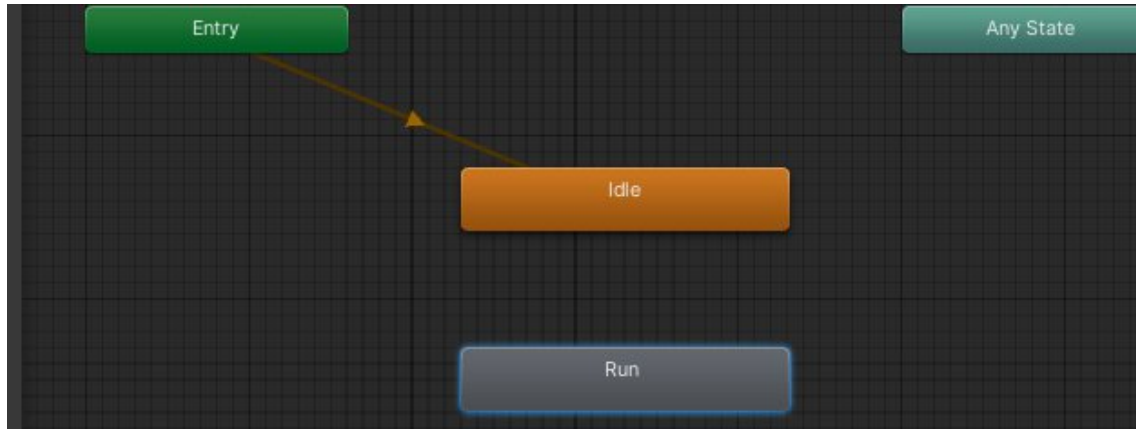
En la animator en la pestaña parámetros añadimos dos parámetros flotat horizontal y vertical



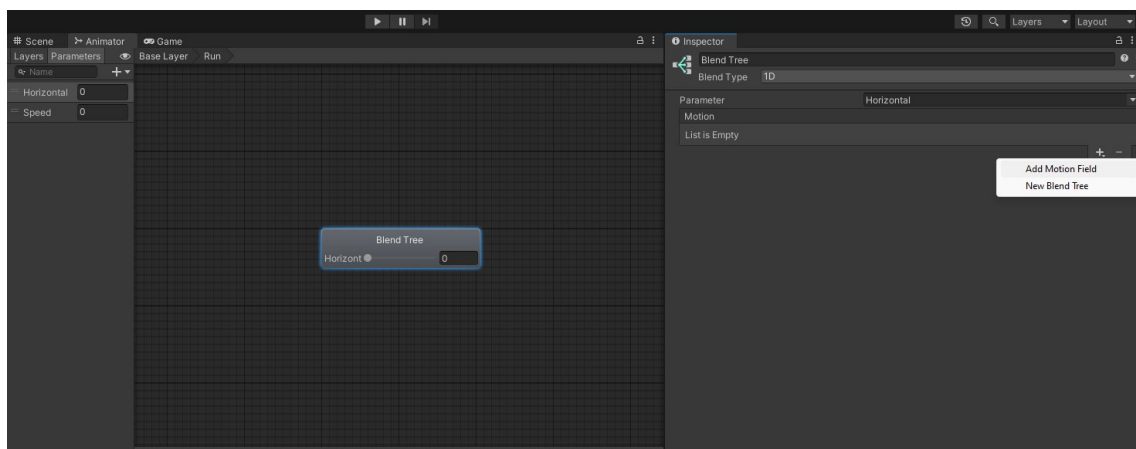
Dentro de Animator agregamos un blend tree y eliminamos lo demas



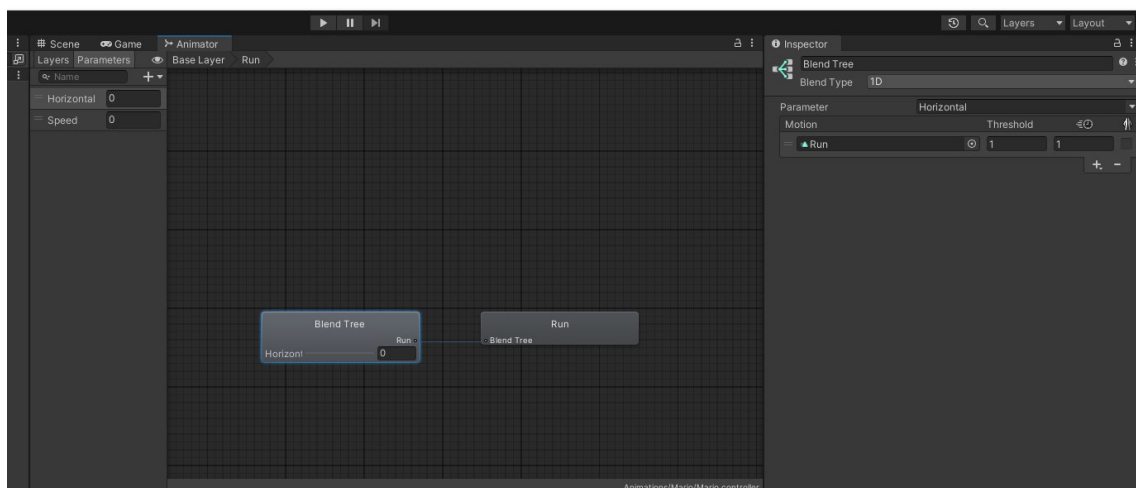
Renombramos como Run y Doble clic



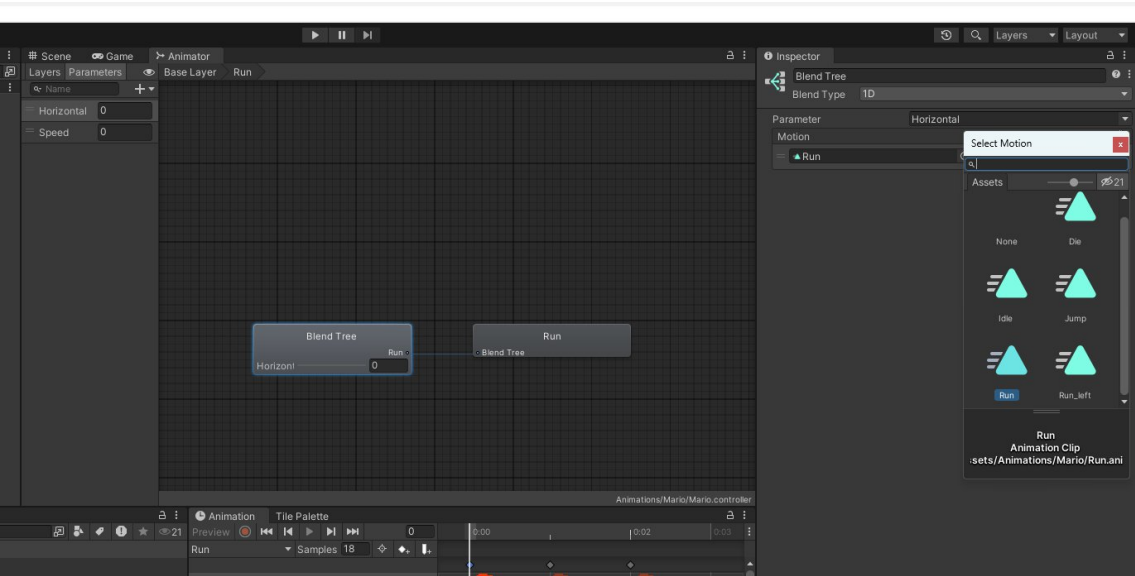
Dentro agregamos un Add Motion Field



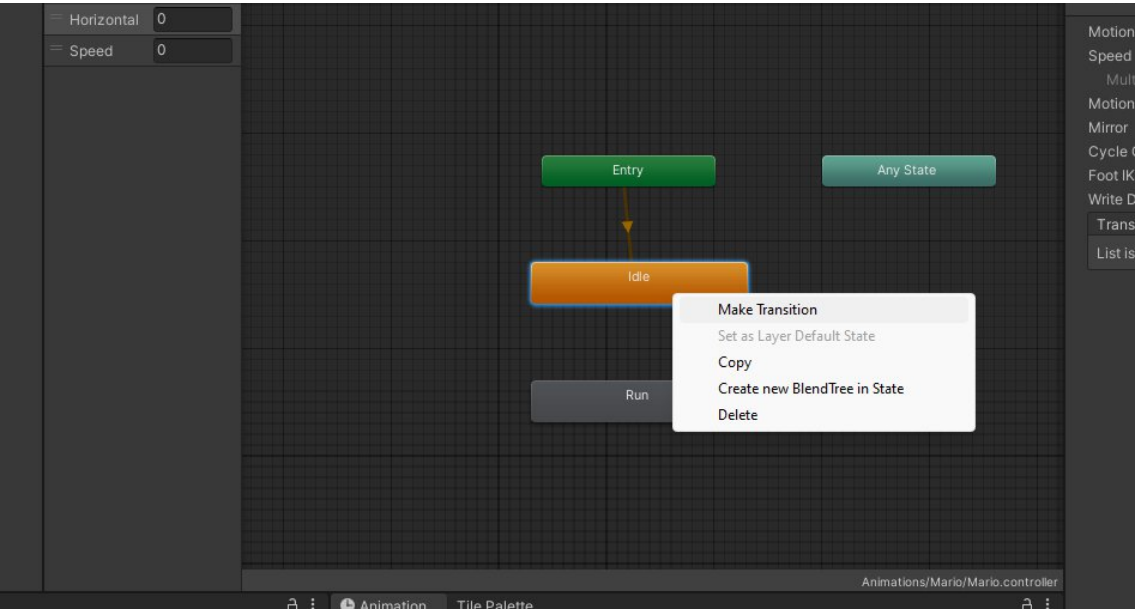
Agregamos Renombramos a Run



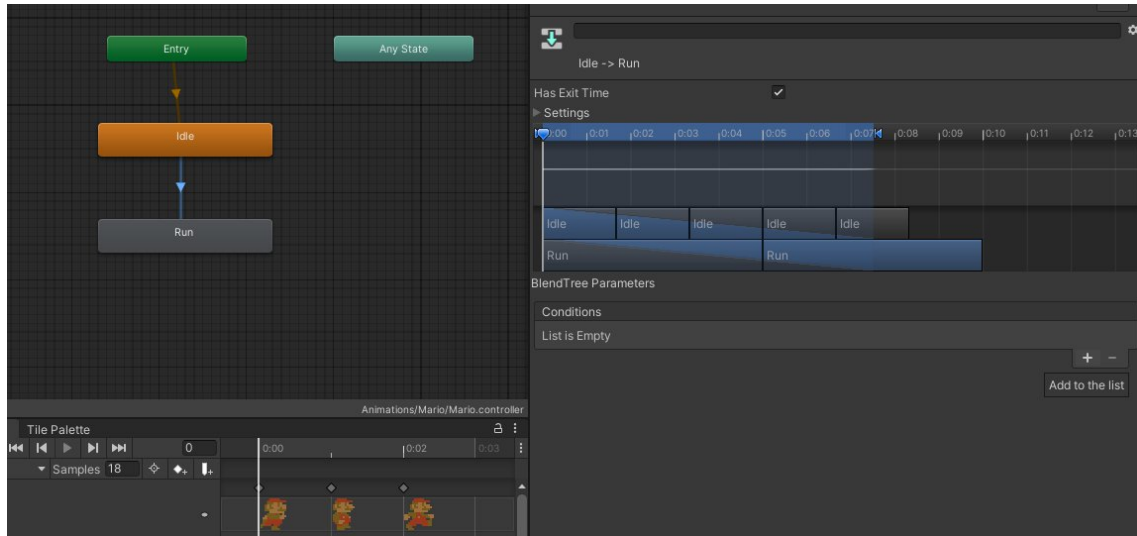
En Motion Seleccionamos la animación Run



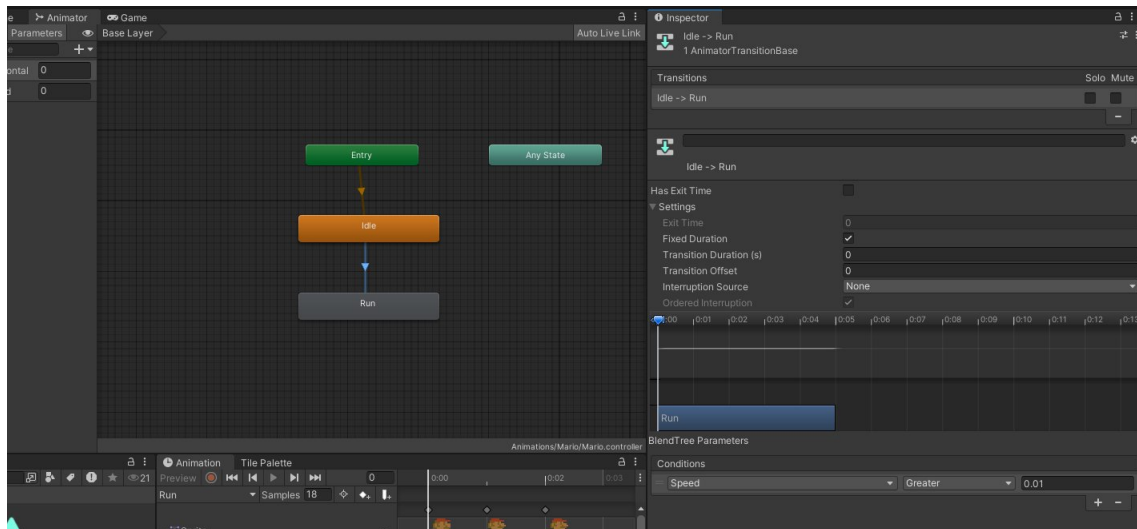
Entramos a Base Layers y en idle agregar transición hacia run



Clic en la flecha y añadimos una condicion



Configuramos a 0 la transicion y desactivamos has exit time, en condicion Greater 0.01



Agregamos private Animator playerAnimator; a nuestro Script PlayerMovement.cs

Tambien inicializamos el componente en Void Start()

```

Assets > Scripts > PlayerMovement.cs > PlayerMovement
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  2 references
6  public class PlayerMovement : MonoBehaviour
7  {
8      1 reference
9      public float moveSpeed = 5f; // Velocidad de movimiento ajustable
10     2 references
11     public float jumpForce = 10f; // Fuerza de salto ajustable
12     6 references
13     public Rigidbody2D rb; // Referencia al componente Rigidbody2D
14     2 references
15     public SpriteRenderer spriteRenderer; // Para cambiar la dirección del sprite
16     1 reference
17     public Transform groundCheck; // Verificación de si Mario está en el suelo
18     1 reference
19     public LayerMask groundLayer; // Capa que define el suelo
20     3 references
21     private Animator playerAnimator;
22     6 references
23     private Vector2 movement; // Vector para almacenar el movimiento del jugador
24     1 reference
25     [SerializeField] private bool isGrounded; // Variable para ver el estado si mario esta tocando o no el suelo
26     3 references
27     public bool isCrouching = false; // Verificar si Mario está agachado
28     3 references
29     private bool isJumping = false; // Verificar si Mario está saltando
30     3 references
31     public bool Hit=false;
32     0 references
33     void Start()
34     {
35         playerAnimator = GetComponent<Animator>();
36     }
37     0 references
38     void Update()

```

Tendriamos nuestro Script PlayerMovement.cs

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f; // Velocidad de movimiento ajustable
    public float jumpForce = 10f; // Fuerza de salto ajustable
    public Rigidbody2D rb; // Referencia al componente Rigidbody2D
    public SpriteRenderer spriteRenderer; // Para cambiar la dirección del sprite
    public Transform groundCheck; // Verificación de si Mario está en el suelo
    public LayerMask groundLayer; // Capa que define el suelo
    private Animator playerAnimator;
    private Vector2 movement; // Vector para almacenar el movimiento del
jugador
    [SerializeField] private bool isGrounded; // Variable para ver el estado si mario
esta tocando o no el suelo
    public bool isCrouching = false; // Verificar si Mario está agachado
    private bool isJumping = false; // Verificar si Mario está saltando
    public bool Hit=false;
    void Start()

```

```

{
    playerAnimator = GetComponent<Animator>();
}
void Update()
{
    // Solo permitir movimiento en el eje horizontal
    movement.x = Input.GetAxis("Horizontal");

    playerAnimator.SetFloat("Horizontal", movement.x);
    playerAnimator.SetFloat("Speed", movement.sqrMagnitude);

    // Cambiar la dirección del sprite según el movimiento
    if (movement.x > 0) // Si Mario se mueve a la derecha
    {
        spriteRenderer.flipX = false; // No voltear el sprite
    }
    else if (movement.x < 0) // Si Mario se mueve a la izquierda
    {
        spriteRenderer.flipX = true; // Voltear el sprite
    }

    // Si está en el suelo y el usuario presiona hacia abajo, Mario se agacha
    if (IsGrounded() && Input.GetAxis("Vertical") < 0)
    {
        isCrouching = true;
    }
    else
    {
        isCrouching = false;
    }

    // Detectar si el jugador quiere saltar y está en el suelo
    if (Input.GetButtonDown("Jump") && IsGrounded())
    {
        isJumping = true;
    }
    if(Hit)
    {
        rb.velocity = new Vector2(rb.velocity.x, jumpForce); // Aplicar el
salto directamente a la velocidad
        Hit = false;
    }

    isGrounded = IsGrounded();
}

void FixedUpdate()
{
    // Mover el personaje solo si no está agachado
    if (!isCrouching)

```

```

    {
        rb.velocity = new Vector2(movement.x * moveSpeed, rb.velocity.y);
    }

    // Aplicar el salto si se ha detectado el input y el personaje está en el
    suelo
    if (isJumping)
    {
        rb.velocity = new Vector2(rb.velocity.x, jumpForce); // Aplicar el salto
        directamente a la velocidad
        isJumping = false; // Evitar que salte de nuevo hasta que vuelva a tocar
        el suelo
    }
}

// Verificar si Mario está en el suelo
private bool IsGrounded()
{
    // Verificar si hay colisión en el área de GroundCheck
    bool grounded = Physics2D.OverlapCircle(groundCheck.position, 0.2f,
    groundLayer);

    // Mostrar en la consola si Mario está en el suelo o no
    Debug.Log("IsGrounded: " + grounded);

    return grounded;
}
}

```

Con eso tendríamos Animador nuestro Mario usando Blend Tree para gestionar la animacion

